NAVAIR Public Release 11-1510 & 2013-576
Distribution Statement A –"Approved for public release; distribution is unlimited"

Open Group Standard

Technical Standard for Future Airborne Capability Environment (FACE™) Edition 1.1



You have a choice: you can either create your own future, or you can become the victim of a future that someone else creates for you. By seizing the transformation opportunities, you are seizing the opportunity to create your own future.

Vice Admiral (ret.) Arthur K. Cebrowski



Copyright © 2013, The Open Group.

The Open Group hereby authorizes you to use this document for any purpose, PROVIDED THAT any copy of this document, or any part thereof, which you make shall retain all copyright and other proprietary notices contained herein.

This document may contain other proprietary notices and copyright information.

Nothing contained herein shall be construed as conferring by implication, estoppel, or otherwise any license or right under any patent or trademark of The Open Group or any third party. Except as expressly provided above, nothing contained herein shall be construed as conferring any license or right under any copyright of The Open Group.

Note that any product, process, or technology in this document may be the subject of other intellectual property rights reserved by The Open Group, and may not be licensed hereunder.

This document is provided "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NON-INFRINGEMENT. Some jurisdictions do not allow the exclusion of implied warranties, so the above exclusion may not apply to you.

Any publication of The Open Group may include technical inaccuracies or typographical errors. Changes may be periodically made to these publications; these changes will be incorporated in new editions of these publications. The Open Group may make improvements and/or changes in the products and/or the programs described in these publications at any time without notice.

Should any viewer of this document respond with information including feedback data, such as questions, comments, suggestions, or the like regarding the content of this document, such information shall be deemed to be non-confidential and The Open Group shall have no obligation of any kind with respect to such information and shall be free to reproduce, use, disclose, and distribute the information to others without limitation. Further, The Open Group shall be free to use any ideas, concepts, know-how, or techniques contained in such information for any purpose whatsoever including but not limited to developing, manufacturing, and marketing products incorporating such information.

If you did not obtain this copy through The Open Group, it may not be the latest version. For your convenience, the latest version of this publication may be downloaded at www.opengroup.org/bookstore.

Technical Standard for Future Airborne Capability Environment (FACE™), Edition 1.1

ISBN: 1-937218-40-9

Document Number: C13J

Published by The Open Group, October 2013.

Comments relating to the material contained in this document may be submitted to:

The Open Group, 8 New England Executive Park, Burlington, MA 01803, United States

or by electronic mail to:

ogface-admin@opengroup.org

Contents

1 Introduction 1
- 1.1 Background 1
- 1.2 FACE Approach 1
- 1.3 Scope of this Document 2
- 1.4 Requirements Terminology 3

2 Architectural Overview 4
- 2.1 The FACE Strategy 4
  - 2.1.1 Portable Applications 4
- 2.2 Barriers to Application Portability 4
  - 2.2.1 Tight-Coupling 4
  - 2.2.2 The Packaging Problem 7
- 2.3 Interface Categories 10
  - 2.3.1 Vertical Interfaces 10
  - 2.3.2 Horizontal Interfaces 10
- 2.4 Permitted and Prohibited Vertical Interfaces 11
  - 2.4.1 Cases for FACE Applications 12
- 2.5 FACE Architectural Segments 14
  - 2.5.1 Introduction to the Architectural Purpose of the FACE Segments 14
  - 2.5.2 Architectural Segment Example 16
- 2.6 Operating System Partitioning 18
- 2.7 FACE Profiles 19
  - 2.7.1 The Security Profile 19
  - 2.7.2 The Safety Profile 20
  - 2.7.3 The General Purpose Profile 20

3 FACE Software Computing Architecture 22
- 3.1 Operating System Segment 22
  - 3.1.1 Operating System Segment Requirements 23
- 3.2 Device Drivers 26
- 3.3 I/O Services Segment 26
  - 3.3.1 I/O Services Segment Requirements 27
  - 3.3.2 I/O Services Segment Definitions 27
  - 3.3.3 I/O Service Management Capability Requirements 28
  - 3.3.4 I/O Data Movement Capability Requirements 28
  - 3.3.5 Device Driver Normalization Adaptation Requirements 29
- 3.4 I/O Interface 29
  - 3.4.1 I/O Services API and Messages 29
  - 3.4.2 I/O Services API Requirements 31
- 3.5 Platform-Specific Services Segment 31

3.5.1 PSS Segment External Interfaces 32
3.5.2 Platform Device Services 33
3.5.3 Platform Common Services 33
3.5.4 Graphics Services Sub-Segment 33
3.5.5 Distributed PSS Components 34
3.5.6 PSS Segment Requirements 34
3.6 Transport Interface 35
3.6.1 Transport Protocol Requirements 35
3.6.2 API Services Requirements 35
3.6.3 API Data Model Requirements 36
3.7 Transport Services Segment 42
3.7.1 Introduction 42
3.7.2 Transport Services Segment Definitions 44
3.7.3 Transport Services Segment Requirements 44
3.7.4 Distribution Capability Requirements 46
3.7.5 Configuration Information Requirements 46
3.7.6 QoS Management Capability Requirements 47
3.7.7 Message Association Management Capability Requirements 47
3.7.8 Data Transformation Capability Requirements 47
3.7.9 Paradigm Translation Capabilities Requirements 48
3.8 Portable Component Segment 48
3.8.1 Definition and Scope of Portable Component Segment 48
3.8.2 Meaning of “Application Portability” 49
3.8.3 Libraries, Run-Times, and Application Frameworks 49
3.8.4 Unit of Portability (UoP) 51
3.8.5 FACE Application Interfaces to Other FACE Entities 56
3.8.6 FACE Application Interfaces to Non-FACE Entities 57
3.8.7 Application Configuration 58
3.8.8 Security Requirements 58
3.8.9 Safety Requirements 58
3.8.10 Application Bundling 58
3.8.11 Deliverable Requirements 59
3.8.12 Logging 60
3.8.13 Graphics 60
3.8.14 User Input Devices 60
3.9 Operating System Interface: FACE Operating System API Standards 60
3.9.1 FACE Security Operating System API 62
3.9.2 FACE Safety Operating System API 63
3.9.3 FACE General Purpose Operating System API 65
3.9.4 Time Management 66
3.10 Logging Services 66
3.10.1 Centralized Logging 66
3.10.2 Local Logging 67
3.11 Graphics Services 67
3.11.1 OpenGL – 2D and 3D Imaging 67
3.11.2 ARINC 739-1 and 739A-1 70
3.11.3 ARINC 661-4 71

3.11.4 Relationship to FACE Architecture....72
3.12 Configuration Services....73
3.12.1 Centralized Configuration Service....75
3.12.2 Common Configuration Schema Requirements....76
3.13 Health Monitoring and Fault Management....76
3.13.1 Introduction....77
3.13.2 HMFM Requirements....78
3.13.3 Operating System Segment Health Monitoring Requirements....78
3.14 Internet Standards Description....79

4 Security....81
4.1 Scope....81
4.2 Guiding Concepts....82
4.2.1 Separation and Partitioning of Security Functions....82
4.2.2 Common Criteria....83
4.3 FACE Security Profile....84
4.4 FACE Security Boundary....85

5 Safety Assurance....87
5.1 Scope....87
5.2 Guiding Concepts....88
5.2.1 Separation and Partitioning of Safety Functions....88
5.2.2 Process....88
5.3 FACE Safety Profile....89

6 Conformance....91

A Operating System POSIX Interface....92
A.1 POSIX API Rules....132

B I/O Interface API....134
B.1 Introduction....134
B.2 I/O Services API and Message Definitions....134
B.3 Language-Specific Data Type Definitions....138
B.4 Open Function....139
B.5 Register Function....139
B.6 Read Function....140
B.7 Write Function....140
B.8 Get Status Function....141
B.9 Close Function....141
B.10 I/O Interface API Message Set....142
B.11 I/O Services Segment Configuration File....147

C Transport Interface API....149
C.1 Introduction....149
C.2 Data Types....149

C.3 C Data Types 153
C.4 C++ Data Types 156
C.5 Create Connection 158
C.6 Destroy Connection 164
C.7 Receive Message 164
C.8 Send Message 166
C.9 Register Read Callback 167
C.10 Get Connection Parameters 168
C.11 Function Calls Restricted to the Transport Services Segment 169
C.12 Transport Services Segment Configuration File 174

D Internet Standards 175
D.1 Network Protocol Requirements Summary 175

Glossary 180

Annex A: Conformance Verification Matrix (CVM) Notes 186

Table of Figures

Figure 1: Portable FACE Applications 4
Figure 2: Traditional Application Coupling 6
Figure 3: FACE Application Coupling 7
Figure 4: Platform-Unique Interfaces 8
Figure 5: Traditional Use of Platform-Unique Interfaces 8
Figure 6: Traditional Problem of Application Portability 9
Figure 7: Application Portability with FACE Computing Environment 9
Figure 8: Vertical Interfaces 10
Figure 9: Horizontal Interfaces 11
Figure 10: Permitted and Prohibited Interfaces 12
Figure 11: Architectural Segments 15
Figure 12: Architectural Segments Example 17
Figure 13: Platform-Specific Segments in Multiple Operating System Partitions 18
Figure 14: FACE Profile Diagram 21
Figure 15: FACE Architectural Diagram 22
Figure 16: Notional FACE System Architecture and Interfaces 23
Figure 17: I/O Services Segment 27
Figure 18: I/O Interface Data Model 30
Figure 19: Notional PSS Segment 32
Figure 20: FACE System Architecture and Interfaces 33
Figure 21: Transport Services API Data Model Definition 37
Figure 22: Transport Services Segment External and Internal Capabilities 43
Figure 23: Platform-Specific Services Segment Example 50
Figure 24: Standardization and Constraint on UoP Interfaces 51
Figure 25: Examples of Application Packaging 52
Figure 26: Inter-UoP & Intra-UoP Communications 56
Figure 27: Platform-Specific Services Segment Example 57
Figure 28: OpenGL Rendering Component Relationship 69
Figure 29: ARINC 739 Component Relationships 71
Figure 30: ARINC 661 Component Relationships 72
Figure 31: Display Components in FACE Architecture Context 73
Figure 32: Fault Detection and Recovery 77
Figure 33: FACE Security Boundary 86
Figure 34: I/O Service Interface Message Definitions 142
Figure 35: MIL-STD-1553 Message Payload Format 145
Figure 36: ARINC 429 Message Payload Format 146
Figure 37: Serial Message Payload Format 146
Figure 38: Discrete Message Payload 146
Figure 39: Transport Networking API Configuration Options 170

Preface

The Open Group

The Open Group is a global consortium that enables the achievement of business objectives through IT standards. With more than 400 member organizations, The Open Group has a diverse membership that spans all sectors of the IT community – customers, systems and solutions suppliers, tool vendors, integrators, and consultants, as well as academics and researchers – to:

- Capture, understand, and address current and emerging requirements, and establish policies and share best practices
- Facilitate interoperability, develop consensus, and evolve and integrate specifications and open source technologies
- Offer a comprehensive set of services to enhance the operational efficiency of consortia
- Operate the industry's premier certification service

Further information on The Open Group is available at www.opengroup.org.

The Open Group publishes a wide range of technical documentation, most of which is focused on development of Open Group Standards and Guides, but which also includes white papers, technical studies, certification and testing documentation, and business titles. Full details and a catalog are available at www.opengroup.org/bookstore.

Readers should note that updates – in the form of Corrigenda – may apply to any publication. This information is published at www.opengroup.org/corrigenda.

This Document

This document is the Technical Standard for the Future Airborne Capability Environment (FACE™) Reference Architecture.

This Edition 1.1 incorporates Technical Corrigendum 1 (Doc. No. U131), which corrects non-normative statements that contain requirements language which are no longer to be considered normative requirements. A note has been inserted after each instance to point to the corresponding clarification of the use of the "requirements" language (shall, will, must) in the CVM table in Annex A.

Trademarks

ArchiMate®, DirecNet®, Jericho Forum®, Making Standards Work®, OpenPegasus®, The Open Group®, TOGAF®, and UNIX® are registered trademarks and Boundaryless Information Flow™, Dependability Through Assuredness™, FACE™, Open Platform 3.0™, and The Open Group Certification Mark™ are trademarks of The Open Group.

Java® is a registered trademark of Oracle and/or its affiliates.

OpenGL® is a registered trademark of Silicon Graphics, Inc. in the United States and/or other countries worldwide.

CORBA®, OMG®, and UML® are registered trademarks and Data-Distribution Service for Real-Time Systems™, DDS™, and Unified Modeling Language™ are trademarks of Object Management Group, Inc. in the United States and/or other countries.

All other brands, company, and product names are used for identification purposes only and may be trademarks that are the sole property of their respective owners.

Acknowledgements

The Open Group gratefully acknowledges the contribution of the following people in the development of this document:

Principal Authors

- Kirk Avery, Lockheed Martin
- Joe Dusio, Rockwell Collins
- Jeff Hegedus, Raytheon
- Patrick Huyck, Green Hills Software
- William Kimmel, NAVAIR
- Joseph Neal, Harris Corporation
- Robert Sweeney, NAVAIR, Technical Working Group (TWG) Chair
- Allan Reynolds, DCS Corporation

Additional Contributors

- William Antypas, Real Time Innovations (RTI)
- Scott Bessler, CMC Electronics
- Mathias Boddicker, DCS Corporation
- Kevin Buesing, Objective Interface Systems
- Judy Cerenzia, The Open Group
- Paul Chen, Wind River
- Gary Gilliland, LynuxWorks
- Charles Stephen Kuehl, Raytheon
- Steve Mills, GoAhead
- Marbert Moore, III, CRL Technologies, Inc.
- Marcell Padilla, CRL Technologies, Inc.
- Bruce Pulliam, Raytheon
- Charlie Rush, Objective Interface Systems

- Joe Schlesselman, Real Time Innovations (RTI)
- Jonathan Strootman, Tucson Embedded Systems
- Terence Thomason, GDAIS
- Jason York, AMRDEC/BAE Systems

With special thanks to W. Mark Vanfleet, NSA.

Referenced Documents

The following normative referenced documents are indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

- ARINC Characteristic 739-1: Multi-Purpose Control and Display Unit (MCDU), June 1990.
- ARINC Characteristic 739A-1: Multi-Purpose Control and Display Unit (MCDU), December 1998.
- ARINC Report 661-4: Cockpit Display System Interfaces to User System, May 2010.
- ARINC Report 664: Aircraft Data Network.
- ARINC Specification 429: Mark 33 Digital Information Transfer System (DITS), May 2004.
- ARINC Specification 653P1-3, Avionics Application Software Standard Interface, Part 1: Required Services, November 2010.
- ARINC Specification 653P2-1, Avionics Application Software Standard Interface, Part 2: Extended Services, December 2008.
- ASN (RD&A): Naval Open Architecture Scope and Responsibilities, Memorandum, August 5, 2004.
- Canadian Trusted Computer Product Evaluation Criteria (CTCPEC), 1993.
- Commercial Internet Protocol Security Option (CIPSO), March 1995.
- Common Criteria (CC), Version 3.1, Release 3, July 2009. (This is an international standard: ISO/IEC 15408: Information Technology – Security Techniques – Evaluation Criteria for IT Security.)
- Department of Defense: Information Assurance Certification and Accreditation (DIACAP) Guidance Document, November 28, 2007.
- Department of Defense: IPv6 Standard Profiles for IPv6-Capable Products, Version 5.0, July 2010.
- Department of Defense: Joint Technical Architecture, Volume I, Version 6, October 2003.
- Department of the Navy (DoN) Chief Information Officer (CIO): DoN XML Developer's Guide, October 2001.
- Department of the Navy (DoN) Chief Information Officer (CIO) Memorandum: DoN Policy on the Use of Extensible Markup Language (XML), December 12, 2002.

- Guidelines and Methods for Conducting the Safety Assessment Process on Civil Airborne Systems and Equipment, ARP 4761, 1996.
- Guidelines for Development of Civil Aircraft and Systems, ARP 4754A, 2010.
- Guidelines for the Use of the C Language in Critical Systems, Motor Industry Software Reliability Association (MISRA), October 2004.
- IEEE Std 1003.1-2008: IEEE Standard for Information Technology – Portable Operating System Interface (POSIX) – Base Specifications, Issue 7, December 1, 2008.
- IEEE Std 1003.13-2003: IEEE Standard for Information Technology – Standardized Application Environment Profile – POSIX – Realtime Application Support, 2004.
- IETF RFC 5424: The Syslog Protocol, March 2009.
- IETF RFC 5570: Common Architecture Label IPv6 Security Option (CALIPSO), July 2009.
- Information Technology Security Evaluation Criteria (ITSEC), Version 1.2, June 1991.
- ISO/IEC 8652:1995/Amd 1:2007: Information Technology – Programming Languages – Ada.
- ISO/IEC 9899:1999: Programming Languages – C.
- ISO/IEC 14882:2003: Programming Languages – C++.
- ISO/IEC 12207:2008: Systems and Software Engineering – Software Life Cycle Processes.
- ISO/IEC 24765:2010: Systems and Software Engineering Vocabulary.
- Java Platform Enterprise Edition (Java EE 6) Specification, Version 6, December 2009.
- Java Platform Standard Edition (Java SE 6) Specification, Version 6, December 2006.
- Modular and Open Avionics Architectures STANAG 4626 (Draft).
- National Information Assurance Certification and Accreditation Process (NIACAP), April 2000.
- NIST SP500-267: A Profile for IPv6 in the US Government, Version 1.0, July 2008.
- OMG CORBA for Embedded Systems (CORBA/e), Version 1.0, November 2008.
- OMG CORBA, Version 3.1, January 2008 (www.omg.org/spec/CORBA).
- OMG: Data Distribution Service (DDS) for Real-Time Systems Specification, Final Adopted Specification, May 2003 (www.omg.org/docs/ptc/03-07-07.pdf).
- OMG: Data Distribution Service (DDS) for Real-Time Systems Specification, Version 1.2, January 2007 (www.omg.org/technology/documents/dds_spec_catalog.htm).
- OMG: OMG Unified Modeling Language (UML) Specification, Version 2.3, May 2010.

- Open Architecture Computing Environment Design Guidance, Version 1.0, August 23, 2004.
- Open Architecture Computing Environment Technologies and Standards, Version 1.0, August 23, 2004.
- OSGi, Version 4.3, April 2011 (www.osgi.org/release4).
- Service Availability Forum (SAF), Application Interface Specification, Release 6, October 2008.
- Software Considerations in Airborne Systems and Equipment Certification, DO-178B, December 1992.
- Trusted Computer System Evaluation Criteria (TCSEC), 1985.
- US Government Protection Profile for Separation Kernels in Environments Requiring High Robustness, Version 1.03, June 29, 2007 (www.niap-ccevs.org/pp/pp_skpp_hr_v1.03).
- XML Schema Definition (XSD), Version 1.0, 2004.

1 Introduction

1.1 Background

Today's Department of Defense (DoD) Airborne systems are typically developed for a unique set of requirements by a single vendor. This form of development has served the DoD well; however, this stovepipe development process has had some undesired side-effects including long lead times, cumbersome improvement processes, lack of hardware and software re-use between various aircraft platforms, which result in a platform-unique design.

The advent of significantly complex mission equipment and electronics systems has caused an increase in the cost and schedule to integrate new hardware and software into aircraft systems. This combined with the extensive testing and airworthy qualification requirements have begun to affect the ability of the DoD aviation community to deploy these new capabilities across the DoD fleet.

The current DoD procurement system does not support the process of hardware and software re-use across different programs. In addition, the current aviation development community has not created sufficient standards to facilitate the re-use of software components across the DoD fleet. Part of the reason for this is the small DoD aviation market. Another part is the difficulty in developing qualified software for aviation. An additional problem is the inability to use current commercial standards because they do not adhere to the stringent safety requirements developed to reduce risk and likelihood of loss of aircraft, reduced mission capability, and ultimately loss of life.

1.2 FACE Approach

The Future Airborne Capability Environment (FACE) was designed as a response to the DoD aviation community's problems described above. The approach used by FACE is to develop a standard for a software computing environment designed to promote software product lines across different air platforms.

Several components comprise the FACE approach to software re-use:

- Business processes to adjust DoD procurement and incentivize industry
- Development process support to promote development of re-usable software components
- A software computing environment standard to promote the development of portable components between differing aviation architectures

The FACE approach allows software-based "capabilities" to be developed as components that are exposed to other software components through defined interfaces. It also provides for the re-

use of software across different hardware computing environments that contain differing platform devices.

FACE will support airworthiness qualification of airborne systems from DO-178B Levels A through E or equivalent qualification requirements. In addition, FACE will support the Common Criteria Evaluation Assurance Level 4 (EAL-4) (threshold) and EAL-7 (objective). *See Note 1*.

Ultimately, the goal of FACE is to reduce development and integration costs and reduce time to field new avionics capabilities.

1.3 Scope of this Document

The following three (3) documents provide the definition and support of the FACE environment:

- The FACE Business Guide
- The FACE Reference Architecture (Standard)
- The FACE Reference Architecture Implementation Guide (future release)

The scope of this document is to define the FACE Reference Architecture used as a standard for developing and verifying infrastructure software components and application components.

All FACE infrastructure implementations will be tested for conformance with the supporting standards and specifications. Applications and services written for FACE will be tested for FACE conformance as defined by the supporting standards and specifications that comprise the infrastructure. FACE is hardware-agnostic. Hardware specificities are abstracted using a Board Support Package (BSP), Chip Support Package (CSP), or other Hardware Abstraction Layer (HAL) as defined by the operating system. Application portability requires that system services, embodied by APIs and software components, should not be hardware-specific. *See Note 1*.

Long lead times, cumbersome improvement processes, lack of re-use, platform-unique design, and extensive testing requirements characterize current DoD aviation software capability. This results in higher costs and the inability to deliver capabilities to the war fighter in a timely manner. To counter these trends, the Naval Aviation Air Combat Electronics program office, enabled by the expertise and experience of the DoD's industrial base, is adopting a revolutionary approach. FACE will enable timely, affordable, cross-platform capability advancements based upon fundamental software engineering principles and pragmatic, practical experience. *See Note 1*.

The goal of FACE is to reduce development and integration costs and reduce time to field new avionics capabilities. FACE establishes a common computing architecture supporting portable, capability-specific software applications across DoD avionics systems.

This document embodies a set of requirements and descriptions referred to as the FACE Standard. The FACE Standard defines software computing environment architectures and interfaces intended for the development of portable applications targeted for general purpose, safety, or security purposes. FACE will use industry standards for distributed communications, programming languages, graphics, operating systems, and other areas as appropriate. *See Note 1*.

The Open Group provides this Technical Standard as well as an accompanying Reference Architecture Implementation Guide. The Conformance Guide will contain the Product Standard that identifies what is tested for conformance certification. *See Note 1*.

This document begins with an architectural overview, followed by detailed description of each architectural segment and interface that outlines requirements and functions of each component of the reference architecture. FACE-specific terms are outlined in the Glossary. Finally, the appendices list the specific interface APIs required for the FACE Standard as well as other applicable standards.

1.4 Requirements Terminology

Can	Describes a feature or behavior that is optional. The presence of this feature should not be relied upon.
May	Describes a feature or behavior that is optional. The presence of this feature should not be relied upon.
Must	Describes a feature or behavior that is mandatory when an implementation that conforms to this document requires that feature or behavior. A software component shall rely on the existence of the feature or behavior within that implementation.
Shall	Describes a feature or behavior that is mandatory for an implementation that conforms to this document. A software component must rely on the existence of the feature or behavior.
Should	Describes a feature or behavior that is strongly recommended for an implementation that conforms to this document. A software component cannot rely on the existence of the feature or behavior.
Will	Same meaning as “shall”; “shall” is the preferred term.

2 Architectural Overview

2.1 The FACE Strategy

2.1.1 Portable Applications

The FACE strategy creates a software environment on the installed computing hardware of DoD aircraft (*aka* platforms) enabling FACE applications to be deployed on different platforms with minimal to no impact to the FACE application.

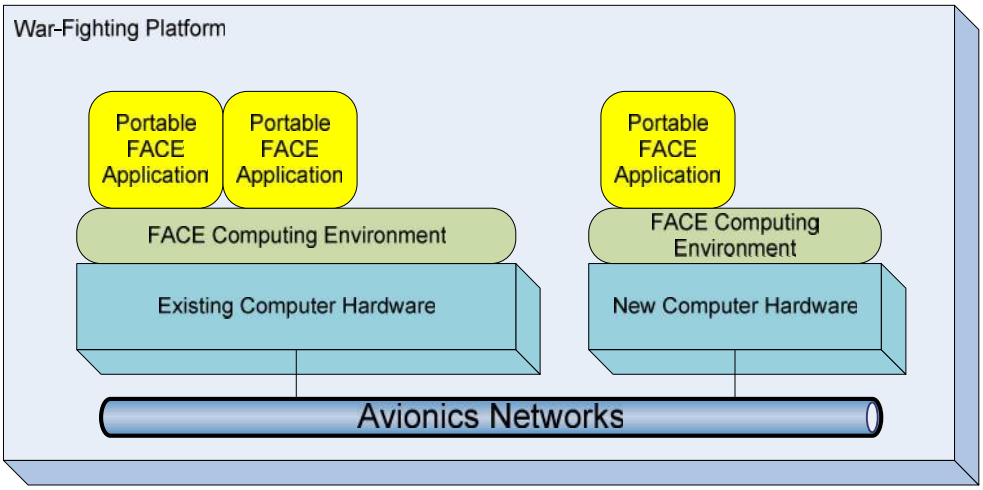


Figure 1: Portable FACE Applications

Provided the software interfaces at the upper edge of the "FACE computing environment" are recreated on computers residing on other platforms, the FACE applications could be redeployed on those other platforms achieving greater application portability.

2.2 Barriers to Application Portability

2.2.1 Tight-Coupling

In computer science, the principle of Separation of Concerns (SoC) is considered a *best practice* because it supports modularity. In short, well-engineered applications implement features as fully encapsulated modules with minimized overlap. An example of such a feature is a software module providing the application with the ability to interface with a specific vendor's MIL-STD-1553 hardware. Such a module would encapsulate all the software necessary to communicate with the hardware. If this hardware was removed, all of the software for that vendor's hardware would also be removed.

The concept of *coupling* refers to the degree of dependency between architectural elements. Coupling between computing platform hardware devices and the software modules communicating with them represent a barrier to application portability. The reason for this barrier to portability is that every platform device has an Interface Control Document (ICD) describing the messages and protocols supported by the hardware. For an application to interface to hardware, it builds in support for those messages and protocols. The message and protocol support in the application is tightly-coupled to the hardware because they depend upon each other to obey the ICD for correct operation.

Tight-coupling is a barrier to application portability because of the prohibitively large amount of work required to modify the application to support different architectural elements. In this case the application's code supporting the existing ICD's messages and protocols are likely to need to be removed and replaced with new code for the new platform device. The amount of work involved to support a new ICD varies, but the need to change application code in order to replace an ICD exists. To minimize the work required to support a new hardware device, the principle of SoC is used to reduce the coupling by isolating the code that implements hardware-specific messages and protocols to a small number of modules.

2.2.1.1 Examples of Tight-Coupling

Figure 2 identifies typical sources (internal and external interfaces) of tight-coupling within an application.

In the center of Figure 2 there is a box labeled *Traditional Application* that is divided into three major sections as per the separation of concerns principle:

- Input/Output (I/O) Concerns
- Business Logic Concerns
- Presentation Concerns

The *I/O Concerns* include capabilities involved with interfacing portable components to platform devices to send/receive data. These concerns are tightly-coupled to specific versions of their hardware/firmware. Changes to the hardware platform devices will drive change into the application(s). Two other examples of tight-coupling are also depicted below – *see Note 1*:

- The interfaces to other co-operating or supporting software applications are often a source of tight-coupling which is a barrier to application portability.
- Traditional applications are compiled for a specific operating system. Since an application compiled for one operating system often will not work on any other operating system, the application's interface to the operating system is another example of tight-coupling, which is a barrier to application portability. *See Note 1*.

The *Business Logic Concerns* include data processing, whether open standards-based or closed-proprietary. The Business Logic code may not know where its data originated or where it will be consumed; it just knows how to process it. These concerns are often tightly-coupled to civilian and military standards for data processing. Tight-coupling here is desirable because these standards represent the application capabilities which are precisely what is needed in their

entirety on the other platforms. There are many examples of this type of processing (e.g., Data Links, Flight Management Functions) and most are standards-based. *See Note 1*.

The *Presentation Concerns* functionality is responsible for human-to-computer interfaces as well as computer-to-computer interfaces. These concerns are often tightly-coupled to hardware and software interfaces such as specific map data, specific mapping programs, and message formats used by remote display clients. Were any of these specifics to change, the *Presentation Concerns* portion of the application would also require change. A few examples of the scope of these interfaces include graphical displays, keyboards, bezel buttons, and message formatting.

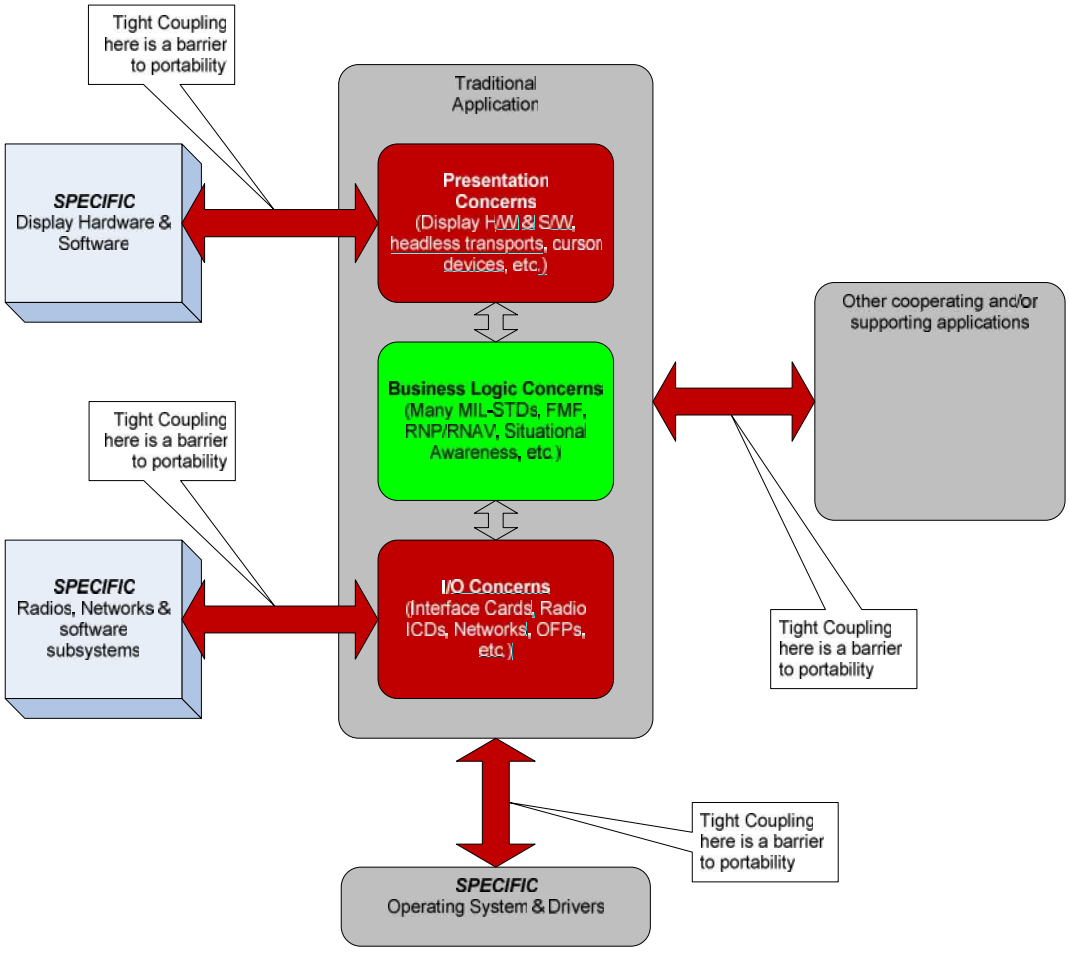


Figure 2: Traditional Application Coupling

2.2.1.2 Addressing Barriers to Application Portability

Interfacing to aircraft platform devices using their own ICDs is not an option due to the tight-coupling this approach introduces. There are two popular strategies to allow for tight-coupling and still mitigate barriers to application portability. The first strategy is to select a standard platform device ICD and have both of the tightly-coupled sides adhere to it. This strategy was not selected by FACE because it would drive changes into most of the products in the vendor base resulting in increased cost. Similarly, selecting a platform device ICD from the vendor base

to be the FACE Standard would drive changes into most of the existing products in the vendor base.

The second strategy, which is one that FACE makes use of, is to move the tightly-coupled concerns out of what is now called the *FACE portable application* and into the *FACE computing environment*. A set of *adapter patterns* would also be added to the FACE computing platform which would handle the variability of platform device ICDs provided by the existing vendor product base on one side and provide a standard FACE-defined ICD on the other side of the adapter pattern. These adapter pattern components are depicted in Figure 3 as the boxes labeled *Adaptation Layer*. Vendor lock-in is avoided and obsolescence management is facilitated by the ability to replace the trio of s*pecific hardware*, the appropriate concerns section (i.e., Presentation or I/O), and the adaptation component. This trio can be replaced without driving change to the portable FACE application.

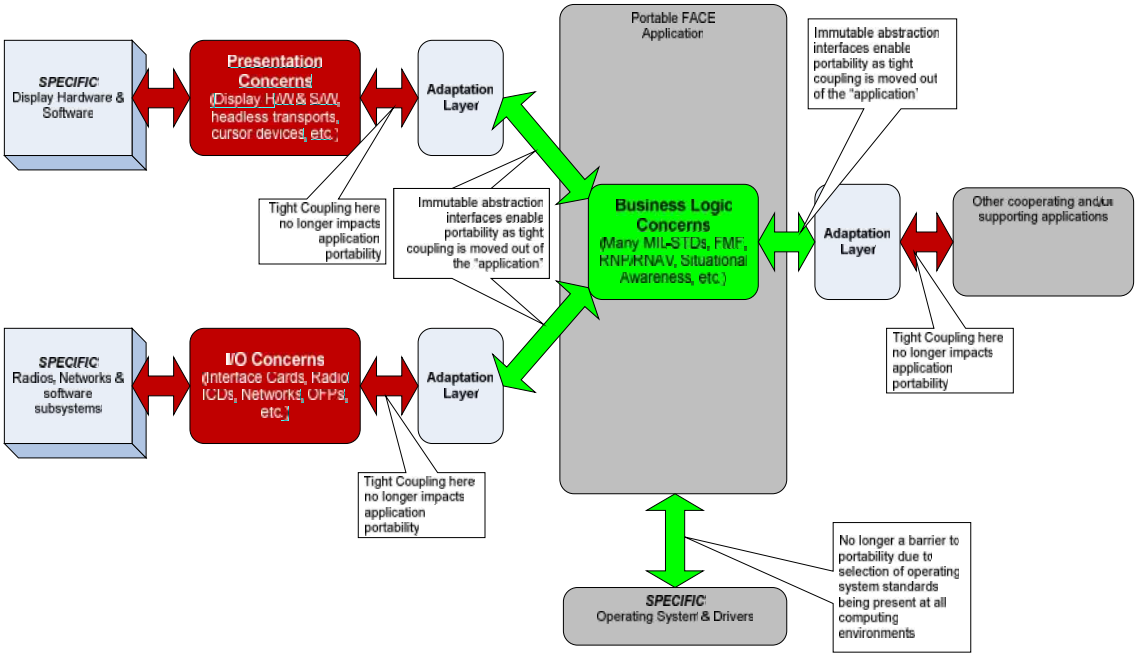


Figure 3: FACE Application Coupling

2.2.2 The Packaging Problem

Another reason that traditional applications are not portable is they are tightly-coupled to a fixed set of interfaces. Consider Figure 4 below, which depicts a notional platform with its set of platform devices as the lower labeled rectangles. Each of these data sources/sinks exposes a device-specific interface as a small square near the top. Each of these platform device-specific interfaces is different, but they all are drawn as the same size square to denote they are part of this Platform X's complement of devices.

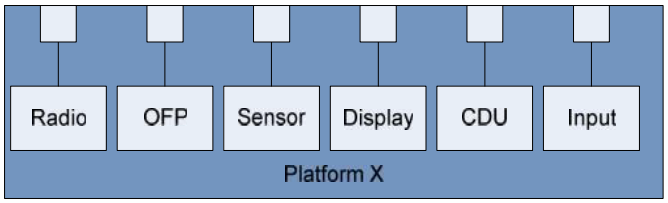


Figure 4: Platform-Unique Interfaces

An application written specifically for this platform would interface with some or all of these interfaces, as shown in Figure 5.

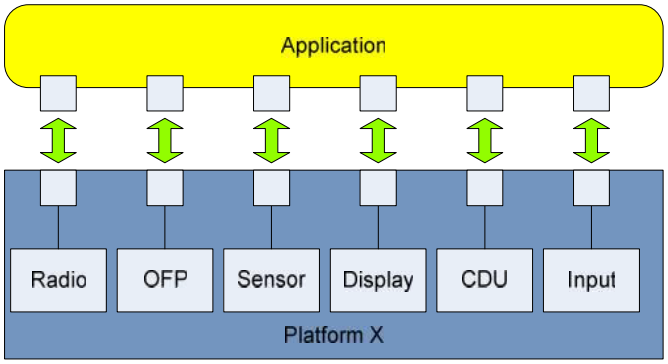


Figure 5: Traditional Use of Platform-Unique Interfaces

This is precisely how some traditional applications have been coded for decades. Support for the ICDs of the platform devices is hard-coded into the application. In Figure 5, the hard-coded ICD support is depicted as squares in the application, which are the same size and shape as those interfaces supplied by the computing platform.

This application will not successfully execute on different computing platforms, even if both computing platforms use the same operating system. This is because the set of interfaces supplied by the computing platforms is not precisely the set of interfaces that was hard-coded into the application. The different shapes shown in Figure 6 below represent different interfaces of the second computing platform, which are not the same interfaces the application expects or requires. *See Note 1*.

Note that the Control Display Unit (CDU) on the second platform (depicted to the right) is precisely the same CDU providing the same interface as the application expects. The application will not function correctly because all the interfaces required by the application are supplied by the computing platform. Simply matching the interface of one hardware device (e.g., CDU) is not sufficient for correct application function. *See Note 1*.

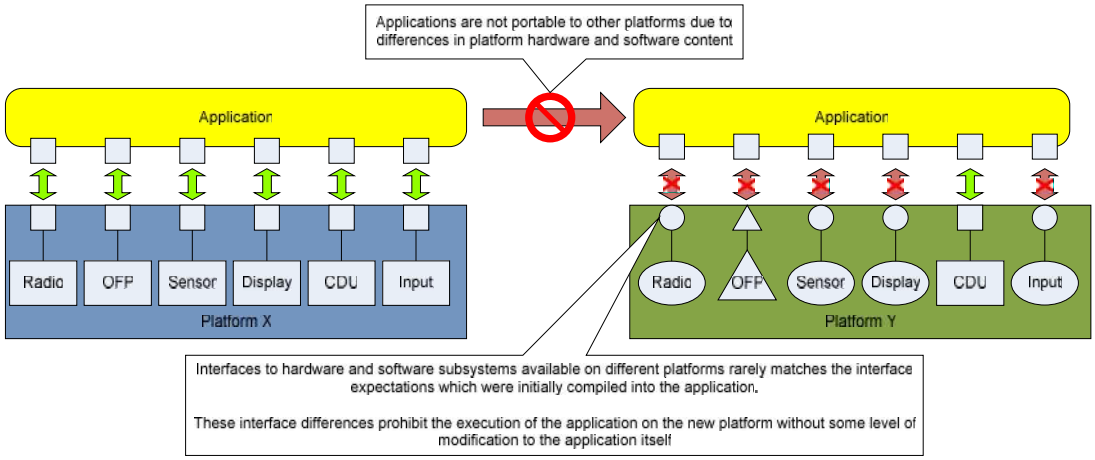


Figure 6: Traditional Problem of Application Portability

2.2.2.1 *Addressing the "Packaging Problem"*

FACE addresses the packaging problem by factoring the tightly-coupled components out of the application and into the FACE computing environment as well as by providing the data sources and sinks of software services, which are also hosted in the FACE computing environment. In addition to refactoring the tightly-coupled components out of the FACE application, the FACE computing environment standardizes the interfaces, as represented by pentagons in Figure 7. These applications can be written to the FACE Standard interfaces. The differences inherent in the computing platforms are abstracted using the FACE Standard. *See Note 1*.

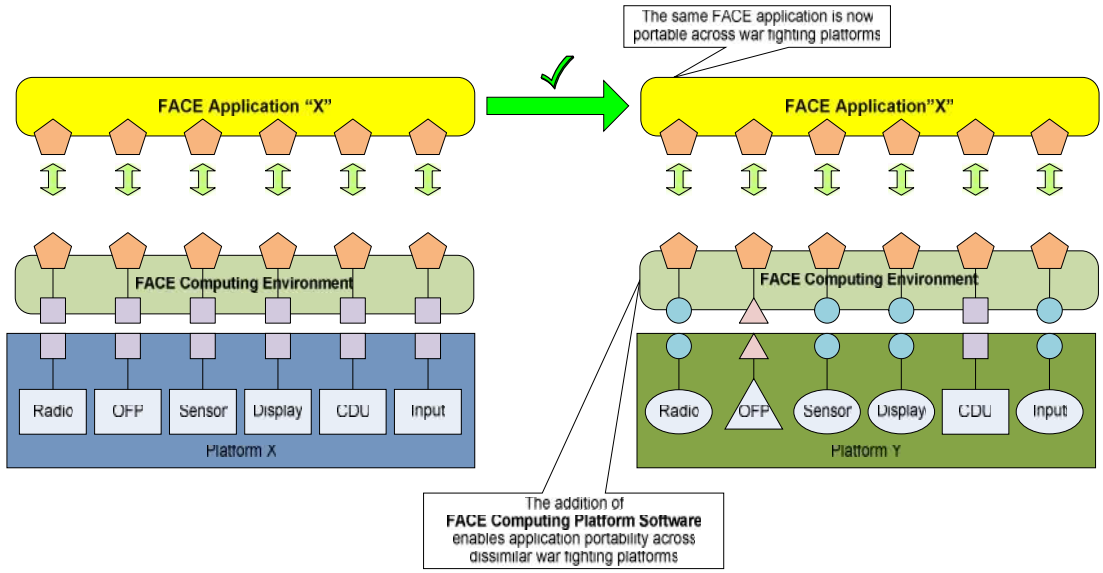


Figure 7: Application Portability with FACE Computing Environment

2.3 Interface Categories

Interfaces are a prime concern in achieving application portability. To facilitate further discussion, the FACE interfaces are grouped into interface categories. There are two primary categories of interfaces: *Vertical Interfaces* and *Horizontal Interfaces*.

2.3.1 Vertical Interfaces

Interfaces are considered to be *vertical* if they support a software layer. There are multiple vertical interfaces. Examples of the vertical interfaces include:

- The interfaces between the operating system *Kernel Space* and all applications executing in what is referred to as *User Space*
- The interfaces between the computing software infrastructure and the application
- The interfaces between the language run-time and the computing software infrastructure

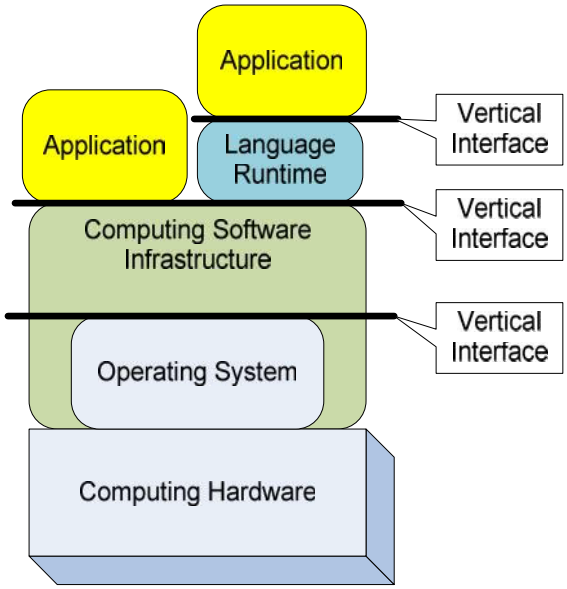


Figure 8: Vertical Interfaces

2.3.2 Horizontal Interfaces

Interfaces are considered to be *horizontal* if they connect applications to other applications, to data sources, or to data sinks. The example figure below depicts horizontal interfaces as black lines running horizontally, vertically, and radially, between the platform devices and the applications, and interfaces between intra-platform applications. Off-board horizontal interfaces are not being addressed during the first edition of the FACE Standard. In this diagram, Local Operational Flight Program (OFP) is synonymous with Platform OFP.

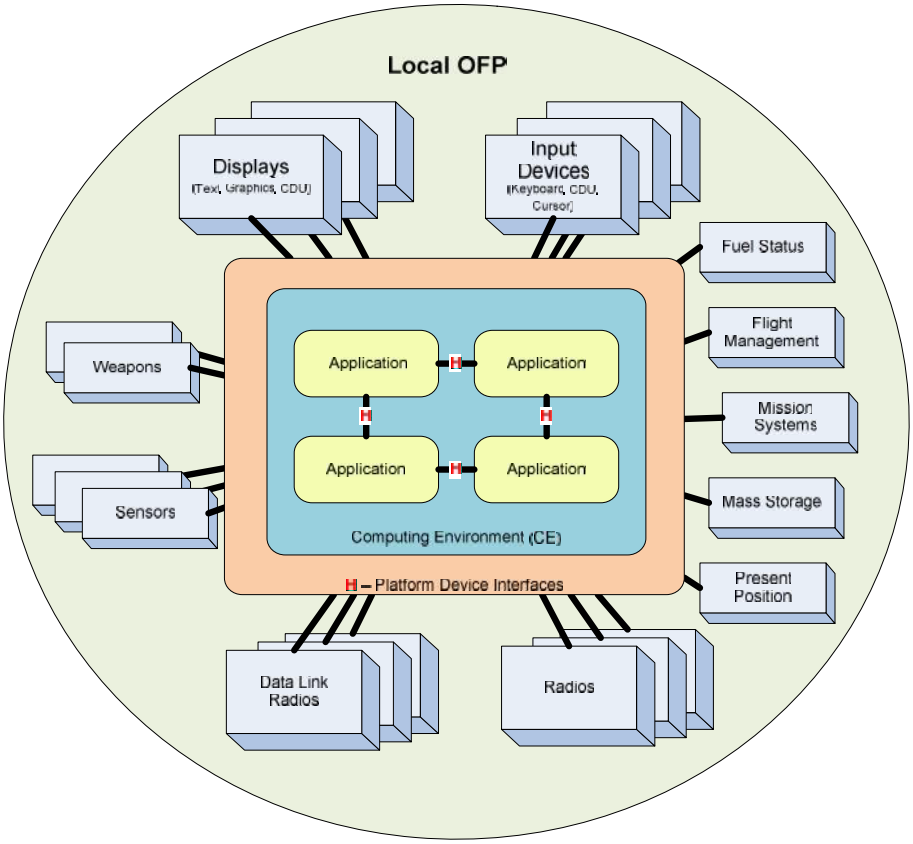


Figure 9: Horizontal Interfaces

2.4 Permitted and Prohibited Vertical Interfaces

Figure 10 depicts eight (8) distinct cases of FACE applications and their vertical interfaces. Four (4) of these cases are prohibited and the remaining four are permitted. The cases are depicted as a vertically-oriented set of components within a rectangle comprised of dashed lines. Each case includes a FACE portable component on the upper side and the operating system at the bottom.

Figure 10 includes yellow boxes labeled *FACE Portable Component*. These are the portable FACE applications referred to above.

Figure 10 also contains green boxes labeled *FACE Infrastructure*. These notional boxes include functionality such as traditional transport middleware, FACE services, and platform-specific processing. This class of software is depicted by a single box only to facilitate the architectural discussion and is not intended to suggest such capabilities are deployed as a monolithic code deliverable.

Most operating systems applicable to the FACE market have additional interfaces (e.g., vendor-specific, additional standards-based interfaces) not directly invoked as a part of a FACE-

conformant application. The large blue box depicted at the bottom of the diagram represents the overall operating system, which includes interfaces not applicable to FACE (and must not be invoked) and other interfaces that are part of the FACE Standard (and can be invoked). Note that for a particular operating system, FACE Standard interfaces may be directly supported by the operating system (natively) or may be indirectly supported using operating system vendor supplied interface libraries operating over the vendor-specific operating system. Either means (or a combination of both) can be used to implement the FACE operating system interfaces. *See Note 2*.

2.4.1 Cases for FACE Applications

Figure 10 depicts different cases (1 through 8), each representing FACE portable components and potential operating system interface usages.

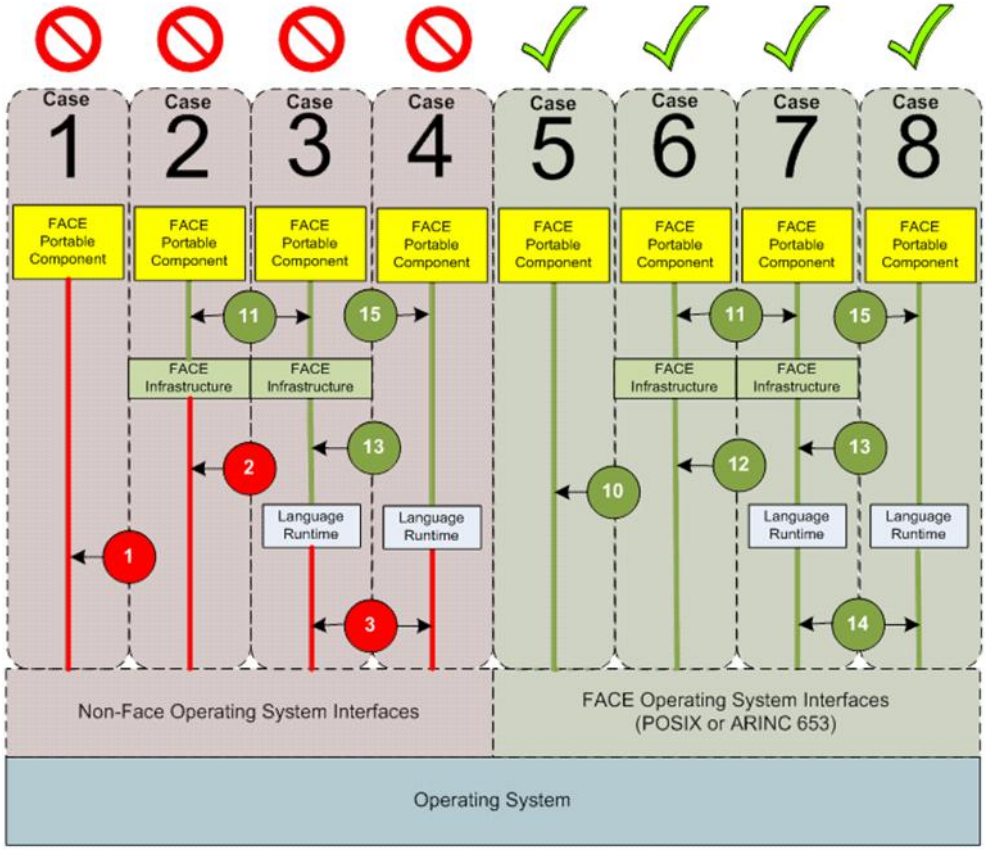


Figure 10: Permitted and Prohibited Interfaces

2.4.1.1 Case 1 (Prohibited)

In this case, a FACE portable component is designed using non-FACE operating system interfaces. The line representing the interface between the FACE portable component and the operating system is labeled with a circle containing the number 1. The interface and its designator are both colored red to denote such interface usage results in a non-portable component.

2.4.1.2 Case 2 (Prohibited)

In this case, a FACE portable component uses Interface 11 to communicate with the FACE infrastructure. The FACE infrastructure uses non-FACE operating system interfaces (Interface 2) to interface to the operating system. Interface 2 is colored red because the use of non-FACE operating system interfaces results in a non-portable infrastructure.

2.4.1.3 Case 3 (Prohibited)

This case has a FACE portable component communicating to the FACE infrastructure through Interface 11. The infrastructure communicates with a language run-time through Interface 13. The language run-time uses non-FACE operating system interfaces through Interface 3. Use of Interface 3 to the non-FACE operating system interface is prohibited, just as with Interfaces 1 and 2 in Cases 1 and 2, respectively. A run-time's use of non-FACE operating system interfaces results in a non-portable run-time.

2.4.1.4 Case 4 (Prohibited)

The FACE portable component in Case 4 is designed to use a language run-time through Interface 15. The language run-time uses non-FACE operating system interfaces through Interface 3. Use of Interface 3 through the non-FACE operating system interface is prohibited. A run-time's use of non-FACE operating system interfaces results in a non-portable run-time.

2.4.1.5 Case 5 (Permissible)

Case 5 has a FACE portable component communicating through the FACE operating system interface, Interface 10. This case is permitted because the FACE portable component interfaces to the operating system through only FACE-conformant interfaces.

2.4.1.6 Case 6 (Permissible)

The FACE portable component in Case 6 communicates with the FACE infrastructure through Interface 11. The FACE infrastructure then uses the FACE operating system interface to communicate with the operating system (Interface 12). This case is permitted because each component communicates through only FACE-conformant interfaces.

2.4.1.7 Case 7 (Permissible)

The FACE portable component in Case 7 communicates with the FACE infrastructure through Interface 11. The infrastructure communicates with a language run-time through Interface 13. The language run-time then uses the FACE operating system interface to communicate with the operating system (Interface 14). This case is permitted because each component communicates through only FACE-conformant interfaces.

2.4.1.8 Case 8 (Permissible)

The FACE portable component in Case 8 communicates with a language run-time through Interface 15. The language run-time uses FACE operating system interfaces through Interface 14. This case is permitted because each component communicates through only FACE-conformant interfaces.

2.5 FACE Architectural Segments

Software architectures for applications that are *point solutions* regularly include units of code that are often called functions, subroutines, classes, or methods. The software architecture identifies the code units, what they do, and how they are interconnected to achieve the desired application-level functionality. In contrast, software architectures for *software product lines* tend to identify two main groups of code units: those that will appear in every variant of the deliverable, and those that are likely to be changed for each version of the deliverable. *See Note 1*.

FACE is a software computing environment that enables product lines for military aviation. FACE provides this software computing environment for every variant of the deliverable to the platform. However, FACE implementations may also select a varying set of applications to supply the desired platform-level capabilities. *See Note 1*.

To use the analogy of an automobile, items such as the frame and the windshield are likely to have precisely the same specifications for all the variants for any given car. However, items such as the motor, wheels, hubcaps, paint, seats, and audio gear are examples of places where there are variances. Similarly, the FACE product line has a fixed structure as well as places where there are variances. This section introduces the structure and those places of variance.

The FACE architecture is comprised of a set of "places" where variance occurs. These points of variance are called "segments". The structure created by connecting these segments together is the beginning of the FACE architecture.

The five (5) segments of the FACE architecture are:

- Operating System Segment
- I/O Services Segment
- Platform-Specific Services Segment
- Transport Services Segment
- Portable Component Segment

Figure 11 identifies the FACE segments and their interconnections.

2.5.1 Introduction to the Architectural Purpose of the FACE Segments

2.5.1.1 Operating System Segment

The Operating System Segment is depicted and considered to be underneath all the other segments and vendor-supplied code. It hosts various operating system interfaces and as such provides the vertical interface to the software in all the other segments.

2.5.1.2 I/O Services Segment

The software in this segment provides a bridge for data from the operating system's device drivers to the Platform-Specific Services (PSS) Segment. The *code modules* in the PSS Segment which model real-world platform devices, such as GPS units, use an I/O Interface such as MIL-

STD-1553 to communicate to the device. This I/O Interface card comes with a driver for the chosen operating system and the FACE I/O service components abstract the communication with that driver. However, there are many I/O Interface card vendors, each of whom provides a driver for the specific models of their cards and each of those software interfaces sets exposed by those drivers will vary from vendor to vendor and operating system to operating system. In an effort to get re-use and portability of FACE components primarily in the PSS Segment, as well as minimize the impact of I/O Interface card obsolescence and the asynchronous vendor updates of the drivers, FACE introduces the I/O Services Segment. The code in this segment is a set of *adapter design patterns* each of which communicates to a vendor-supplied driver (depicted as the lower edge), and then converts that data to a FACE standardized interface (depicted as the upper edge). If card obsolescence or driver changes occur, the *adapter* in this segment would be replaced by a modified device driver. No other changes could be required because the impact of those changes is fully encapsulated by the single adapter. In addition, and maybe more importantly, the drivers potentially use non-FACE Operating System Interfaces and therefore the direct use of the drivers would result in coupling to the operating system. *See Note 1*.

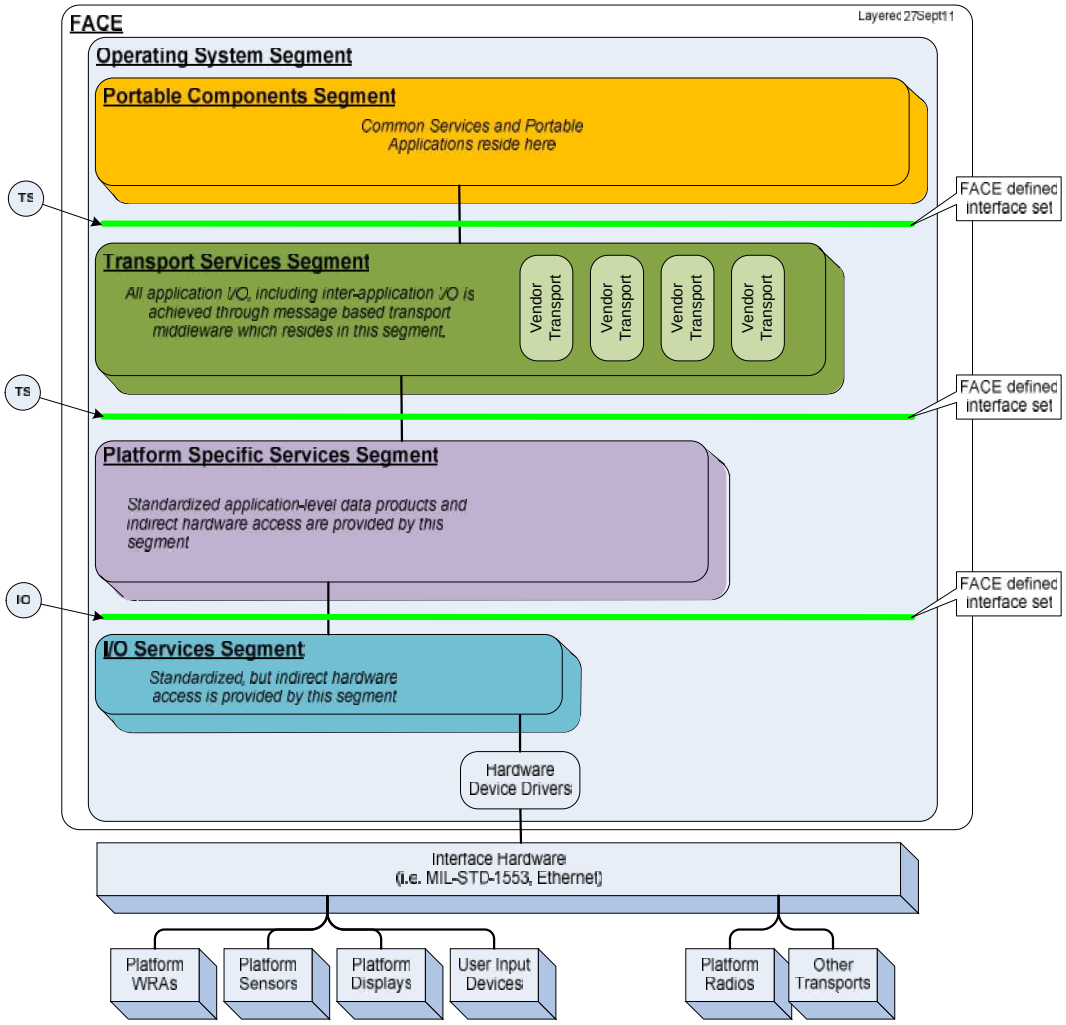


Figure 11: Architectural Segments

2.5.1.3 *Platform-Specific Services Segment*

The Platform-Specific Services (PSS) Segment comprises those portions of software that, in traditional software architectures, are typically tightly-coupled to the platform-specific device ICDs. Reallocating such software to the PSS Segment facilitates creating portable components (Portable Component Segment). The pathway for data between these portable components and the newly reallocated portions of code making up services within the PSS is through the Transport Services Segment. These PSS components will *model* or otherwise encapsulate the business logic of the platform devices connected to the FACE system. The data from these services is used as sources of platform data for the portable components as mentioned above. *See Note 1 and Note 3.*

2.5.1.4 *Transport Services Segment*

This segment moves all data between the Portable Component Segment and the PSS Segment. The following are architectural elements which connect to the Transport Services Segment:

- Portable Component Segment
- Platform-Specific Services Segment
- Operating System Segment

While this structural arrangement resembles familiar architectural patterns (i.e., Broker, Mediator Pattern, Hub, and Spoke), it is not intended to be used exclusively in this manner. The primary purpose of the Transport Services Segment is to provide a means to move data in such a way that the applications need not know or care where or how the remote data end-point is located or implemented. This abstraction of data sources and sinks is required to enable application portability.

2.5.1.5 *Portable Component Segment*

The Portable Component Segment is populated with a set of portable FACE applications and common services, which provide the desired platform-level capabilities when executing together. The FACE applications are made portable by factoring out portions of the application that are tightly-coupled to platform device I/O and Human Machine Interface (HMI) and keeping those portions in the Common Services section of this segment as well as in portions of the PSS Segment. The set of interfaces that connect the Portable Component Segment to the remainder of the FACE architecture are designated as the Transport Services (TS) Segment Interface in Figure 12.

2.5.2 Architectural Segment Example

Figure 12 includes notional examples of the software capabilities that reside in each of the FACE segments. This is a notional diagram intended to clarify the purpose of the segments and does not represent a prescriptive description of elements that are required for FACE conformance. The rules and descriptions of the components that belong in each segment will be defined in Section 3. *See Note 1.*

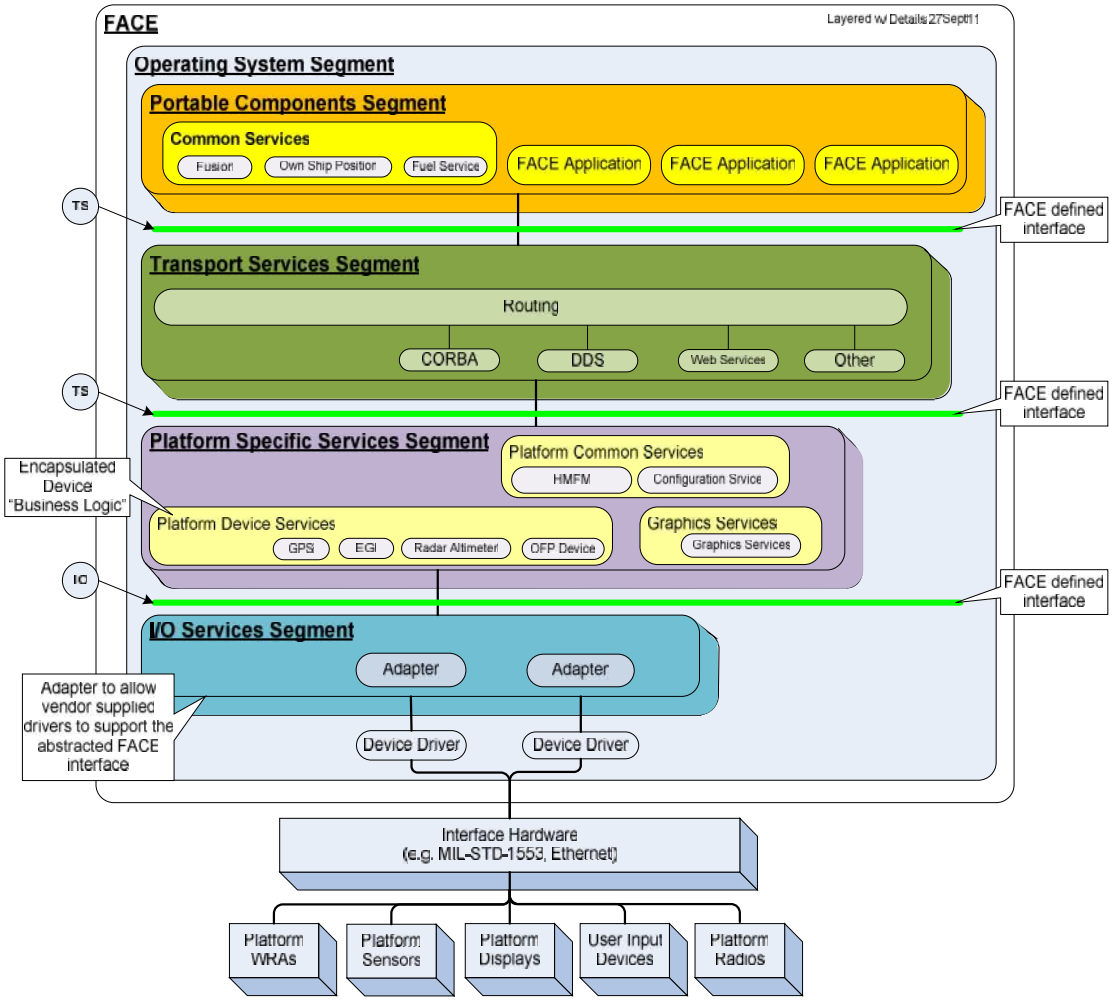


Figure 12: Architectural Segments Example

The Portable Component Segment is populated with two types of portable components: Common Services and FACE applications. All data to and from the Portable Component Segment are routed through the Transport Services Segment.

Common Services is comprised of software components that provide application-level services. To cite two examples, the *Own Ship Position* produces a fix (location in three-dimensional space) and the *Graphics Services* provides a means to deliver computed outputs to a platform=-specific display device.

All data that might be consumed or produced by the applications, including the data supplied from the PSS Segment, is connected to the routing function in the Transport Services Segment as well as any third-party transport mechanisms.

Between the third-party transports (e.g., DDS, CORBA, Web Services) and the routing function are adapter design patterns that provide a consistent data interface to the routing function and allow the vendor base to modify their products without driving change into FACE beyond the

adapter. This approach eliminates the need and associated cost and delay involved with driving FACE interface standards into the vendor base.

The majority of components within the PSS Segment model the physical hardware devices and encapsulate their Interface Control Documents. The examples in the diagram are a Global Positioning System (GPS), an Embedded GPS Inertial Navigation Unit (EGI), a radar altimeter, and access to an unspecified OFP device, perhaps to obtain the platform's fuel status. These specific components reside within the Platform-Specific Device Services Sub-segment. There are also common platform device services that provide Health Monitoring Fault Management (HMFM) and configuration services.

The I/O Services Segment contains software components that abstract the input and output of vendor-supplied device drivers in a FACE architecture. Similar to the approach of the adapters used between the Transport Services Segment's routing function and the third-party transports, these adapters save time, money, and effort by allowing the vendor base to create, modify, and replace their devices drivers in a way that limits the impact of device driver changes to any FACE solution and does not require the vendor base to implement additional FACE interfaces.

2.6 Operating System Partitioning

Some FACE solutions need to execute on a time and space partitioned operating system to achieve goals such as separation of different levels of safety or security, or to just provide isolation between applications (e.g., that were developed by different organizations). FACE conformance is not predicated upon having partitions, merely that the FACE Standard supports this functionality. Deploying a FACE solution across multiple partitions provides opportunities to divide architectural segments into smaller deliverables and to install them in different partitions. A varying number of applications can exist in any one of the partitions. Figure 13 depicts the architectural capabilities of code that reside in the PSS Segment and has been implemented as two deliverables in separate partitions. *See Note 1*.

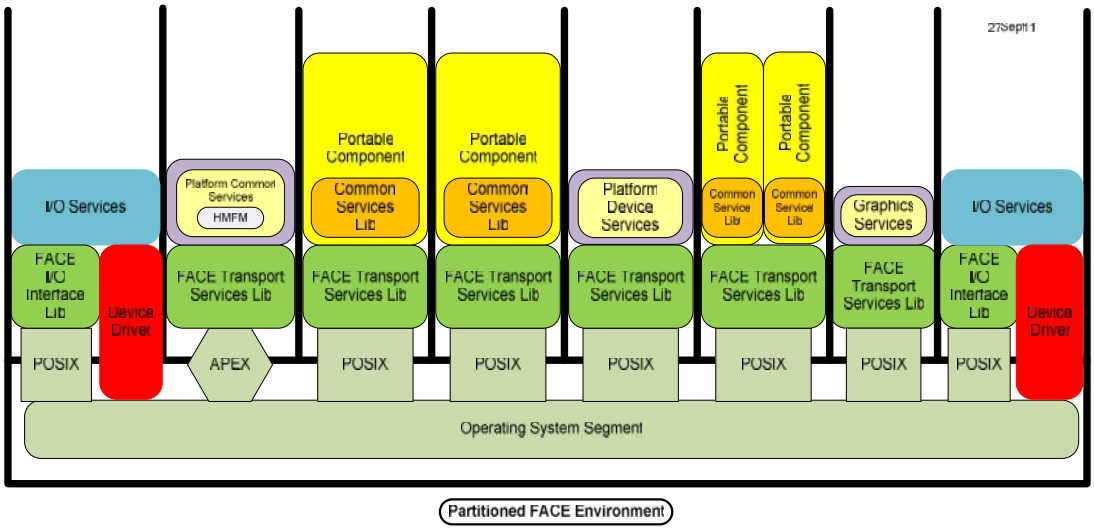


Figure 13: Platform-Specific Segments in Multiple Operating System Partitions

2.7 FACE Profiles

The FACE architecture supports general-purpose, safety-critical, or secure avionics capabilities, or any combination of the three capability types through the use of profiles. Profiles will affect the interfaces and functionalities available within a FACE implementation. *See Note 2.*

A profile within a standard is a subset of a full standard (e.g., POSIX Real-Time Profiles). This tailored or subset version of a full standard has features removed in order to meet additional implementation requirements for performance, safety, or security, or any combination of the three. Profiles within software standards describe the included and excluded functionality for a domain within the larger set of functionality in a full standard. Profiles can address functionality, performance attributes, and safety or security concerns within the full standard. Profiles can have a significant impact on the quality attributes of a software standard. *See Note 1.*

Profiles within the FACE Standard are necessary to handle the differing requirements imposed by security and safety domains of avionics. A full standard might not have implementations available that are amenable to generation of appropriate artifacts needed for safety or security evaluations. Likewise, if a full standard did meet safety and security requirements, then the standard might be too stringent and limit the type of applications and services that use the standard. The security domain imposes the most stringent constraints on the FACE Standard; the safety domain imposes somewhat lesser constraints; and the general-purpose domain imposes the fewest constraints.

To develop applications and services, the profiles referenced within this document will define the number and set of usable APIs. The most restrictive is the Security Profile, which focuses mainly on multiple classification domains, and may also be capable of satisfying the safety requirements. The Safety Profile, which is less restrictive than the Security Profile, focuses on a set of standards and supporting certifications that are sufficient for safety requirements, but are not necessarily sufficient for security requirements. The least restrictive profile is the General Purpose (GP) Profile, which is a larger set of standards that addresses services and applications that do not require stringent safety or security approvals/certifications. *See Note 2.*

The prime contractor, system integrator, application developer, and the customer should specify the requirements for the capability, as well as the corresponding architecture, in order to choose the appropriate profile (or profiles) for developing the capability-driven applications and services. The evaluation should take into account migrating applications and services from one profile to a more stringent profile. The FACE Reference Architecture Implementation Guide provides guidance on combining the software standard profiles with an appropriate architecture. The profile will affect not only the software architecture, but may also affect the hardware used in the system. All parties involved should understand the impact that hardware has and adjust the system design accordingly. *See Note 1.*

2.7.1 The Security Profile

The Security Profile shall constrain the API set available to develop high-assurance security applications and services within a FACE Computing Environment that supports multiple levels of security. Security principles, as discussed in Section 4, are the basis for the Security Profile. Key tenets of this profile include an appropriately certified Multiple Independent Levels of Security (MILS) separation kernel, enabling multiple secure enclaves, coupled with packet-

based labeled data communication paths, providing active enforcement of the approved security policy. Services associated with the Security Profile need to comply with the appropriate security assurance objectives, which are likely to require certification to NSA high robustness or other high-assurance certification or accreditation requirements. The target level of the certification is Common Criteria (ISO/IEC 15408) EAL6 or higher. The Security Profile APIs are a subset of the Safety Profile API. This allows for application portability between profiles. A minimum set of essential APIs, Middleware, Hardware Abstraction Requirements, and MILS National Security Administration (NSA) requirements will define the Security Profile. The US Government Protection Profile for Separation Kernels in Environments Requiring High Robustness, Version 1.03 – known as Separation Kernel Protection Profile (SKPP) – will be the foundation for the FACE Security Profile. The actual evaluation level will be determined by the system-level requirements. *See Note 4.*

2.7.2 The Safety Profile

The Safety Profile will constrain the API set available to develop safety-related applications and services. Safety principles, as discussed in Section 5, are the basis for the Safety Profile. Software applications and components associated with the Safety Profile support safety certifiability. The Safety Profile will include all Security Profile APIs as its baseline. The Safety Profile shall meet real-time requirements and allow for a high level of determinism. The Safety Profile does not mandate nor prohibit the use of time and space partitioning. It is left to the discretion of the System Integrator to determine the appropriate level of independence per capability, subsystem, or system. The Safety Profile is not restricted for use by safety-related applications. It may also be used by general-purpose applications that do not perform a safety function. This allows for increased portability between the Safety and General Purpose Profiles. In order to migrate between the Safety Profile and the Security Profile there is a potential for a significant amount of rework to ensure that the services and applications are certifiable to security requirements. *See Note 5*.

2.7.3 The General Purpose Profile

The FACE General Purpose (GP) Profile API set has been developed for general-purpose avionics applications and services. The GP Profile can meet real-time, deterministic or non-real-time, non-deterministic requirements depending on the requirements of the implemented capability subsystem, or system. The GP Profile does not mandate nor prohibit the use of time and space partitioning. This will be left to the discretion of the System Integrator to determine the appropriate level of independence per capability, subsystem, or system. *See Note 1*.

To migrate between the GP Profile and the Safety or Security Profiles, there is potential for a significant amount of rework to ensure that the services and applications are using the appropriate APIs that are certifiable to safety or security requirements.

Figure 14 shows a representation of system APIs and restrictions per profile.

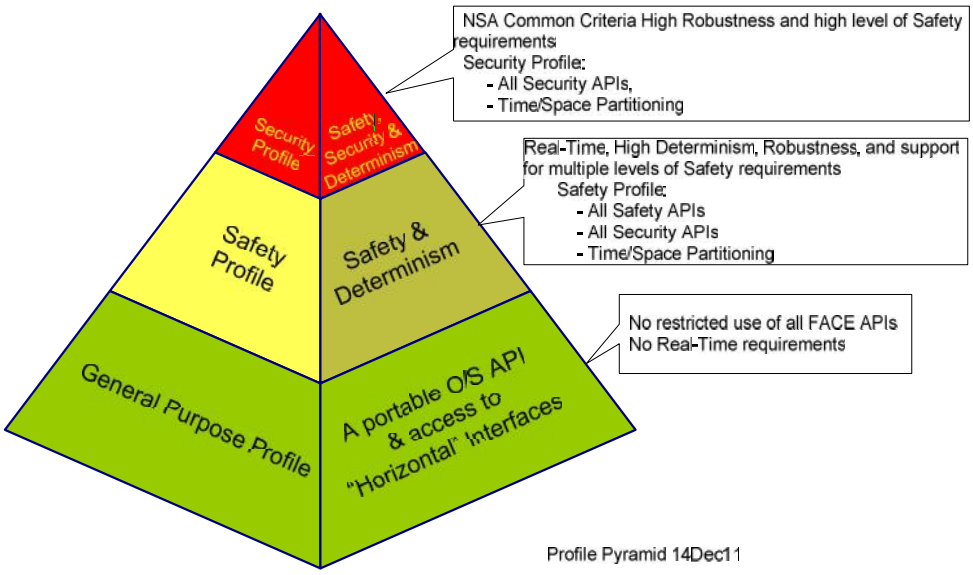


Figure 14: FACE Profile Diagram

3 FACE Software Computing Architecture

The FACE Software Architecture, as depicted in Figure 15, provides the basis for accomplishing the overarching FACE goals of reducing development and integration costs, as well as time to field. Each component and interface is described in further detail, in the following sections.

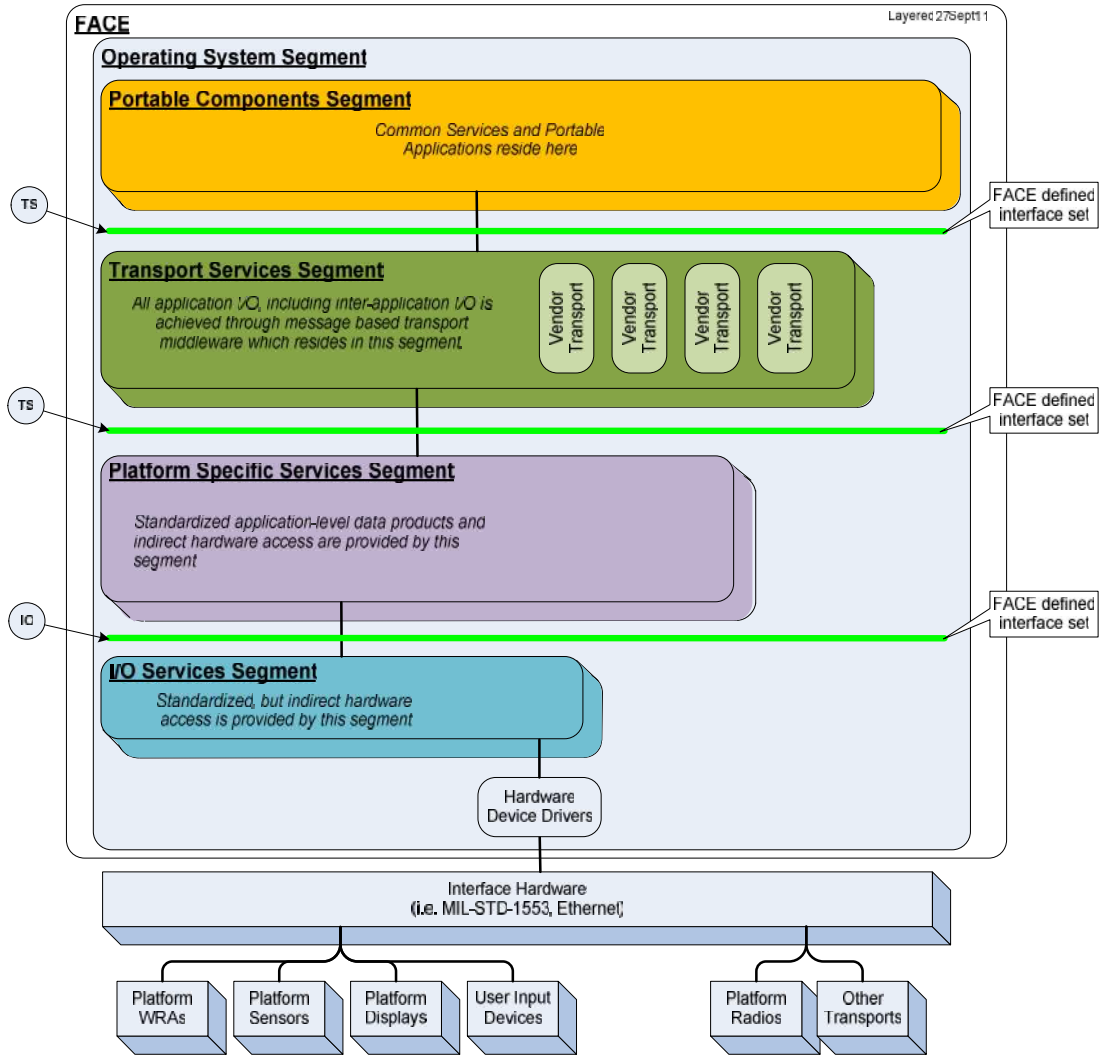


Figure 15: FACE Architectural Diagram

3.1 Operating System Segment

The Operating System Segment provides and controls access to the computing platform for the other FACE segments. The Operating System Segment uses, as appropriate, processor control

mechanisms, such as memory management units and register access controls (e.g., user *versus* kernel privileges) to restrict FACE segments to their required computing platform resources and operational capabilities. This level of control, when supported, permits various levels of independence between FACE segments, permitting greater portability across FACE-conformant computing platforms.

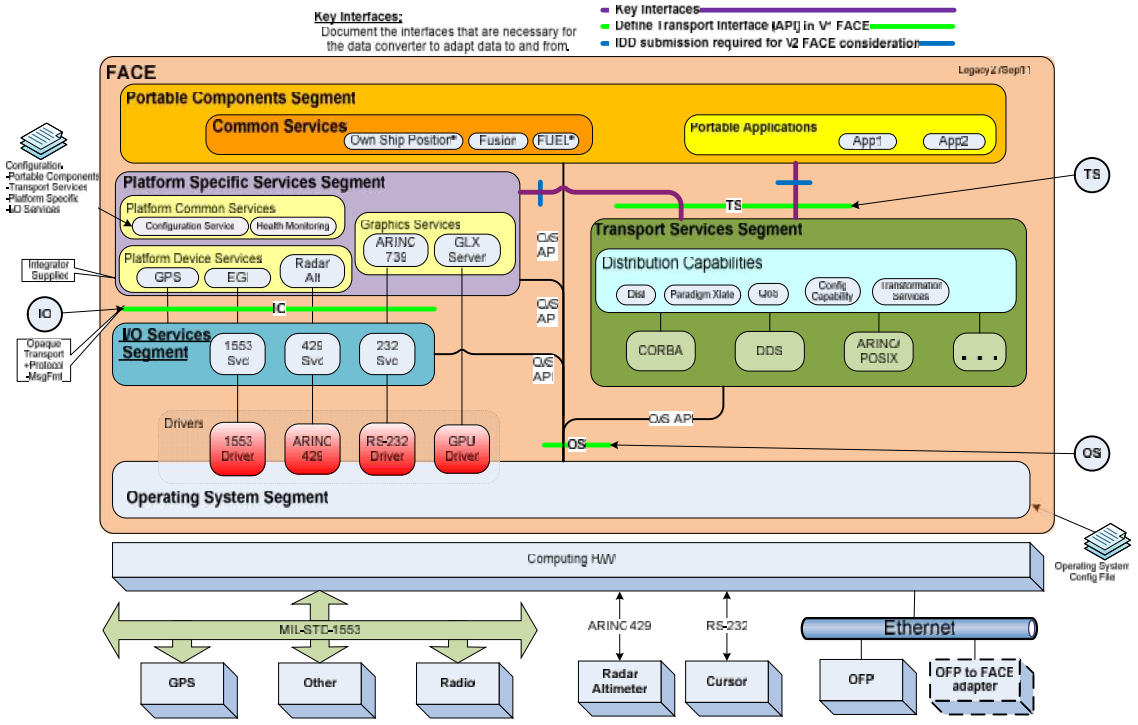


Figure 16: Notional FACE System Architecture and Interfaces

The Operating System Segment also includes I/O APIs (i.e., networking system, file system) that already adhere to published standards-based operating system interfaces to meet the platform interface requirements.

In addition, the Operating System Segment implements basic and sophisticated capabilities and environments for software execution. Access to these capabilities is provided to the other FACE segments via implementations of the Operating System APIs. The Operating System Segment capabilities will be implemented using processor-specific software that will execute in combinations of most privileged (e.g., kernel privileged) and least privileged (e.g., user privileged) processor execution environments. Regardless of where an Operating System Segment capability executes, the capability is still considered part of the Operating System Segment (i.e., it is not just software that executes with kernel privileges). Operating System APIs are defined in Section 3.9 and Appendix A. *See Note 6.*

3.1.1 Operating System Segment Requirements

The Operating System Segment shall (for all profiles unless otherwise noted):

1. Include support for partition, process, thread, and memory management capabilities.

2. Run the operating system (i.e., run software programs and use internal data structures to manage the computing platform and control resources in support of execution of application software).

3. Allocate partitions to enforce isolation between FACE segments, groups of FACE segments, applications, and processes that require isolation:

   — Control each partition's access to computing hardware and computing platform resources. Access is controlled based on configuration data, in accordance with Configuration Services, described in Section 3.12.

   — Support ARINC 653-defined configuration data types (XML-based schema) for allocating computing platform resources.

   — Support configuration data categories, including:

      - Partitions
      - Partition time windows
      - Health monitoring
      - Memory regions
      - Inter-partition communications (i.e., between partitions)
      - File system
      - Device driver access
      - Permission to set calendar time (including time zone) that is visible to all partitions
      - POSIX named semaphore access

4. Provide operational environments for execution of POSIX and/or ARINC 653 capabilities defined in the FACE profiles:

   — A POSIX partition shall use the FACE profiled POSIX APIs with the following exceptions:

      - If a POSIX partition requires communication with an ARINC 653 partition, then ARINC 653 sampling and queuing ports must be used by the POSIX partition for communication.
      - If a POSIX partition requires communication with an ARINC 653 Health Monitor, then ARINC 653 Health Monitoring APIs must be used by the POSIX partition.

   — An ARINC 653 partition shall use FACE profiled ARINC 653 APIs exclusively.

5. Provide a standards-based conformant API set.

   — The Operating System Segment shall support POSIX and ARINC 653 Operating System APIs that provide access to the Operating System Segment to other FACE

segments using the FACE General Purpose, Safety, and Security Profiles. Operating System APIs provided within each profile are detailed in Section 3.9.

— The FACE General Purpose and Safety Operating System APIs must include file system support (when implemented by the computing platform selected by the system integrator). The intent of file system is to provide multi-application access to non-volatile (i.e., across power-cycles) data storage. *See Note 7.*

— File system support includes:

- Non-volatile data storage and flushing of buffered data into non-volatile data storage
- Files, directories, and volumes
- Ability to allocate read and write access permissions of volumes to specific partitions (i.e., no default access permissions)
  - For Safety profiled partitions, zero or one partitions may be allocated write access permissions to a volume.
  - Zero or more partitions may be allocated read access permissions to a volume.
- Accounting for mounting or insertion of portable data media (e.g., externally mounted storage media), when supported by the computing platform
- Blocks of data that are atomically updated (i.e., data block is ensured to be completely saved in non-volatile storage or if interrupted, none of the data block is saved). This capability ensures consistency of the stored blocks of data. If a file is shared by multiple applications, file data structures (e.g., terrain database structures) should be developed and published in support of application portability.

6. Provide partitioning:

— Time and space partitioning for the Security Profile.

— Time and space partitioning may apply to the Safety Profile when safety requirements warrant.

— Time and space partitioning may apply to a General Purpose computing platform when safety or security requirements warrant.

— Partitions may be allocated unique time windows (i.e., one or more separate time windows allocated to only one partition).

— Two or more partitions may be allocated identical time windows (e.g., to permit a server to immediately respond to a client or to permit an equivalent of the POSIX multi-process behavior using partitions, or both). When partitions are allocated identical time windows, all threads associated with the applications assigned to the partitions must compete on a priority preemptive basis and may not be considered sufficiently segregated to allow qualification at different levels of criticality.

— Permit the system integrator to choose the type of partitioning (e.g., time; space; time and space; none) for a computing platform running in the FACE Safety and General Purpose Profile. The minimum suggested type is space partitioning. *See Note 8.*

3.2 Device Drivers

Device drivers provide adaptation and isolation of hardware devices so that they can be used by other entities that are using the computing platform. Vendor-supplied drivers may export operating system-specific interfaces (i.e., non-standard), or may use operating system-specific interfaces to interact with the operating system.

FACE systems will use device drivers as supplied by the operating system, device driver, or other (e.g., platform integrator) vendor. Due to differences in driver API sets, the I/O Services Segment will be responsible for providing adaptation to the supplied drivers. Vendors should implement device drivers separate from the I/O Services Segment (i.e., some level of abstraction should be provided). Future editions of FACE will standardize the driver interfaces on a per I/O class basis. *See Note 9.*

3.3 I/O Services Segment

I/O Services provides a collection of I/O protocol abstractions that are used by components in the PSS Segment. I/O Services provides a common API for I/O devices that do not have a standardized API contained within the Operating System Interface. The first edition of FACE contains a minimal set of I/O Services which will be re-evaluated with all future editions for inclusion of other possible I/O Services candidates. Figure 17 shows a notional view of the I/O Services Segment and how it relates to the PSS Segment. The I/O Services Segment, along with the message-based I/O services API, is intended to provide a standardized mechanism for the PSS Segment to access avionics I/O hardware. These components isolate the PSS Segments from the unique implementations of vendor-specific device drivers and operating system device driver API sets. The I/O Services Segment may reside in partitions or address spaces other than the ones occupied by the PSS Segment for which they provide I/O Services. This necessitates the development of a message-based API. The I/O Services Segment includes an I/O Management Capability that will be responsible for initializing, configuring, and managing the specific I/O Services. The I/O Services Segment must be modular to allow for the easy addition of I/O components. *See Note 6.*

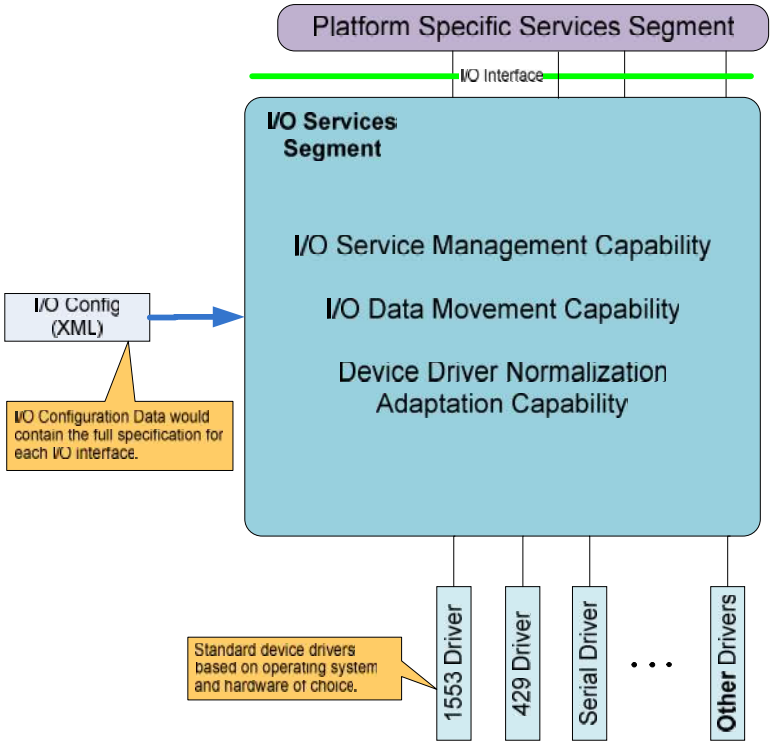


Figure 17: I/O Services Segment

3.3.1 I/O Services Segment Requirements

The following requirements are imposed on the I/O Services Segment:

1. The I/O Services Segment shall provide a common standard API and message-based interface mechanism to the PSS Segment for all FACE profiles. This enables distributed and non-distributed I/O Services.

2. The I/O Services Segment shall provide the following capabilities – *see Note 10*:

   — Device Driver Normalization

   — I/O Service Management

   — I/O Data Movement Capability

3.3.2 I/O Services Segment Definitions

- I/O Service: A collection of software components that provides a unified view of an I/O Interface to all software components that are using that interface.
- I/O Data Movement: The process of moving data between two end-points (Logical Ports).
- Logical Ports: The path over which data is moved between the I/O Service and the PSS Segment. This is in reference to system data addressability between partitions or computing platforms.

3.3.3 I/O Service Management Capability Requirements

1. The I/O Services Segment shall provide an I/O Service Management capability to facilitate the following:

   — Initialization of the I/O Devices

   — Configuration of the I/O Devices

   — Configuration of the I/O Services Segment

2. The I/O Service Management capability shall ensure that any I/O device initialization that is not part of the operating system's initialization occurs at the initialization of the system or partition.

3. The I/O Service Management capability shall ensure that I/O device initialization is completed prior to I/O Service configuration.

4. The I/O Service Management capability shall ensure that I/O Service configuration occurs at the initialization of the system or partition.

5. I/O Service reconfiguration shall be permitted at run-time in response to changes in the operating environment.

3.3.3.1 Device Initialization Capability Requirements

1. The I/O Service shall bring the I/O device to an operational level as defined by the device initialization data.

2. The Device Initialization capability shall accept device initialization data from a configuration capability as defined in Section 3.12.

3.3.3.2 Service Configuration Capability Requirements

1. The Service Configuration capability shall accept service configuration data from a configuration capability as defined in Section 3.12.

2. The Service Configuration capability shall include definition of all logical ports and the configuration data associated with each logical port.

3. The Service Configuration capability shall configure the I/O device.

4. The Service Configuration capability shall support I/O Service software configuration. *See Note 11*.

3.3.4 I/O Data Movement Capability Requirements

1. The I/O Data Movement capability shall move data between the I/O device and PSS Segment using defined logical ports.

2. The I/O Data Movement capability shall provide time stamping of data messages from I/O channels as required.

3. The I/O Data Movement capability shall support data message transport integrity operations as required.

4. The I/O Data Movement capability shall support data tagging (e.g., for security) as required.

3.3.5 Device Driver Normalization Adaptation Requirements

I/O Services will provide adapters that read and write data from and to individual, vendor-supplied I/O drivers using non-FACE, vendor-specific interfaces, and make that data available through, or accept it from, the I/O Interface, as described in Appendix B.

3.4 I/O Interface

The I/O Interface provides a standard API and message set to facilitate the use of individual I/O Services (IOS). The I/O Interface defines a generic API and message set for all I/O Interfaces. Some examples of these I/O Interfaces are MIL-STD-1553, ARINC 429, and Asynchronous Serial (RS-232, RS-422, and RS-485). All conformant FACE systems with I/O Services Segments shall implement the APIs defined in this section. If distributed I/O Services are implemented (i.e., IOS and PSS Segments are in different address spaces) then the APIs and their associated messages must be implemented. The I/O APIs are defined in Appendix B. *See Note 6.*

3.4.1 I/O Services API and Messages

The API set for IOS is:

```
FACE_IO_Open()
FACE_IO_Close()
FACE_IO_Register()
FACE_IO_Read()
FACE_IO_Write()
FACE_IO_GetStatus()
```

For implementations where the PSS Segment and the I/O Services are in the same address space (partition or process), these APIs may be direct function calls that are implemented by the I/O Service.

For implementations where the PSS Segment (or other users of I/O Services) and the I/O Services do not reside in the same address space (partition or process), these APIs must be implemented as a message-based API. The API caller will create an appropriate message and send it across the configured ARINC 653 IPC or POSIX IPC (depending on the implementation). The receiver of the message shall parse and process the message appropriately. The message connections will be defined in the system configuration information. Each connection shall be a unidirectional named entity. Figure 18 depicts the Data Model for the I/O Interface. In this model, the Message Instance is the message passed between components. The I/O Type Payload Definitions are the descriptions of the payloads that can be included within the Message Instance. Refer to the I/O Interface API set in Appendix B, which provides details on the APIs and their associated parameters and messages. *See Note 12.*

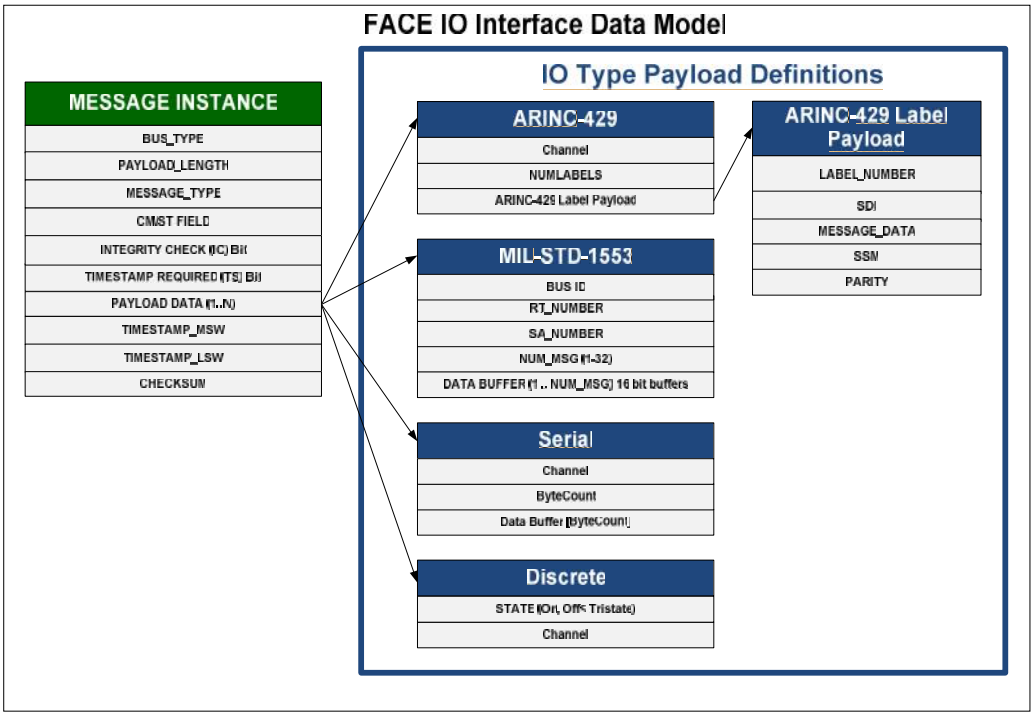


Figure 18: I/O Interface Data Model

3.4.1.1 I/O Message Instance

The I/O message instance provides a standard method of packaging data described by multiple avionics bus architectures. The message instance provides basic information required or provided by an I/O Service in a distributed I/O system. The message instance encapsulates the I/O-specific payloads into a standard format for FACE. The definition of the messages and data details of the I/O message instance and the I/O-specific payloads are documented in Appendix B.

- Each transaction between an I/O Service and the PSS Segment will consist of a single I/O message instance.
- The I/O message instance shall include:

  — BUS TYPE – Defines the type of avionics bus that the message is associated with (ARINC 429, MIL-STD-1553, Serial, etc.).

  — PAYLOAD LENGTH – Defines the size of message payload in bytes.

  — MESSAGE TYPE – Defines the type of message being transmitted.

  — COMMAND STATUS (CM/ST) FIELD – Indicates whether the message is a command message to or from the I/O Service, a status message to or from the I/O Service (e.g., health status (bit)), or a data transaction to or from the I/O Service.

  — INTEGRITY CHECK – Indicates to the receiving segment that an integrity check is to be calculated and the results compared to the INTEGRITY VALUE.

— TIMESTAMP REQUIRED – Indicates that time stamping of the message instance is required. If set, then the timestamp field must be filled out prior to transmission.

— PAYLOAD DATA – Contains the I/O unique payload. Each payload may have additional I/O unique header information.

— TIMESTAMP – Contains the time of validity for the message instance. This field must contain a valid time if the TIMESTAMP REQUIRED field is set.

— INTEGRITY VALUE – Contains the 32-bit field used for integrity checking. This value must be calculated prior to message transmission if an integrity check is required. Upon message receipt the integrity value should be calculated and compared to the INTEGRITY VALUE field.

3.4.2 I/O Services API Requirements

1. Provides a standardized API and associated message set between the I/O Services Segment and PSS Segment.
2. The I/O Interface API sets must be implementable as direct function calls to allow for communication with PSS Segments that are in the same address space (partition).
3. The I/O Interface API sets must be implementable as a message-based API to allow for communication with PSS Segments that are in different address spaces (partition or computing platform).
4. The I/O Interface API sets shall facilitate data exchange. *See Note 9.*
5. The I/O Interface API sets shall include messages for I/O device configuration and control capabilities.
6. The I/O Interface shall allow for communication with multiple sources and destinations through the same I/O Service.
7. The same API set shall be used for applications developed for the FACE General Purpose, Safety, and Security Profiles.

3.5 Platform-Specific Services Segment

The Platform-Specific Services (PSS) Segment creates an infrastructure unique to the air platform that provides device data to the portable applications located in the Portable Component Segment. The PSS Segment is broken into three sub-segments – *see Note 6*:

- Platform Device Services
- Platform Common Services
- Graphics Services

The Platform Device Services Sub-segment is made up of components to remove platform device-unique variability from the Portable Component Segment. The components of the Platform Device Services act as software abstractions of platform hardware devices to provide data and control capabilities to the Portable Component Segment.

The Platform Common Services Sub-segment defines a set of service components, zero or more of which may be implemented on a FACE computing platform, including Configuration Services, Platform Logging, and Health Monitoring. The Platform Common Services contain only service types that are agreed upon by the FACE Consortium and cannot be augmented without updating the FACE Standard.

The Graphics Services Sub-segment provides a set of graphics services to the Portable Component Segment. The graphics services provided vary by platform requirements and are selected by the platform integrator.

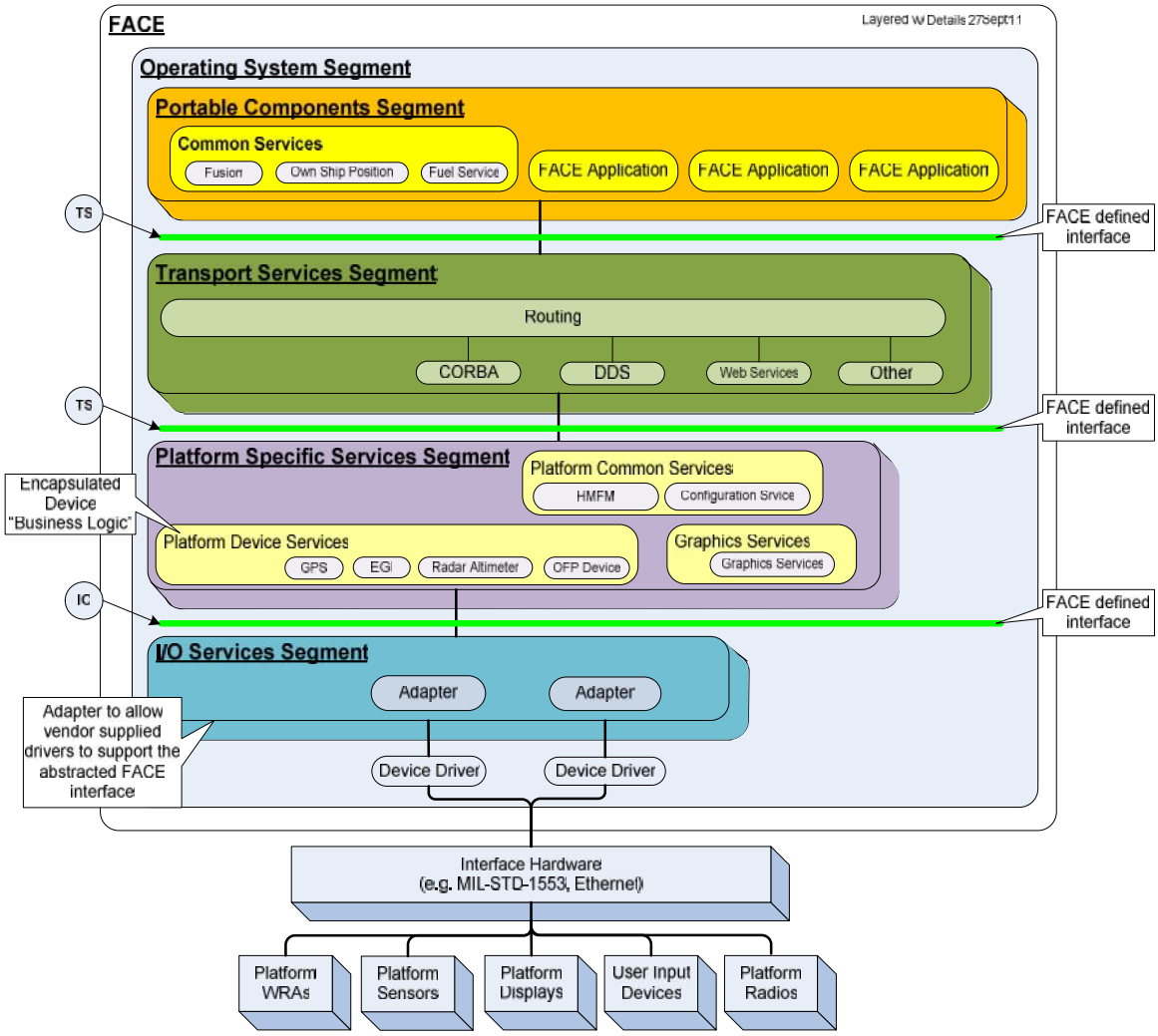


Figure 19: Notional PSS Segment

3.5.1 PSS Segment External Interfaces

Bi-directional communications between the PSS Segment and Transport Services Segment shall use the FACE-specified Transport Interface. Bi-directional communications between the PSS Segment and the I/O Services Segment (a set of portable services) shall use the FACE-specified I/O Interface. As an exception to the use of the I/O Services Interface, graphics components

within the Graphics Services Sub-segment may communicate directly with the GPU driver. *See Note 9*.

The data representation for the I/O Interface will be defined by platform devices or external systems' ICDs. *See Note 9*.

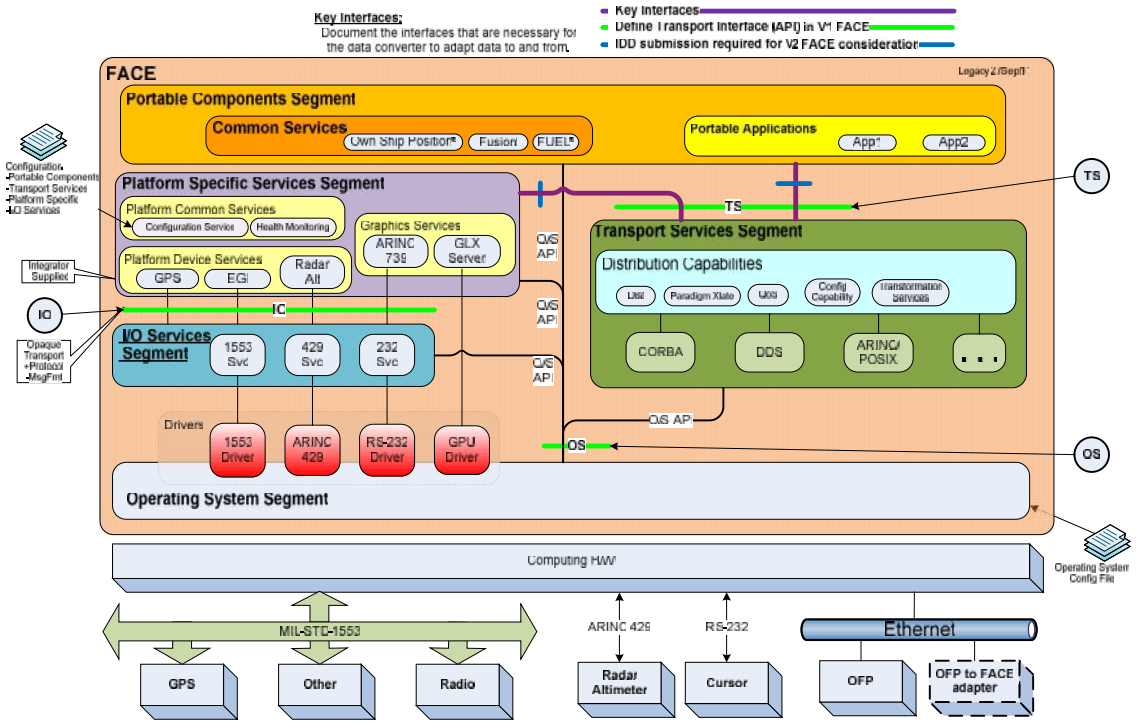


Figure 20: FACE System Architecture and Interfaces

3.5.2 Platform Device Services

The Platform Device Services Sub-segment may include components that provide control for, receive data from, and send data to platform devices or external systems. The Platform Device Service components will be capable of handling failover and redundancy where required. *See Note 9*.

3.5.3 Platform Common Services

The Platform Common Services Sub-segment shall provide common services to all other FACE computing environment segments. It shall communicate using the Transport Interface to the Portable Component Segment and the I/O Interface to the I/O Services Segment. Platform Common Services includes Section 3.10, Section 3.12, and Section 3.13 if implemented on a given system. *See Note 9*.

3.5.4 Graphics Services Sub-Segment

A system may have multiple graphics services components that will reside in the Graphics Services Sub-segment. These components receive ARINC 661-4, X11_R7 Byte streams, and/or ARINC 739/739A standardized streams of data for rendering. These components may

communicate directly with the GPU driver or indirectly to other platform devices through the I/O Interface. See Section 3.11 for detailed information. *See Note 9.*

3.5.5 Distributed PSS Components

A system may have multiple PSS Segments that can reside in multiple partitions or on multiple computing platforms.

These distributed PSS components shall communicate through the Transport Services Segment and Transport Interface. *See Note 9.*

3.5.6 PSS Segment Requirements

The Platform-Specific Services (PSS) Segment shall adhere to the following set of requirements – *see Note 13*:

1. The Platform Device Services components shall include components that provide control for, receive data from, and send data to a platform device or other external systems.
2. The PSS Segment components shall conform to the Transport, I/O, and Operating System Interfaces.
3. The Transport Interface shall be used for inter-partition/process or intra-partition/process communications between components within the PSS Segment.
4. The interfaces between the PSS Segment and the Transport Services Segment are considered key interfaces.
   - The data descriptions for key interfaces must be open and published if not already part of the FACE Standard. *See Note 9.*
   - Any data input or output shall be published in accordance with the Transport Services API data model definition. See Section 3.6.
5. All data that is transmitted to the Portable Component Segment shall use the Transport Interface and the mechanisms provided by the Transport Services Segment.
6. All communication with the I/O Services Segment shall go through the I/O Services Segment interface (I/O Interface).
7. All components of the PSS Segment shall use the Operating System Interface to access the functions provided by the Operating System Segment.
8. Platform Device Service Configuration information shall provide:
   - Mission-Specific Configuration – specifies mission-unique configuration information such as device installed, T/M/S-specific data processing, payloads available, etc. *See Note 9.*
   - Platform-specific functionality configuration information.
9. Components within the Platform Device Services shall be configurable for platform-specific and mission-specific functionality through platform and mission configuration information.

10. Platform-specific components shall be accessible by multiple system consumers and producers.

11. The Platform Device Services Sub-segment may be required to address redundant sources of communication (e.g., pathways or busses) and pathway failover conditions, which are needed to communicate with specific platform devices and external systems without loss of data.

12. Graphics components of the Graphics Services Sub-segment may communicate directly with the GPU using its proprietary driver interface APIs.

3.6 Transport Interface

The Transport Services Interface provides a standardized interface library for application layer component communication. Such components reside in the Portable Component Segment and the PSS Segment. The standardization of this interface allows application layer developers to create re-usable software components/products, and will facilitate the affordable integration of these portable avionics applications into disparate architectures and aviation platforms that are FACE-conformant. The Distribution Capability within the Transport Services Segment manages the interface, and provides for the distribution of information based upon data from the Transport Services Segment Configuration Capability. *See Note 6.*

3.6.1 Transport Protocol Requirements

1. The Transport Services API shall support the communication and message services required to meet the system performance (throughput, latency, delivery guarantees, etc.) for the associated data exchanges. This API is based on industry standards and will evolve as new technologies become available. The near-term standards supported by the Transport Services API are, but are not limited to, POSIX, ARINC 653, CORBA, DDS, and ARINC 664. *See Note 14.*

3.6.2 API Services Requirements

1. The Transport Services API shall support the following communication styles – *see Note 14*:

   — Buffering/queuing

   — Single Instance Messaging (e.g., sampling port, shared memory)

2. The Transport Services API shall support the following synchronization styles – *see Note 14*:

   — Blocking

   — Non-blocking

3. The Transport Services API shall support the following message structures – *see Note 14*:

   — Fixed Length

   — Variable Length

4. The Transport Services API shall support the following message timing types – *see Note 14*:

   — Periodic

   — Aperiodic

5. The Transport Services API shall support the following message distribution types – *see Note 14*:

   — Broadcast

   — Multicast

   — Unicast

6. The Transport Services API shall provide the following services as defined by Appendix C:

   — *create_connection*()

   — *destroy_connection*()

   — *receive_message*()

   — *send_message*()

   — *register_read_callback*()

   — *get_connection_parameters*()

7. The Transport Services API shall support OpenGL standardized library direct calls.

3.6.3 API Data Model Requirements

Figure 21 describes the Transport Services API data model definition.

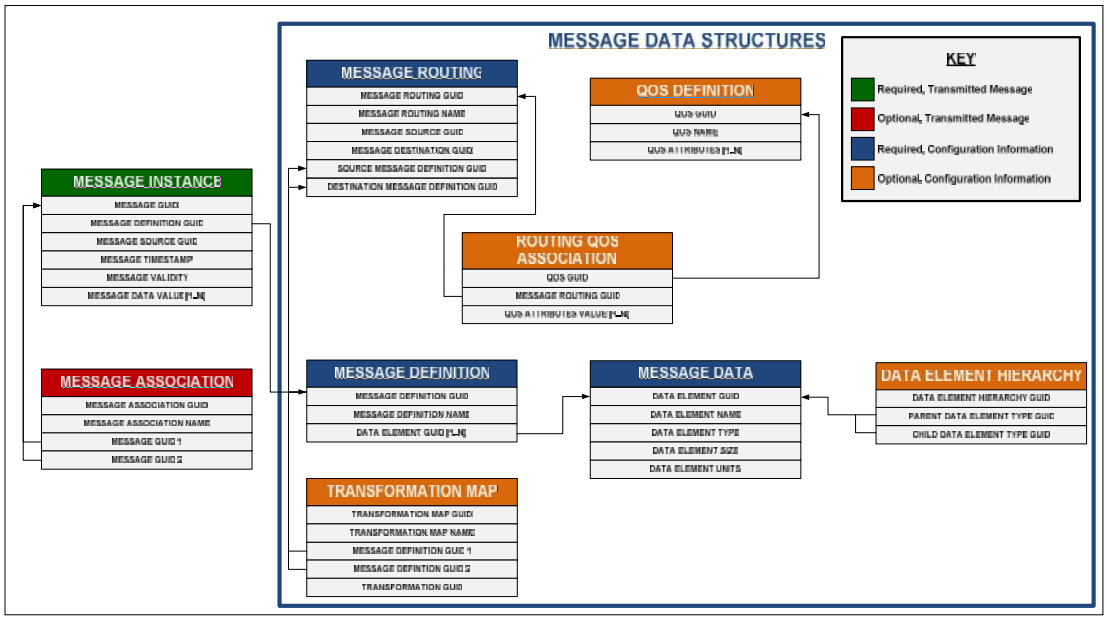


Figure 21: Transport Services API Data Model Definition

3.6.3.1 Message Instance

The Message Instance in Figure 21 is the basic element of data exchange within the Transport Services API Data Model. The Message Instance is a concrete instantiation of the message type for the Transport Services Interface. The Message Instance is the data transferred between software components at run-time. The Message Instance does not include the Message Data Structures associated with routing, QoS, definition, data, hierarchy, and transformation.

The documentation of the following Message Data Structures shall be included in the Technical Data Package (TDP) provided with each software component:

1. Message Routing
2. Message Definition
3. Message Data

The documentation of the following Message Data Structures is not guaranteed to be present in the TDP provided with each software component:

1. Message Association
2. Routing QoS Association
3. QoS Definition
4. Data Element Hierarchy
5. Transformation Map

This Message Data Structures description will be described in the design documentation to allow for integration of software components in to multiple systems.

1. The Transport Services Interface shall consist of one (1) Message Instance.

2. The Message Instance shall consist of the following data elements:

   — **Message GUID (Globally Unique Identifier)** – The Message GUID uniquely identifies each instance of the "Message Instance" entity.

   — **Message Definition GUID** – The message definition GUID is a reference to a specific Message Definition instance that defines the structure of the message.

   — **Message Source GUID** – The Message Source GUID is a reference, through the unique identifier, to a source of Message Instances.

   — **Message Timestamp** – The Message Timestamp is the timestamp describing when the message was written by the application to the message interface.

   — **Message Validity** – The Message Validity attribute is used to indicate the validity of the data (i.e., field not used, valid, invalid, stale, no data) as ascertained by the creating application.

   — **Message Data Value** – The Message Data Values are the corresponding values of the Data Element entities specified by the referenced Message Definition. These values compose the body of each message.

3.6.3.2 Message Association Structure

The Message Association Structure is used to create and distribute a message indicating an association between two Message Instances. The Message Association can be used by a software component to understand if an associated Message Instance exists and can be requested. The Association can also be used to associate two Message Instances at differing classification levels.

1. Each Message Instance may consist of one (1) Message Association.

2. The Message Association Structure shall consist of the following data elements:

   — **Message Association GUID** – The Message Association GUID uniquely identifies each instance of the Message Association entity.

   — **Message Association Name** – The Message Association Name is a human-readable description of the association. The name need not be unique, may be an enumerated description, or one of a predefined set of association descriptions.

   — **Message GUID 1** – Message GUID 1 references the Message GUID of the associated instance of a "Message Instance" entity.

   — **Message GUID 2** – Message GUID 2 references the Message GUID of the associated instance of a "Message Instance" entity.

3.6.3.3 Message Data Structures

The Transport Services Interface shall consist of the following seven (7) Message Data Structures:

1. Message Routing Structure
2. Quality of Service (QoS) Definition Structure
3. Routing Quality of Service (QoS) Association Structure
4. Message Definition Structure
5. Message Data Structure
6. Data Element Hierarchy Structure
7. Transformation Map Structure

3.6.3.4 Message Routing Structure

The Message Routing Structure statically defines the routing of messages within the system by the Transport Services Interface. There is one entry for each point-to-point path for each message type.

1. The Message Routing structure shall consist of the following data elements:

   — **Message Routing GUID** – The Message Routing GUID uniquely identifies each instance of the Message Routing entity.

   — **Message Routing Name** – The Message Routing name is a human-readable description of the message route. Each Message Routing Name is unique.

   — **Message Source GUID** – The Message Source GUID is a reference, through the unique identifier, to a source of Message Instances.

   — **Message Destination GUID** – The Message Destination GUID is a reference, through the unique identifier, to a destination of Message Instances.

   — **Source Message Definition GUID** – The Source Message Definition GUID is a reference to a Message Definition that defines a Message Instance intended for transport from a message source.

   — **Destination Message Definition GUID** – The Destination Message Definition GUID is a reference to a Message Definition that defines a Message Instance intended for transport to a message destination.

3.6.3.5 QoS Definition Structure

The QoS Definition Structure defines the QoS parameters that may be applied to the transport of data within the system by the Transport Services Interface.

1. The QoS structure shall consist of the following data elements:

— **QoS GUID** – The QoS GUID is a unique identifier for each instance in the QoS Definition entity.

— **QoS Name** – The QoS Name is a unique name for the QoS capability or feature. Examples of QoS Name might be Reliable Delivery or Priority.

— **QoS Attributes** – The QoS Attributes are the parameters associated with a particular QoS capability or feature. An example of the QoS Attributes associated with the 'Reliable Delivery" QoS capability might be "kind" and "max_blocking_time".

3.6.3.6 Routing QoS Association Structure

The Routing QoS Association Structure associates QoS features or capabilities with specific values of the QoS parameters attached to an individual data connection specified by a Message routing instance that may be applied to the transport of data within the system by the Transport Services Interface.

1. The Routing QoS Association Structure shall consist of the following data elements:

— **QoS GUID** – The QoS GUID is a reference to a specific QoS Policy defined in the QoS Definition entity.

— **Message Routing GUID** – The Message Routing GUID is a reference to a specific Message Routing instance that specifies a particular data transport path.

— **QoS Attributes Values** – The QoS Attribute Values are the specific value of the QoS parameter specified by the QoS definition referenced by the QoS Definition entity referenced by the QoS GUID. Examples of QoS Values for the QoS policy "Reliable Delivery" might be "Reliable" and "0" for the attributes "kind" and "max_blocking_time".

3.6.3.7 Message Definition Structure

The Message Definition Structure defines the unique structure of each message type within the system by the Transport Services Interface.

1. The Message Definition Structure shall consist of the following data elements:

— **Message GUID** – The Message Definition GUID is a unique identifier for each instance of the Message Definition entity.

— **Message Name** – The Message Definition Name is a unique human-readable description of the message.

— **Data Element GUID** – The repeating element Data Element GUID is a reference to entries in the Data Definition entity. This set of Data Element entities specifies the content of the message defined by an instance of Message Definition.

3.6.3.8 Message Data Structure

The Message Data Structure defines each data element used by the Transport Services Interface.

1. The Message Data structure shall consist of the following data elements:

— **Data Element GUID** – The Data Element GUID is a unique identifier for each instance of a data element.

— **Data Element Name** – The Data Element Name is a unique human-readable description of the data element.

— **Data Element Type** – The Data Element Type is the type of the data element (e.g., float, int, char, string, etc.).

— **Data Element Size** – The Data Element Size is the size of the data element field.

— **Data Element Units** – The Data Element Units is the units of the data element (atomic element within the data dictionary).

3.6.3.9 Data Element Hierarchy Structure

The Data Element Hierarchy Structure provides the means to define complex data types in terms of the composition of simple types used by the Transport Services Interface.

1. The Data Element Hierarchy Structure shall consist of the following data elements:

— **Data Element Hierarchy GUID** – The Data Element Hierarchy GUID is a unique identifier associated with each Data Element Hierarchy instance, and is a candidate key.

— **Parent Data Element Type GUID** – The Parent Data Element Type GUID is a reference to the Data Element GUID attribute of the Data Element Definition entity. This is the entry point into a complex data structure.

— **Child Data Element Type GUID** – The Child Data Element Type GUID is a reference to the Data Element GUID attribute of the Data Element Definition entity.

The combination of the three GUIDs creates a candidate key for the Data Element Hierarchy instance. There may be two or more Data Element Definition entities referenced as Child Data Element entities associated with a single Parent Data Element entity. Data Elements referenced as a child may be either simple or complex (a Parent Data entity may be associated with a Data Element entity that is also a Parent Data entity). Data Element Definition entities referenced as a Child Data Element may be referenced more than once by a Parent Data Element to create complex data types with repeating elements of a single data type.

3.6.3.10 Transformation Map Structure

The Transformation Map Structure provides the means to define linear data conversions and data marshalling transformations to be used by the Transport Services Interface on a given message definition entity.

1. The Transformation Map Structure shall consist of the following data elements:

— **Transformation Map GUID** – The Transformation Map GUID is a unique identifier associated with each Transformation Map instance.

— **Transformation Map Name** – The Transformation Map Name is a unique human-readable description of the transformation.

— **Transformation Message Definition GUID 1** – The Transformation Message Definition GUID 1 is a unique identifier associated with the source message definition instance.

— **Transformation Message Definition GUID 2** – The Transformation Message Definition GUID 2 is a unique identifier associated with the destination message definition instance.

— **Transformation GUID** – The Transformation GUID 1 is a unique identifier associated with the transformation instance.

2. The data representation of the PSS Segment to the Portable Component Segment or other PSS Segments within the system shall adhere to the FACE standardized data representation unless a FACE standardized data representation does not exist for that type of information. If the data representation does not exist in the FACE Standard then the data must be described in a data dictionary; i.e., Interface Design Definition (IDD) provided to the customer (i.e., government, contractor or other supplier) by the developer with the appropriate data rights to enable the data description to become part of the FACE Standard.

3.7 Transport Services Segment

3.7.1 Introduction

The purpose of the Transport Services Segment is to provide a re-usable software library, communicating through a standardized interface, and facilitating affordable integration of portable avionics applications into disparate architectures and aviation platforms. The Transport Services Segment provides a standardized interface mechanism for application layer components located in the Portable Component Segment and PSS Segment. This Segment handles the distribution, routing, prioritization, addressability, association, abstraction, and transformation of avionics application data. The Transport Interface is defined in Section 3.6 and Appendix C.

Figure 22 summarizes the Transport Services Segment external interfaces and internal capability interfaces.

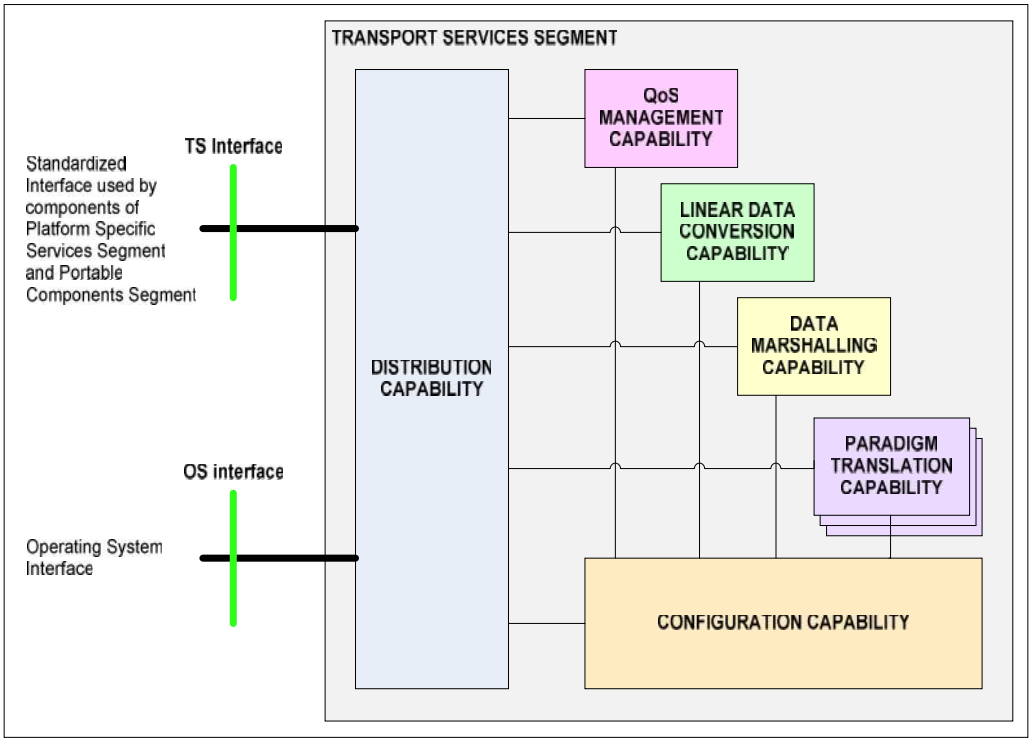


Figure 22: Transport Services Segment External and Internal Capabilities

The Transport Services Segment will support a variety of messaging paradigms (including but not limited to publish/subscribe messaging and request/response messaging) and provide a standardized API for implementation of the communication mechanisms required for these messaging paradigms. These paradigms will accommodate the transport of data and support delivery type configuration. There are existing standards that specify these messaging paradigms. These include, but are not limited to Common Object Request Broker Architecture (CORBA), Data Distribution Service (DDS), and World Wide Web Consortium (W3C) Standards. *See Note 6.*

The Transport Services Segment library includes each capability identified in Figure 22. The capabilities used by each instantiation of the Transport Services Segment will depend on the configuration information of a given instantiation. As a minimum, a given instantiation will include the Distribution Capability and the Configuration Capability. The use of the QoS Capability, Message Association Capability, Data Transformation Capability, and Paradigm Translation Capability depends on the system integrator requirements for a given instantiation of the Transport Services Segment. *See Note 6.*

It is the responsibility of the System Integrator to select the specific implementation for a given effort. The System Integrator's selection of a specific paradigm or set of paradigms will be dependent on many factors, including requirements such as safety and security. Components designed to be Transport Service-agnostic will provide the portability necessary to affordably survive technology refresh driven by an obsolescence event without compromising safety or security. *See Note 6.*

The Transport Services Segment will support message interfaces for communications, message passing (including networking), and synchronization. As a minimum, the Transport Services Segment will support the following message types: Fixed Length; Variable Length, Periodic, Aperiodic, Broadcast, Multicast, and Unicast. *See Note 6.*

The Transport Service Interface API, API Services, and Data Model elements are described in Section 3.6: Transport Interface and Appendix C.

3.7.2 Transport Services Segment Definitions

- Broadcast – A broadcast is a message sent from a single source to all destinations.
- Command/Response – A messaging paradigm in which a computing process exchanges data with a separate computing process and is able act on the data returned from the other computing process to generate a response.
- Data Transformation – The assembling and marshalling of the data structures and data from one-to-many Segment messages into a message data structure that can be understood by one-to-many Portable Component Segment components or *vice versa*.
- End-points – The allowable origin and destination segments for data routed through the Transport Services Segment. The Transport Services Segment end-points include the PSS Segment and Portable Component Segment components.
- Messaging Paradigm – The set of rules by which computer systems and applications share data between different entities (e.g., Publish/Subscribe and Request/Response).
- Multicast – A multicast is a message sent from a single source to a defined set of destinations.
- Paradigm Translation – The transformations necessary to convert data communication messages from one communication framework to another communication framework (e.g., CORBA to DDS, DDS to CORBA).
- Publish/Subscribe – A messaging paradigm in which a computing process does not require knowledge or presence of other computing processes in order to produce data for use by those other computing processes.
- Quality of Service – A set of quality requirements for the collective behavior of one or more data exchange objects. Quality of Service (QoS) comprises requirements for the attributes of a data exchange connection. Examples of attributes subject to QoS include: persistence, lifespan, history, priority, urgency, delivery reliability, deadline, latency, periodicity, order, and ownership.
- Unicast – A unicast message is a message sent from a single source to a single destination.

3.7.3 Transport Services Segment Requirements

1. The Transport Services Segment shall provide for data exchange between end-points using the Transport Services Interface.

2. The Transport Services Segment shall use Operating System APIs to support communications.

3. The Transport Services Segment shall manage the distribution, configuration, association, Quality of Service (QoS), transformation, and translation of data between end-points. *See Note 15.*

4. The Transport Services Segment shall consist of the following capabilities:

   — Distribution Capability

   — Configuration Capability

5. The Transport Services Segment shall support instantiation of the following capabilities if required by the system:

   — Message Association Capability

   — QoS Capability

   — Data Transformation Capability

   — Paradigm Translation Capabilities

6. The Transport Services Segment shall support the following paradigms:

   — Publish/Subscribe

   — Command/Response

7. The Transport Services Segment paradigms shall support the following characteristics:

   — Synchronous Communications

   — Asynchronous Communications

8. The Transport Services Segment shall support the following messaging interfaces:

   — Communications/Message Passing (including networking standards)

   — Synchronization

9. The Transport Services Segment shall support the following communication mechanisms:

   — Networking Standards (e.g., POSIX sockets)

   — Cache (e.g., Shared Memory)

10. The Transport Services Segment communication mechanisms shall support the following message structures:

    — Fixed Length

    — Variable Length

11. The Transport Services Segment communication mechanisms shall support the following message timing types:

— Periodic

— Aperiodic

12. The Transport Services Segment communication mechanisms shall support the following message distribution types:

— Broadcast

— Multicast

— Unicast

13. The configuration of the Transport Services Segment shall be achieved through FACE Configuration Services identified in Section 3.12.

3.7.4 Distribution Capability Requirements

1. The Distribution Capability shall be configured at initialization using Transport Services configuration information.
2. The Distribution Capability shall be configurable to accommodate multiple independent paradigm solutions.
3. The Distribution Capability shall be responsible for accepting data from data producers and distributing it to data consumers based on the configuration information.
4. The Distribution Capability shall use the Message Association Capability to manage information and data messaging associations and tagging.
5. The Distribution Capability shall use the Data Transformation Capability to perform the following functions if required:

— Linear data conversions

— Assembly and time stamping of data

6. The Distribution Capability shall use Paradigm Translation Capabilities to accommodate translations between disparate paradigm solutions (e.g., CORBA, DDS).

3.7.5 Configuration Information Requirements

1. Transport Services Segment Configuration Information shall include configuration information for the following capabilities:

— Distribution Capability

— Configuration Capability

— Message Association Capability

— Quality of Service (QoS) Capability

— Data Transformation Capability

— Paradigm Translation Capabilities as required

2. The Transport Services Configuration Capability shall be responsible for managing the interface to the platform configuration management service.

3. The Transport Service Configuration Capability shall be responsible for distributing information between each Transport Services Segment capability and the platform configuration management service.

Information describing the parameters defined in the Transport Services Segment configuration information is located in Appendix C.

3.7.6 QoS Management Capability Requirements

1. The QoS Management Capability shall be configured at initialization through information provided by the Transport Services Capability information.

2. The QoS Management Capability shall be responsible for managing the QoS of the messages being transported by the Transport Services Segment as per the information provided from the Configuration Capability.

3.7.7 Message Association Management Capability Requirements

1. The Message Association Management Capability shall be configured at initialization through information provided by the Transport Services Configuration information.

2. The Message Association Management Capability shall be responsible for managing message associations and data tagging as per the information provided from the Configuration Capability.

3.7.8 Data Transformation Capability Requirements

1. The Data Transformation Capability shall be configured at initialization through information provided by the Transport Services Capability information.

2. The Data Transformation Capability shall be responsible for managing the assembly/marshalling of data based upon the configuration information provided by the Configuration Capability.

3. The Data Transformation Capability shall use information from data producers to assemble messages for data consumers.

4. The Data Transformation Capability must provide the most recent data to consumers based upon the consumer request rate. Note that if the request rate is greater than the production rate, then stale data will be provided. Also note that the Data Transformation Capability will not provide any buffering of producer data.

5. The Data Transformation Capability shall be responsible for the time stamping of Transport Services Segment data.

6. The Data Transformation Capability must be responsible for handling linear data conversions based upon the configuration information provided by the Configuration Capability.

3.7.9 Paradigm Translation Capabilities Requirements

1. The Paradigm Translation Capabilities shall be responsible for transforming data from one paradigm to another (e.g., CORBA to DDS).
2. Each Paradigm Translation Capability shall provide an independent capability, translating from one distinct paradigm to another distinct paradigm.
3. Each independent Paradigm Translation Capability shall be able to be instantiated individually or in conjunction with other independent translation capabilities within a distributed Transport Services Segment instantiation.

3.8 Portable Component Segment

3.8.1 Definition and Scope of Portable Component Segment

The Portable Component Segment is an architectural concept that describes a set of FACE deliverables that share similar properties.

The Portable Component Segment is not a deliverable container for software. Rather, it is a portion of a FACE solution whose contents are entirely independent from other FACE Segments, meaning that no software deliverable could be part of the Portable Component Segment and simultaneously be part of any other segment.

Software deliverables are considered to be part of the Portable Component Segment when they share the following properties:

- The deliverables provide mission-level capabilities.
- The deliverables include a capability, which could execute on multiple varying instantiations of FACE computing hardware and FACE software computing environments.
- The deliverable exclusively interfaces to the remainder of the FACE environment through the Transport Interface for data exchanges.
- The deliverable exclusively uses the Operating System APIs for operating system support.
- The deliverable does not use the I/O Interface.
- The deliverable shall use graphics services directly per Section 3.11. *See Note 6.*

The Portable Component Segment logically hosts a wide range of applications and services. A set of application and service examples is as follows:

Application or Service Type	Description
Pure Data Processing	This application/service class adds processing or other manipulation to incoming data sets and does not render a display. These applications and services are portable across FACE computing environments and intended for high levels of re-use and re-deployment due to the common and/or generalized capabilities.

Application or Service Type	Description
Computing Support	Software deliverables that support other applications and services in the portable application segment are included in this application class. Examples include programming language-based run-times, application frameworks, and non-FACE software.
Display Application	This application/service class renders display content. These applications may also process data and therefore provide a superset of functionality over Pure Data Processing applications.
Integration Services	This application/service class is populated with capabilities that facilitate the integration activities involved in building a FACE solution from FACE components.

Table 1: Portable Component Types

3.8.2 Meaning of "Application Portability"

Applications are designated as *portable* when they can be re-deployed on different computing hardware and/or FACE software environments without requiring more than a recompilation of the application and a re-linking of software libraries, language run-times, and/or frameworks.

The particular instances of software libraries, language run-times, and frameworks may be replaced with instantiations that are appropriate for the target computing hardware and FACE software environment, but only if the APIs between the main application and these replacements conform to the same APIs used by the deliverables they replace.

3.8.3 Libraries, Run-Times, and Application Frameworks

Application source code for production systems is never purely monolithic. Source code is segregated into sub-routines at design time. It is a common practice to extract highly re-used subroutines from applications and to deploy them as a software library, which may then be linked to the original application and many others to provide a consistent *bolt-on* capability at a reduced software maintenance cost.

There are no restrictions made by programming languages on what capabilities can be achieved by an application and therefore there are no such restrictions on application-external libraries that are linked to the main application. There are several different methodologies for connecting application-external libraries to the application cores, such as the automatic link-editing of libraries performed by modern IDEs, programming language run-time interfaces, operating system interfaces, and application frameworks. All these are examples of externalized subroutines that are made accessible to the main application.

Libraries, Operating System APIs, programming language run-times, and application frameworks all share a common property because they are all forms of externalized subroutines. The shared property is the definition of an API as the means of access for client applications. Figure 23 depicts the main application as an oval and subroutines as rectangles with rounded corners. Each of the subroutines, whether internal or external, is accompanied by a circle labeled *API* which represents the API for that subroutine set. There are often a large number of APIs used by production quality applications; Figure 23 only depicts a few.

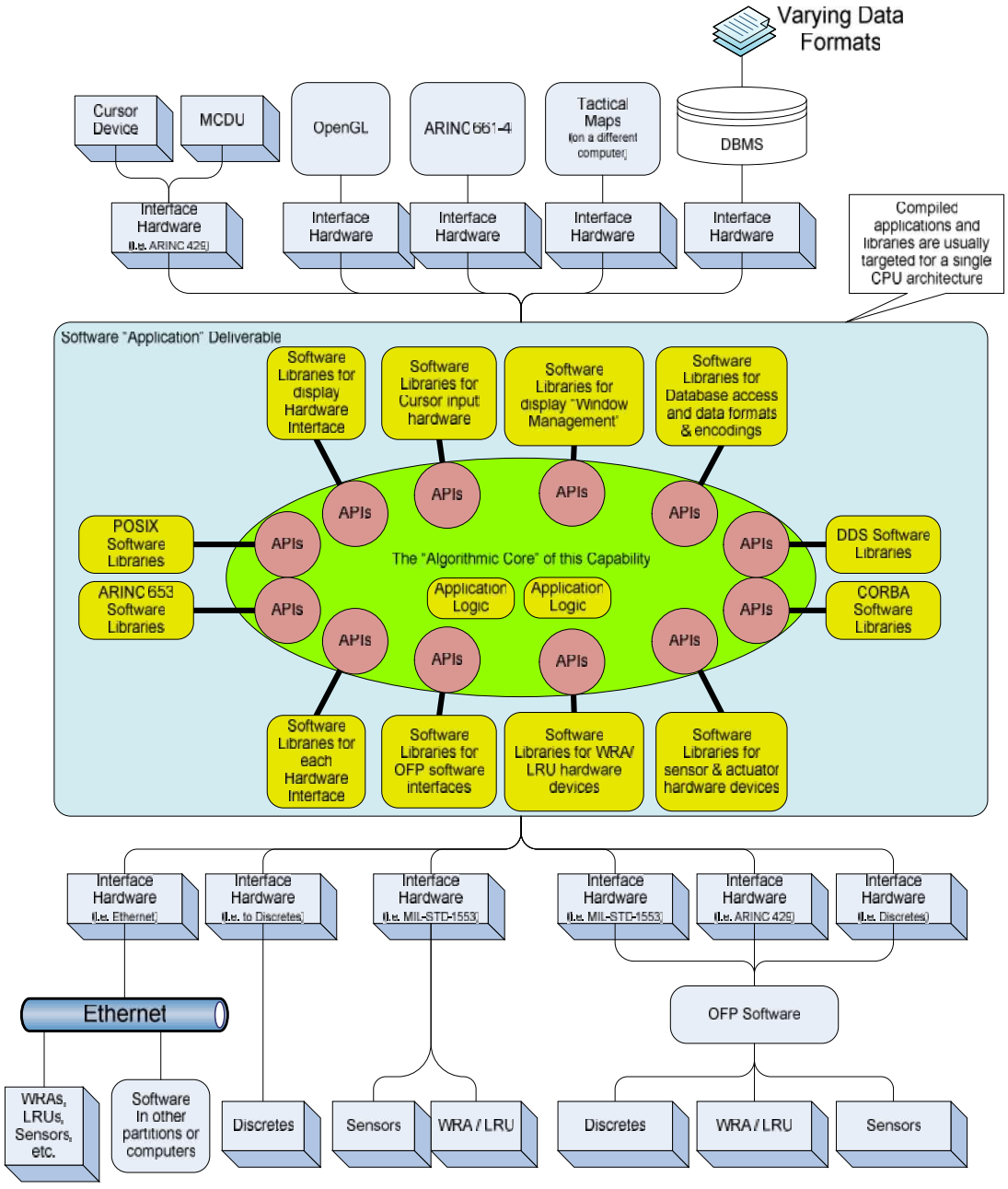


Figure 23: Platform-Specific Services Segment Example

Any modifications to the set of APIs (usually initiated as externally driven changes to the library) mandates an effort to modify the main application's core. An API-driven change to the application core could be restricted to the application code calling the library's APIs. This effort could expand to include changes in display content, and the core data model (i.e., data structures), and processing sequences and methodologies. The cost, skilled effort, and calendar impact to achieving application portability when APIs are changed can be high, which necessitates analysis of API modifications.

3.8.4 Unit of Portability (UoP)

Applications provide functional capabilities to users at the mission level. To illustrate the scope of what is meant by *mission-level capabilities*; a few examples include flight management functionality, navigation, and weapons management.

Mission-level capabilities which are implemented in a way that enables their re-deployment to other war-fighting platforms are more cost-effective than an approach that requires the development of an additional *point-solution* that provides the same capability for the each new war-fighting platform. This increase in cost-effectiveness is due to reductions in source code modification, testing, training, integration, qualification, and maintenance.

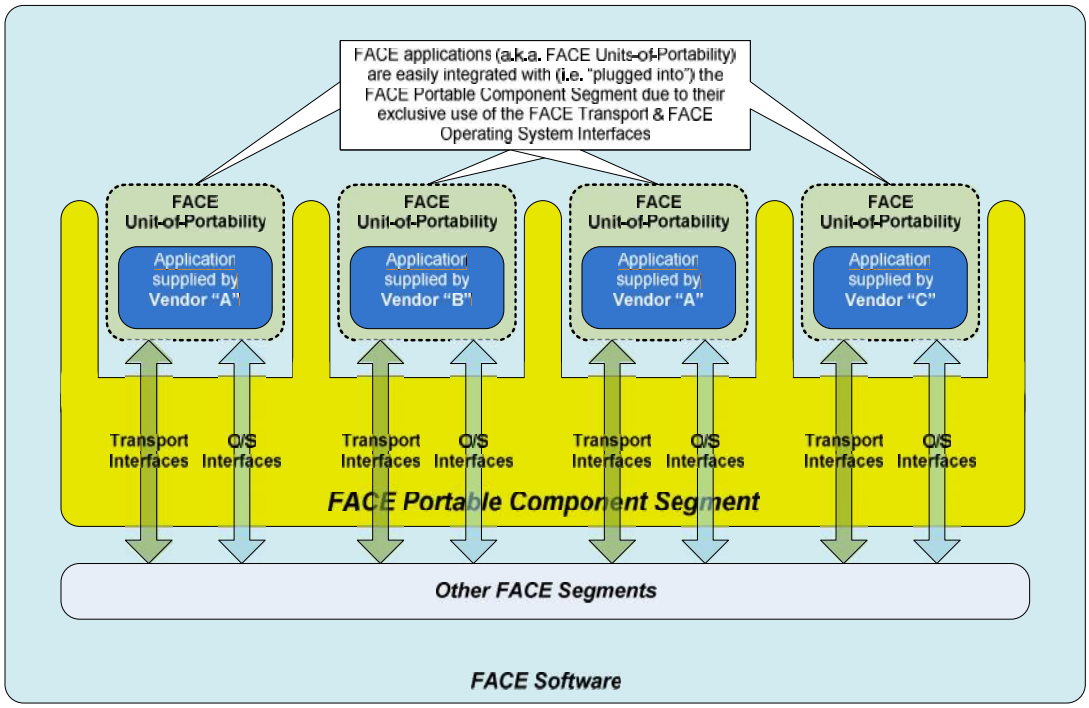


Figure 24: Standardization and Constraint on UoP Interfaces

Applications are portable across FACE-conformant war-fighting platforms which are similarly equipped. For example, an application may be re-deployed to a different aircraft that also has a FACE environment. But, if the application requires altitude data, the target aircraft needs to have some source of altitude data (i.e., similarly equipped) to ensure correct operation of the application. This application portability, as well as the ability to support applications from different vendors on a single FACE computing environment, is made possible because all applications are constrained to using only the Transport Interface and the Operating System Interface for all data traversing the *Unit of Portability* (UoP hereafter) boundary as depicted in Figure 24.

3.8.4.1 UoPs Defined

There are two categories of UoPs: the *FACE UoP* (F-UoP hereafter) and the *Capability UoP* (C-UoP hereafter), both defined and described below.

FACE Unit of Portability

The F-UoP describes the set of software which provides one or more mission-level capabilities as well as every other software deliverable that is required for complete and correct execution of that mission-level capability (e.g., run-time executives and application frameworks) excepting the set of software which is accessed through the Transport, or Operating System Interfaces.

Capability Unit of Portability

The C-UoP is that subset of the F-UoP software set that contains the portable mission-level capability or capabilities.

C-UoPs are a helpful method of packaging in cases where incremental capability is desired on an existing war-fighting platform that already contains the required infrastructure (e.g., run-time executives or application frameworks). In these cases , the C-UoP may be deployed on the target platform without also redeploying the remainder of that capability's F-UoP.

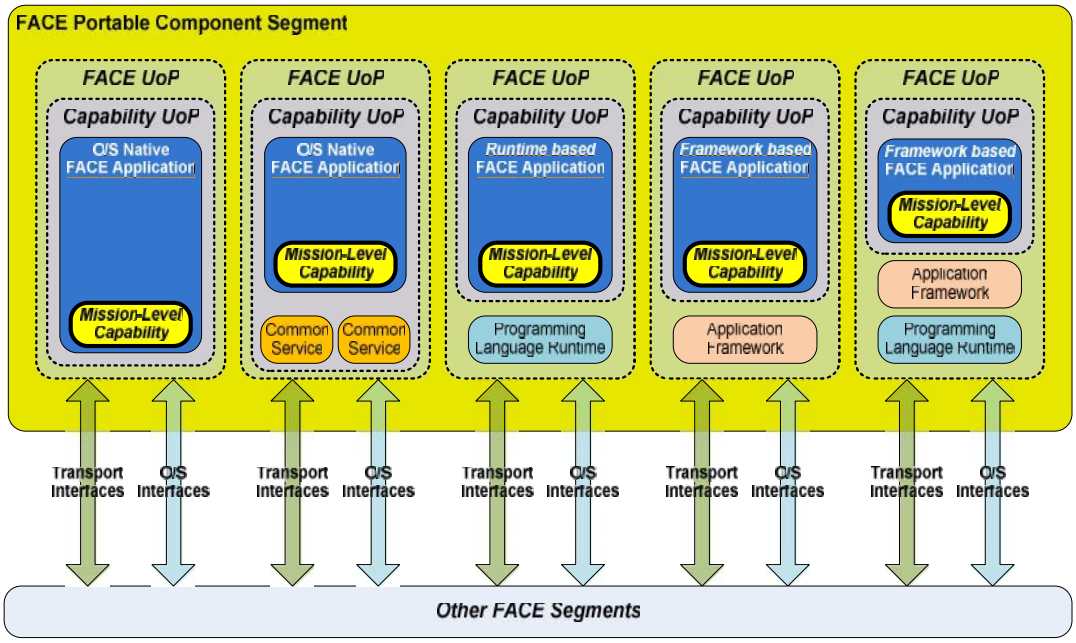


Figure 25: Examples of Application Packaging

Mission-level capabilities are present in all applications; however, the packaging inherent in their instantiation varies. Figure 25 depicts five packaging options for mission-level capabilities in deep blue and yellow colored rectangles. From left to right in this depiction, the capabilities are packaged within their C-UoP as follows:

1. A traditional compiled application that is capable of executing natively on an operating system
2. A traditional compiled application which is capable of executing natively on a FACE operating system and also uses PSS Common Services

3. An application that requires one or more programming-language based run-time executives in order to execute

4. A set of components that execute on an application framework

5. A set of components that execute on an application framework that itself executes on a programming language run-time

These packaging option examples depicted above do not represent the exhaustive set of packaging options allowed in FACE environments. The packaged mission-level capability defines the C-UoP boundary. C-UoPs exist within F-UoPs and are also depicted in Figure 25. From left to right in this diagram the F-UoPs contain the following:

1. A C-UoP, which is a traditional compiled application that is capable of executing natively on an operating system. In this case, the C-UoP and the F-UoP have identical contents.

2. A C-UoP, which is identical in type to the previous example, but this C-UoP differs in that it uses common services, but does not interface to them using the Transport Interface. In such cases, the C-UoP and the common services together as a set comprise the content of F-UoP.

3. A C-UoP, which is written in a programming language that requires a run-time executive to execute. It also contains the programming language-based run-time executive required to run the C-UoP as part of the F-UoP.

4. A C-UoP, which is written as a set of components to an application framework. It also contains the application framework which is required to run the C-UoP as part of the F-UoP.

5. A C-UoP, and application framework identical to the previous example excepting that an additional programming language run-time executive is also present because the application framework was written in a language that requires such a run-time executive.

UoP-Related Requirements

FACE UoPs (F-UoPs):

1. Shall use no interfaces traversing the F-UoP boundary, other than the Transport Interface and the Operating System Interfaces

2. Must provide mission-level capability when they are designated as an application – *see Note 16*

3. Must provide a well-defined function when they are designated as a service

4. Shall include all software to provide the advertised capability (i.e., run-time layers and application frameworks) – *see Note 9*

Capability UoPs (C-UoPs):

1. Shall be restricted to using the following interfaces for all data interchanges that traverse the C-UoP boundary:

   — The Transport Interface

— The FACE Operating System Interfaces

— Standards-based interfaces for programming language-based run-time executives

— Standards-based interfaces for application frameworks

2. Must provide one or more mission-level capabilities when designated as an application (this requirement does not apply to common services hosted in the Portable Component Segment)

3.8.4.2 Bundling of F-UoP Content

In all cases where either:

- Application-to-Application linkages or dependencies exist, or
- Application-to-"Common Services" linkages or dependencies exist, and

in either of these cases where the Transport Interface is not used as the sole communication mechanism for all communications between them, then that set of applications or applications and services are considered to be a single UoP and shall be delivered, tested, maintained, and certified concurrently as if they were a single deliverable.

3.8.4.3 Fractional UoP Portability Protection

The fact that UoPs are a set of software brings into question precisely how much, or which parts, of the F-UoP's internals are portable.

The C-UoP shall be portable in conformance with the FACE definition of application portability. The C-UoP is the portion of the F-UoP whose portability is desired owing to its encapsulation of the desired capability, ConOps, displays formats, display symbologies, workflows, etc. The FACE definition of application portability permits recompilation of the C-UoP source code for the target environment as well as the recompilation or replacement of linkable libraries which are appropriate for the target computing environment. S*ee Note 9*.

Protection of the portability of those portions of an F-UoP that are not included in the C-UoP is not mandated. In most cases, alternate implementations of standards-based run-time executives and application frameworks are designed and built to execute on the target computing platform are available commercially.

A non-exhaustive set of examples based upon the packaging options in Figure 25 are provided in Table 2.

Examples (below)	**Portability Protected Code and Required Tasks**	**Code not Required to be Portable and Required Tasks**
Operating System native FACE application	The portability of the Mission-Level Capability is desired, so a re-compilation of that source code for the desired target computing platform may be indicated.	
	Portability of third-party libraries	Portability of third-party libraries

Examples (below)	Portability Protected Code and Required Tasks	Code not Required to be Portable and Required Tasks
	which supply mission-level capabilities is desired which indicates the need for a recompilation of the source code if available.	which supply mission-level capabilities is desired. If the third-party vendor has a different version of the libraries which meet the same API, but are already compiled for the target computing platform, then select the target appropriate library vendor.
Operating System native FACE application using FACE common services which are not accessed using the Transport Interface	Include Rows 1 and 2 from above.	Include Rows 1 and 2 from above.
	The portability of common services is protected. In this example, the Transport Interface is not the exclusive means of communication between the C-UoP and the common services, so they are treated as if they were an internal part of the C-UoP.	Alteration of the C-UoP's access mechanism to the common services from an internal strategy to one that uses the Transport Interface has the effect of moving the common services outside of the C-UoP.
Run-time-based FACE application	Include Rows 1 and 2 from above.	Include Rows 1 and 2 from above.
	Recompilation may not be necessary if Java is used.	The portability of the programming language run-time is not desired. Select an implementation of the run-time which is appropriate for the target platform.
Framework-based FACE application	Include Rows 1, 2, and 6 from above.	Include Rows 1, 2, and 6 from above.
		The portability of the application framework is not desired. Select an implementation of the application framework which is appropriate for the target platform.
Framework-based FACE application requiring a run-time	Include Rows 1, 2, 6, and 8 from above.	Include Rows 1, 2, 6, and 8 from above.

Table 2: Examples of Fractional UoP Protection

3.8.4.4 Inter-UoP and Intra-UoP Communications

To support modularity, ease integration and curtail proliferation of proprietary interfaces between UoPs; all communications between UoPs shall exclusively use the Transport Interfaces as depicted below.

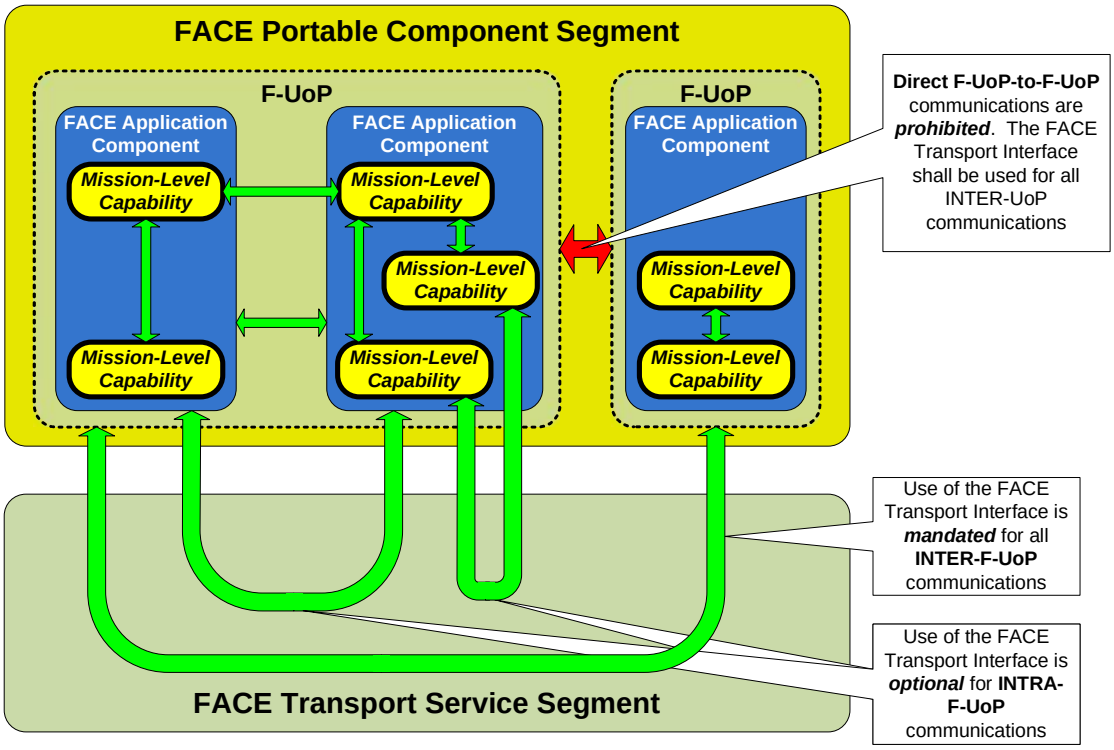


Figure 26: Inter-UoP & Intra-UoP Communications

Communication between the internal contents of an F-UoP need not use the Transport Interface. However, the use of the Transport Interface as a communication mechanism between software entities within a FACE UoP is permitted. If the Transport Interface is selected, a communications mechanism between software entities within a FACE UoP, it need not be used as the exclusive intra-UoP communication mechanism. Allowing mixed intra-UoP communication mechanisms facilitates the migration of existing vendor product lines towards higher modularity through the phased decoupling of monolithic applications into several more cohesive UoPs.

3.8.5 FACE Application Interfaces to Other FACE Entities

FACE data, both consumed and produced by applications, traverses the Transport Services Segment. Refer to Figure 27 for the remainder of this topic. Applications access the Transport Services Segment by means of the Transport API (see Appendix C). The Transport Interface API is implemented by a library, which is linked to the application's core.

To summarize the use of the Transport Interface API, the application creates a connection to a FACE data source or sink destination by a name assigned to that destination by means of the transport configuration files/mechanism. The transport library then opens a data pathway to the desired destination software entity and returns a connection ID. Assuming that the connection is bi-directional, the application uses the transport API and the connection ID to send or poll for data at the destination. The application may also register one of its subroutines as a handler for callbacks from the destination when polling is not used. See Appendix C for a complete

description of the Transport Interface API. In the event of conflict between this introduction and Appendix C, Appendix C shall be deemed to be correct. *See Note 9.*

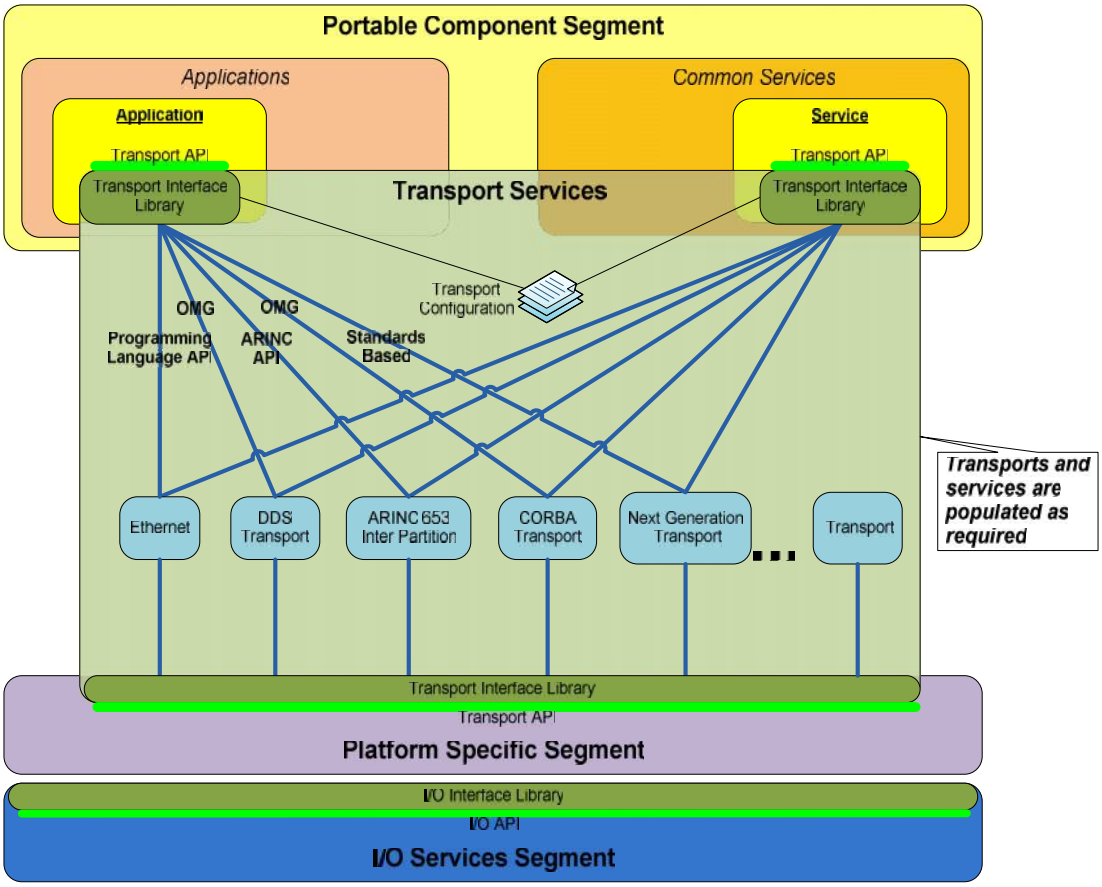


Figure 27: Platform-Specific Services Segment Example

3.8.6 FACE Application Interfaces to Non-FACE Entities

Non-Face Entity to be Integrated with FACE	Governance
Libraries	Libraries shall adhere to one of the following rules: • Be portable – *see Note 17* • Be re-compilable from source code, to be able to execute on the target FACE computing environment – *see Note 17* • Use standard APIs or interfaces such that another implementation that does run on the target FACE computing environment may be substituted, thereby enabling the capability to be portable to other FACE platforms It is highly desirable that the APIs provided by replacement libraries fully conform to the APIs used by the application and the libraries they replace.

Non-Face Entity to be Integrated with FACE	Governance
Supporting sub-systems	Libraries shall adhere to one of the following rules – *see Note 16*: • Be portable • Be re-compilable from its source code, to be able to execute on the target FACE computing environment • Use standard APIs or interfaces such that another implementation that does run on the target FACE computing environment may be substituted, thereby enabling the capability to be portable to other FACE platforms It is highly desirable that the APIs provided by replacement libraries fully conform to the APIs used by the application and the libraries they replace. Examples include but are not limited to: • Programming language run-time environments • Application frameworks
Mission-level capability	A FACE-conformant "wrapper" providing full encapsulation of the non-FACE capability is required and the capability is to be considered to be a non-portable library. FACE applications will use the "wrapper" (*aka* proxy) exclusively as a means to access the non-FACE capability.

Table 3: FACE Application Interfaces to Non-FACE Entities

3.8.7 Application Configuration

To prevent the development of many nearly identical capabilities, applications shall support the configuration of run-time behavior whenever they are started or restarted. Configuration services make appropriate sets of configuration information available to the applications during initialization. This implementation is a loosely-coupled way to configure services and applications in an extensible fashion. Configuration information for applications, their supporting run-times, frameworks, etc. are also considered to be parts of the Portable Component Segment. Refer to Section 3.12 for a complete treatment of application configuration.

3.8.8 Security Requirements

The set of security requirements levied against applications varies by FACE profile. See Section 4 for a complete treatment of application security requirements.

3.8.9 Safety Requirements

The set of safety requirements levied against applications varies by FACE profile. See Section 5 for a complete treatment of application safety requirements.

3.8.10 Application Bundling

In all cases where dependencies Application-to-Application or Application-to-Common Services exist and the Transport Interface is not used as the sole communication mechanism between them, then that set of applications and services are considered to be a single F-UoP and shall be delivered, tested, maintained, and certified concurrently as if they were a single deliverable.

3.8.11 Deliverable Requirements

3.8.11.1 Application Characterization

Application characterization is a quantified description of the *Unit of Portability* (UoP) for the purpose of facilitating integration without exposing the internal design of the deliverable. To that end, the UoPs are treated as *black boxes* in as much as the externally observable parameters and operationally relevant details are described. As such, enumerations of provided capabilities, parameterization and configurability of those capabilities, as well as instructions on how to use and integrate the Computer Software Configuration Items (CSCIs) will be described in such a way that assists the integration effort. The characterization documentation should not divulge the implementation, or how such capability was achieved in respect of IP rights. Every F-UoP shall be accompanied with a statement including the following:

1. Type of UoP:
   - — Examples include: traditional application, run-time application, framework application – *see Note 18*
   - — Run-time and framework-based capabilities shall also include the standard, including version of the run-time and framework on which it was designed/tested.
2. Size of the UoP as a program:
   - — Supplier shall report the non-volatile storage requirements for executables and data at rest
   - — Shall include the specific configuration used when collecting the size metric
   - — May include a separate measurement of file size of configuration information
3. An enumeration with a brief description of the mission-level capabilities provided by the UoP
4. A list of CSCIs required to achieve the advertised mission-level capabilities
5. An enumeration and description of the external interfaces to the UoP
6. A description of configurable parameters, a description of their meaning, permissible values and their meaning
7. Provide the Application IRS; i.e., data representation and data binning
8. Provide Application timing dependencies (i.e., output rate, input rate and other data dependencies); these shall include the specific configuration used when collecting the performance metric
9. Provide CPU utilization (e.g., scheduling requirements); this shall include the specific configuration used when collecting the performance metric
10. Provide networking requirements; i.e., TCP, UDP
11. Provide transport mechanism requirements (i.e., Pub/Sub, Request/Response)

3.8.12 Logging

Applications are free to choose their own local logging system. Centralized logging shall conform to requirements stipulated in Section 3.10. *See Note 9.*

3.8.13 Graphics

FACE ensures the interoperability of the graphical output from FACE applications across a wide range of platform display systems to protect the relevance of FACE applications, as they are re-deployed. It is expected that FACE will be deployed on a wide range of platforms with a diversity of display hardware. The hardware and firmware are constraints to FACE, because the scope of FACE does not extend to hardware. *See Note 6.*

FACE specifies the output formats of:

- ARINC 661-4 due to its recommendation in DO-297 (IMA) and wide adoption
- X11 Byte Stream for its relationship to OpenGL and the wide adoption of both
- ARINC 739-1 and ARINC 739A-1 for their ability to interface with MCDUs

See Section 3.11 for a detailed description of graphics rendering for FACE applications.

3.8.14 User Input Devices

The integrator is responsible for combining the graphical output from multiple FACE and potentially non-FACE sources onto a partitioned display surface (screen). The integrator is also free to supply logical layers as a display methodology. The responsibility to marshal keyboard, cursor, bezel, etc. user input back to the appropriate application should rest with the integrator supplied software that manages the display surface partitioning and provides layer management.

3.9 Operating System Interface: FACE Operating System API Standards

FACE operating systems shall provide POSIX and ARINC 653-based operating environments and API subsets.

FACE operating system APIs shall correspond to the FACE profiles – *see Note 14*:

1. Provide Operating System API standards-based interfaces for FACE General Purpose, Safety, and Security Profiles.
2. Include Ada and C language bindings corresponding to the specific FACE profiles. C++ and Java language bindings may be provided through other standards.

Appendix A details the POSIX API (and API characteristics) supported for each FACE profile.

Each partition shall host applications designed for the FACE Operating System API.

Each partition may use a different operational environment (POSIX or ARINC 653).

A computing platform may simultaneously support partitions of different FACE Operating System APIs profiles and operational environments. The FACE Standard recognizes that the APIs associated with the General Purpose, Safety, and Security Profiles provide divergent capabilities. The divergence of these capabilities may prevent, or represent considerable safety or security risks to, including safety and security risks) simultaneous hosting of General Purpose, Safety, and Security profiled applications on the same computing platform. When system requirements include applications of different FACE profiles, the use of multiple computing platforms may be required. Other architectural capabilities (e.g., virtualization), if supported by the processor and operating system selected by the system integrator, may provide other means to simultaneously host on the same computing platform applications that are using different FACE profiles.

Based on the profile, FACE operating systems will provide support for POSIX sockets and ARINC 653 sampling/queuing ports to be used for inter-partition communications (i.e., the lower-layer transport mechanism that other FACE standardized interface software libraries may be developed around).

A General Purpose profiled application, when communicating with Safety and Security profiled applications, will use the same restricted inter-partition communications as the Safety and Security profiled applications. A General Purpose profiled application, if communicating to another General Purpose profiled application, may use additional sockets services (if supported by the transport mechanism).

For Safety and Security Operating System API implementations, FACE operating systems shall include capabilities to:

1. Send messages on POSIX sockets and receive the same messages in another partition using POSIX sockets.
2. Allow POSIX partitions to:
   - — Use ARINC 653 sampling and queuing ports for inter-partition communications with ARINC 653 partitions
   - — Use ARINC 653 Health Monitoring APIs to communicate with an ARINC 653 Health Monitor
3. Send messages on an ARINC 653 port and receive the same messages in another partition using an ARINC 653 port.
4. Ensure messages are received in the order they are sent.
5. Ensure delivery of available messages.
6. Ensure complete messages are received (no silently truncated messages).
7. Block on an empty socket (receive) or a full socket (transmit).
8. Specify which partitions are specifically authorized to bind/connect to an endpoint, receive on an endpoint, or transmit on an endpoint.
9. Configure and get the status of a socket:

— Specify the size of a socket's receive buffer (space reserved for the socket) and a socket's transmit buffer

— Specify and enforce a socket's maximum receive and transmit message size

— Error status

3.9.1 FACE Security Operating System API

The FACE Security Operating System shall support ARINC 653 and POSIX.

The FACE Security Operating System API for ARINC 653 shall consist of:

- ARINC 653 Part 1: All services associated with Avionics Application Software Standard Interface Part 1 – Required Services.
- Services associated with the following categories of Avionics Application Software Standard Interface Part 2 – Extended Services:

  — Memory Blocks

- All other Part 2 services are intentionally excluded.

A Security Operating System API for POSIX is currently under development by The Open Group MILS API Working Group (within the RTES Forum). FACE shall define a Security API in alignment with the MILS API. It will be compatible with a time and space partitioned operating environment, and will include subsets of APIs from the following categories – *see Note 19*:

- Subsets of APIs from the following POSIX.13 Profile 51 service categories:

  — C Language Math

  — C Language Support

  — Threads

  — Thread Attributes and Priority Scheduling

  — Mutexes

  — Shared Memory Objects

  — Signals and Real-time Signals

  — Semaphores

  — Timers

  — Single-Process

- Subset of POSIX APIs associated with:

  — Sockets

- ARINC 653 Sampling and Queuing Port APIs (when communicating with an ARINC 653-based application)

All other POSIX APIs are intentionally excluded, including POSIX_DEVICE_IO. Security profiled applications will communicate through TSS to other partitions/segments that perform device I/O. *See Note 20*.

Selection of POSIX APIs for the FACE Security Operating System API will take into account the following principles – *see Note 21*:

- Minimize exposure surface for analysis and certification, especially when alternatives are available (e.g., single process; use partitions to support multiple applications on the same computing platform). *See Note 9*.
- Support APIs that include required memory resources as an input over APIs that use internal memory pools (i.e., prevent potential information leakage due to resource re-use by portions of the application that handle different security levels). *See Note 9*.
- Support APIs that will not result in memory fragmentation. *See Note 9*.

3.9.2 FACE Safety Operating System API

The FACE Safety Operating System shall support ARINC 653 and POSIX.

The FACE Safety Operating System API for ARINC 653 shall consist of:

- ARINC 653 Part 1: All services associated with Avionics Application Software Standard Interface Part 1 – Required Services.
- Services associated with the following categories of Avionics Application Software Standard Interface Part 2 – Extended Services:
  - — File System
  - — Sampling Port Extensions
  - — Memory Blocks

All other services, including Logbooks, are explicitly excluded (File System services can be used instead of Logbooks).

The FACE Safety Profile for POSIX will be made up of two sub-profiles – *see Note 22*:

- Base sub-profile – *see Note 22*:
  - — Consists of the essential APIs needed to create safety-related software capabilities. *See Note 22*.
- Extended sub-profile – *see Note 22*:
  - — Contains the Base sub-profile – *see Note 22*
  - — Provides an extended API set allowing more advanced software concepts – *see Note 22*:

- Multi-process – *see Note 22*
- TCP networking – *see Note 22*
- Memory allocation and de-allocation – *see Note 22*

In the Extended sub-profile some APIs may have safety-related considerations, including – *see Note 22*:

- Some APIs are prohibited by other safety-related software standards (e.g., MISRA C, aircraft manufacturer directed software standards). *See Note 22*.
- Some APIs have associated side-effects (e.g., state of child process following *fork* API call) that limit what other software capabilities can be safely invoked. *See Note 22*.
- Significant behavior differences, as permitted by the POSIX specification that, when not accounted for by an application, result in safety impacts when ported to another computing platform. *See Note 22*.
- Behavior differences, as permitted by the POSIX specification, that may limit the availability of resources allocated to a partition (e.g., limits in resource recovery when invoking POSIX close/delete APIs). *See Note 22*.
- Behavior differences, as permitted by the POSIX specification, as to where resources are allocated from. *See Note 22*.

Note that some of these considerations may be applicable to the Base sub-profile APIs.

Conformance to the FACE Safety Profile for POSIX will allow for two options:

- Conformance to the Base sub-profile
- Conformance to the Extended sub-profile (includes the Base sub-profile)

The FACE Safety Profile for POSIX shall consist of – *see Note 22*:

- Subsets of APIs (supported subset may be different than the FACE Security Operating System API) from the following POSIX.13 Profile 53 service categories – *see Note 22*:

  — C Language Math – *see Note 22*

  — C Language Support – *see Note 22*

  — Threads – *see Note 22*

  — Thread Attributes and Priority Scheduling – *see Note 22*

  — Mutexes – *see Note 22*

  — Shared Memory Objects – *see Note 22*

  — Message Queues – *see Note 22*

  — Signals and Real-time Signals – *see Note 22*

  — Semaphores – *see Note 22*

— Timers – *see Note 22*

— Single-Process – *see Note 22*

— Multi-Process – *see Note 22*

— Spawn – *see Note 22*

— File System – *see Note 22*

— Device IO – *see Note 22*

- Subset of APIs associated with – *see Note 22*:

  — Sockets – *see Note 22*

- ARINC 653 Sampling and Queuing Port APIs (when communicating with an ARINC 653-based application) – *see Note 22*

All other POSIX APIs are intentionally excluded.

The selection of POSIX APIs for the FACE Safety Operating System API considered the following principles:

- Minimize exposure surface for certification (i.e., affordability), especially when alternatives are available (e.g., single process; use partitions to support multiple processes on the same computing platform).
- Support APIs that include required memory resources as an input over APIs that use internal memory pools (i.e., minimize coupling and lower the complexity).
- Support APIs that will not result in memory fragmentation.

3.9.3 FACE General Purpose Operating System API

The FACE General Purpose Operating System shall support POSIX only, or POSIX and ARINC 653.

The FACE General Purpose Operating System API for ARINC 653 shall consist of:

- All services associated with Avionics Application Software Standard Interface Part 1 – Required Services.
- Services associated with Avionics Application Software Standard Interface Part 2 – Extended Services, except:

  — Logbooks (File System services can be use instead of Logbooks)

  — Multiple Module Schedules (not a capability that should be allocated to a General Purpose application)

The FACE General Purpose Operating System API for POSIX shall consist of – *see Note 23*:

- APIs associated with POSIX Profile 53, except for:

— POSIX Trace APIs

- Additional APIs associated with:

  — Wide character support

  — POSIX_FILE_ATTRIBUTES

- ARINC 653 Sampling and Queuing Port APIs (when communicating with an ARINC 653-based application)

Shell and shell utility support (part of POSIX Profile 54) has been excluded from the FACE General Purpose Operating System API for POSIX. FACE application and component code shall *not* be dependent on the existence of Shell and Shell utilities (even if optionally supported by an Operating System implementation). *See Note 9.*

3.9.4 Time Management

System time will be maintained in accordance with the appropriate standards defined by the Operating System Interfaces for the appropriate FACE Operating System API sets. *See Note 8.*

As specified in Section 3.9, all FACE Operating System API sets will support POSIX clocks and timers and ARINC 653 Part 1 Time Management Services. *See Note 24.*

Partial support for the POSIX CLOCK_REALTIME, which permits a calendar-based clock time to be set that becomes available to all partitions. Due to safety and security-related considerations in the POSIX definitions of this clock's behavior, the configuration data will specify which partitions (preferably one partition) have permissions to set (using the *clock_settime* API) this clock. All partitions may have access to read this clock, but an error will be returned for any partition that attempts to set an alarm based on this clock that is based on absolute time (setting of an alarm based on a relative time is supported). Partitions authorized to set the calendar time will also be authorized to set the time zone value that is visible to all other partitions.

3.10 Logging Services

Logging services are optional; however, the ability to store software detected events and conditions for review at a later time are supported by a FACE logging system. Logging Services will fall into one of two categories – *see Note 6*:

- Centralized
- Local

3.10.1 Centralized Logging

The logging service is hosted in the Platform Common Services Sub-segment of the PSS Segment.

Centralized logging services in a given system must be implemented using the Transport Services API with the message data format based on RFC 5424: The Syslog Protocol. The

logging services shall be accessible by all segments of the FACE architecture. This service may be used by the system integrator to log faults, platform data, and more. Health Monitor Logging shall be accomplished as described in Section 3.13.

3.10.2 Local Logging

Software developers may choose a local logging method.

3.11 Graphics Services

FACE Graphics Services shall support data flows that conform to the OpenGL, ARINC 739-1 and 739A-1, and ARINC 661-4 Standards. The OpenGL Standard is a widely adopted set of standards supporting 2D and 3D graphics imaging. The ARINC 739 Standards support alphanumeric displays associated with avionics Multi-purpose Control and Display Units (MCDUs). The ARINC 661 Standard provides a higher-level abstraction of the display subsystem whose implementation is dependent on using the display system's infrastructure for primitive rendering. *See Note 6.*

This section describes the three display frameworks and the underlying standards supported by the FACE architecture.

The choice of graphics services used by an application (i.e., OpenGL, ARINC 661, and ARINC 739) are the responsibility of the application developer. The inclusion of the necessary supporting display infrastructures (e.g., GL/EGL/GLX Services, ARINC 661 Services, and ARINC 739 Services) to support the system applications are the responsibility of the platform integrator.

3.11.1 OpenGL – 2D and 3D Imaging

OpenGL is a widely adopted group of specifications defining a cross-language, cross-platform API for writing applications that produce 2D and 3D computer graphics. The OpenGL specifications and available implementations routinely support imaging and display on local graphics hardware as well as remote graphics hardware operating in a networked environment. In addition, well supported OpenGL specifications and implementations exist for operation in the FACE General Purpose and Safety Profiles. No OpenGL specification has been defined with consideration for security. It is anticipated that an OpenGL specification appropriate for safety will be appropriate for use for the Security Profile. *See Note 6.*

The OpenGL specifications are classed into the following broad capabilities:

- Functional and interface descriptions supporting 2D and 3D image rendering
- Window creation and management
- Remote display support

3.11.1.1 OpenGL Use in the General Purpose Profile

The following specifications shall define the OpenGL functional behavior and APIs for the General Purpose Profile.

Function	Specification
2D & 3D Graphics Rendering and API	OpenGL Graphics System: A Specification (Version 1.2.1), Khronos Group (OpenGL 1.3)
Window Management for Local Display Support	OpenGL ES Native Platform Graphics Interface (Version 1.1), Khronos Group (EGL 1.1)
Window Management and Remote Display Support	OpenGL Graphics with the X Window System (Version 1.3), OpenGL (GLX 1.3)
X11 Byte Stream Encoding	GLX Extensions for the OpenGL Protocol Specification (Version 1.3)

Table 4: OpenGL Functional Behavior Specifications

There are a several standards supported variations of OpenGL, each with its own set of associated versions. The OpenGL version selected for this profile was chosen to maximize application portability between the Safety and the General Purpose Profiles, without significantly impacting imaging capabilities in an avionics environment. Software application portability between the FACE profiles is enhanced by the fact that OpenGL has a safety-critical specification discussed in upcoming sections.

Note that window creation and management can be performed using either EGL or GLX Standards. Within the avionics environment, both have substantial support and can co-exist. EGL is a smaller command set, limited to local rendering only, within a single window. Due to its more limited capabilities, EGL is simpler to implement, making it the preferred solution for safety certifiable solutions, which generally operate in a more limited context.

Conversely, GLX provides both local and remote window management capabilities based on the X11 Standard and is widely available in the workstation and embedded marketplace. GLX provides functional support for multiple applications to render images within independent windows on the same display. GLX also provides the functional support for indirect rendering (drawing on a network connected display). GLX supports this capability by encapsulating OpenGL commands into serialized OpenGL command lists, transporting those command lists to the remote display engine (the GLX Server), and then executing those command lists on the remote target display. The state-of-the-art in the embedded industry is to provide a subset of GLX which supports essential window creation and remote rendering capabilities and a translation layer to convert GLX windowing commands into EGL windowing commands for execution on the target display. This results in a simpler and lighter-weight implementation than the public reference implementation based on X11.

Figure 28 provides a visual representation of the relationship of the various OpenGL components in both direct and indirect rendering environments. In the direct rendering case (the left path), the Graphics application uses either EGL (to draw to the full screen window) or GLX to create multiple display windows and then writes into those windows using OpenGL functions. Since the window was created for a directly connected display (using a NULL or local network address), the OpenGL commands are passed directly to the graphics drivers.

In the indirect rendering case (the right data path), the Graphics application uses the network support within GLX to create a display window (with the network address of the target display) and then writes into that window using OpenGL functions. Since the display window was

created for a non-local target display, the OpenGL commands are serialized and transmitted as packets to the remote display network location. When received by the target display manager, those commands are de-serialized and passed to the local display drivers.

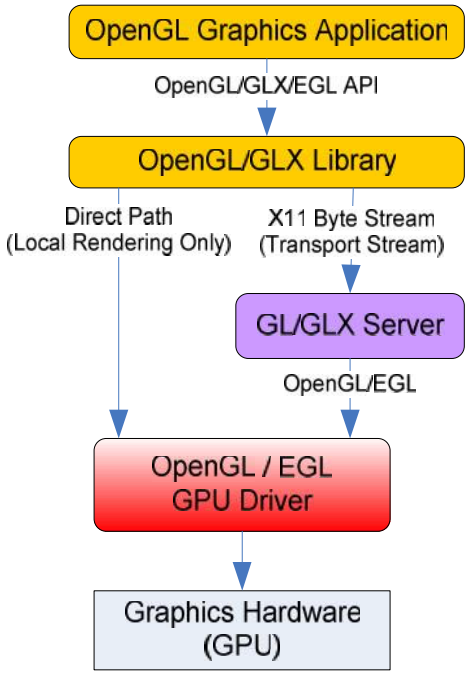


Figure 28: OpenGL Rendering Component Relationship

3.11.1.2 OpenGL Use in the Safety Profile

The FACE Safety Profile supports a subset of the graphics commands and capabilities of the General Purpose Profile. The FACE Safety Profile is based around the OpenGL Safety-Critical specification, developed specifically to address safety-critical environments. The OpenGL Safety-Critical specification was defined as a subset of essential services included in the OpenGL 1.3 specification. The resulting specification is effectively a subset of the OpenGL 1.2.1 specification used as the basis of the General Purpose Profile. Commercial implementations exist from multiple vendors and have been certified at Level A within the DO-178B process.

The following specifications shall define the OpenGL functional behavior and APIs for the Safety Profile.

Function	**Specification**
2D & 3D Graphics Rendering and API	OpenGL SC Safety-Critical Profile Specification (Version 1.0.1), Khronos Group (OpenGL SC)
Window Management – *see Note 9*	OpenGL ES Native Platform Graphics Interface (Version 1.0), Khronos Group (EGL 1.1)
Remote Display Support – *see Note 9*	OpenGL Graphics with the X Window System (Version 1.3), OpenGL (GLX 1.3)

Table 5: OpenGL Safety Profile Specifications

Currently DO-178B, Level A certified implementations exist only for OpenGL SC and EGL supporting direct rendering. No DO-178B, Level A standardized implementation exists supporting full GLX or indirect (remote) rendering. However, the existing GLX Standard could be modified and standardized to select the subset of GLX services essential for remote rendering. This small subset of the GLX Standard could serve as the safety-critical standard supporting indirect graphics rendering and should be a topic for future standardization.

3.11.1.3 OpenGL Use in the Security Profile

The FACE Security Profile uses the same standards and specifications as the Safety Profile.

Indirect rendering, however, may be affected by the security policy enforced by the underlying network. Commands which require input from the OpenGL server (remote display) would need to be deprecated from the supported commands. This appears to have limited impact on the OpenGL commands set, impacting primarily one command: *glReadPixel*().

3.11.2 ARINC 739-1 and 739A-1

ARINC 739 defines an interface between a system and a Multi-Purpose Control and Display Unit (MCDU). The interface is a message-oriented description and protocol which shall use FACE Transport Services.

The following specifications shall define the ARINC 739 functional behavior and protocol. Development of an application interface specification is deferred for a future release of the FACE Standard.

Function	Specification
Single MCDU Specification	ARINC 739-1: Multi-Purpose Control and Display Unit (MCDU)
Dual MCDU Specification	ARINC 739A-1: Multi-Purpose Control and Display Unit (MCDU)

Table 6: ARINC 739 Functional Behavior Specifications

Figure 29 provides a visual representation of the relationship of the various ARINC 739 components in a generic rendering environment. In this case, the Graphics application uses an ARINC 739 Library to create an ARINC 739 command stream which is passed (potentially over a network) to the ARINC 739 Server. The ARINC 739 Server then transmits the commands to the MCDU using the ARINC 739 protocol using the ARINC 429 or other FACE interface driver.

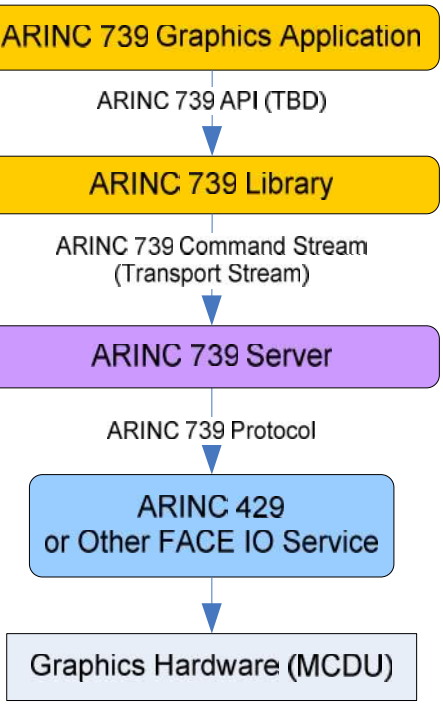


Figure 29: ARINC 739 Component Relationships

The General Purpose, Safety, and Security Profile implementations of this graphics interface are identical.

3.11.3 ARINC 661-4

The ARINC 661-4 Standard defines the cockpit display using a markup language similar in style to XML. The cockpit display is compiled into a standard binary Definition File (DF) which is interpreted by an embedded run-time, the Cockpit Display System (CDS). The CDS generates the display using the graphics infrastructure available on the host system. For FACE this graphics infrastructure would be OpenGL or ARINC 739.

The following specifications shall define the ARINC 661-4 functional behavior and protocol.

Function	Specification
Cockpit Display System Interface and Functional Specification	ARINC 661-4: Cockpit Display System Interfaces to User System

Table 7: ARINC 661-4 Functional Behavior Specifications

The General Purpose, Safety, and Security Profile implementations of this graphics interface are identical.

Figure 30 provides a visual representation of the relationship of the various ARINC 661 components in a generic rendering environment. In the case of ARINC 661, the CDS is the primary controlling mechanism, which executes a pre-compiled binary image representing the ARINC 661 Graphics application. The resulting commands are then transmitted (potentially over

a network) to the ARINC 661 server which translates the commands to the local rendering infrastructure.

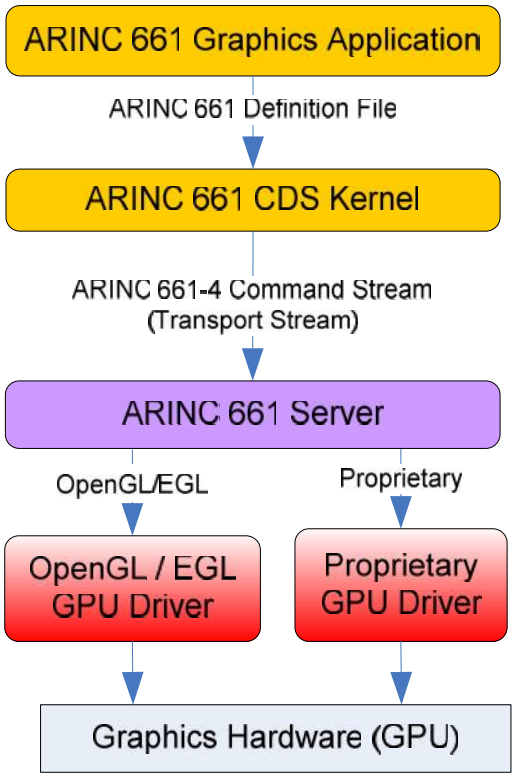


Figure 30: ARINC 661 Component Relationships

3.11.4 Relationship to FACE Architecture

The imaging components described above are placed in a FACE context using the following FACE architecture diagram, modified to include the described graphics components. Graphics applications (e.g., App1, App2, and App3), and their associated graphics libraries (e.g., OpenGL, ARINC 739, and ARINC 661 CDS) are placed in the FACE Portable Component Segment. These applications act as graphics clients, dependent on services provided by their respective graphics servers, which reside in the Graphics Services Sub-segment of the PSS Segment, as seen in Figure 31. The graphics servers are similarly dependent on the local host infrastructure. For OpenGL, the local graphics hardware is accessed using an OpenGL driver and EGL window management API. For ARINC 661, the server is responsible for integration with the local graphics infrastructure. For ARINC 739, the MCDU hardware is accessed using the ARINC 429 (or other FACE-conformant) driver services available using the I/O Services Segment.

Where the graphics client and servers reside within the same FACE partition, the clients and servers are free to communicate using their native mechanisms. For instance, local OpenGL clients may access the OpenGL (GPU) driver directly for maximum efficiency, bypassing the GLX server component entirely. In the situation above, this direct access mode is supported due to the intrinsic bypass capabilities provided by the OpenGL and GLX Standards and the ready availability of COTS implementations supporting this access mode.

Where the graphics client and servers reside on different FACE partitions, the client and server are required to communicate using the FACE Transport Services using their associated Standard streaming protocols (e.g., X11 Byte Stream, ARINC 739, and ARINC 661 Command Stream). *See Note 25*.

Where graphics frameworks contend for common hardware – for instance, GLX and ARINC 661 servers contending for a common display – it is the responsibility of the integrator to resolve the contention within the Graphics Services infrastructure.

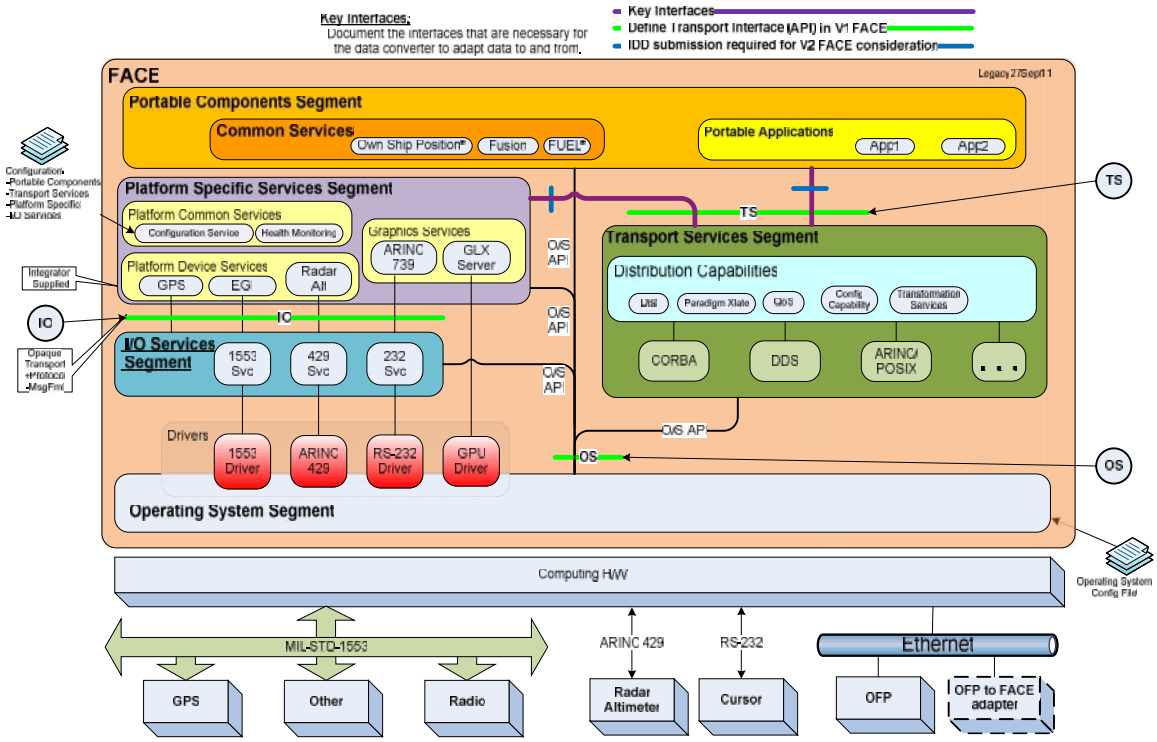


Figure 31: Display Components in FACE Architecture Context

3.12 Configuration Services

The overarching goals of FACE are to reduce development and integration costs as well as time to field. Configuration supports these goals by increasing the flexibility of software and simplifying software integration and maintenance tasks. In order to enable the integration of FACE-conformant software components into multiple environments without modification, the FACE Standard provides for Configuration, as listed below.

1. A FACE-conformant environment must be capable of providing FACE-conformant software components with configuration information during initialization of the software component, and at other times as required.

2. FACE-conformant software components must use this configuration information, as required, to initialize parameters of the software component, or to enable or disable certain software component capabilities.

3. The parameters or capabilities which are configurable shall be defined in the specification for each FACE-conformant software component. Minimum schema data element requirements for configuration format are listed below:

   — Data elements and structure are defined using XSD:

      - Descriptive information

   — Data elements are constrained using XSD:

      - Data types
      - Limits
      - Precision
      - Valid values

   — XML Schema Version

4. Configuration capabilities may be required for software components in a FACE-conformant system, including but not limited to the following:

   — Portable Application Components

   — Common Services

   — Transport Services

   — Platform-Specific Services

   — I/O Services

   — Device Drivers

   — Operating System

   — Partitions

   — Software Components required for Transport Interface or I/O Interface

5. Configuration shall be supported under the following circumstances:

   — Configuration at Build or Compilation Time:

      - Changing these values requires re-compile and changes the executable.

   — Configuration at Initialization Time:

      - The executable does not change.
      - The values are loaded during the Initialization phase.

   — Configuration at Execution Time (post init/run-time):

      - The executable does not change.
      - The values are loaded during the Execution phase.

3.12.1 Centralized Configuration Service

A FACE-conformant system shall provide a Centralized Configuration Service. The primary purpose of the Centralized Configuration Service is to enable sharing of configuration information among system components and services. The Centralized Configuration Services shall be developed in accordance with the requirements identified below:

1. The Centralized Configuration Service must be capable of providing configuration information for the following FACE software components when that configuration information needs to be shared between multiple components:

   — Portable Components and Common Services

   — Transport Services

   — Platform-Specific Services

   — I/O Services

   — Device Drivers

   — Operating System – *see Note 9*

      - Partitions – *see Note 9*

   — Software Component libraries required for Transport Interface or I/O Interface

2. Communication with the Centralized Configuration Service shall be available on the Transport Interface using Transport Services.

3. Communication between the I/O Services and the Centralized Configuration Service shall use the I/O Interface.

4. The Centralized Configuration Service shall be a Platform Common Service. *See Note 9.*

5. The Centralized Configuration Service shall provide a mechanism for a software component to make configuration information available to other applications.

6. The Centralized Configuration Service shall provide a mechanism for a software component to publish configuration information required by the application. *See Note 16.*

7. The Centralized Configuration Service shall provide a mechanism for a software component to identify specific elements of the configuration information that are to be shared.

8. A FACE-conformant system shall provide a mechanism for a software component to identify specific elements of the configuration that are private to the application, not to be shared with other applications or services.

9. Each piece of configuration information published by a software component shall be identified as shared (available to all components) or private.

10. The Centralized Configuration Service shall require each specific element of configuration information to be identified with a unique name.

— A FACE-conformant environment shall ensure that all applications agree on the value of each uniquely named, shared, configuration item.

11. The Centralized Configuration Service shall provide a mechanism for referencing configuration information published by other components.

12. The Centralized Configuration Service shall provide a mechanism for retrieving a uniquely named portion of a configuration information set.

13. In a FACE-conformant environment, the messages required for communication with the Centralized Configuration Service using the Transport Interface or I/O Interface shall be identical.

14. Communication with Centralized Configuration Services shall be through the Transport or I/O Interfaces.

15. A FACE-conformant system shall validate all configuration information against a Common Configuration Schema.

16. A FACE-conformant Centralized Configuration Service shall provide a mechanism to load an XSD file representing the Common Configuration Schema.

17. A FACE-conformant Centralized Configuration Service shall provide a mechanism to retrieve an XSD file representing the Common Configuration Schema.

3.12.2 Common Configuration Schema Requirements

1. The FACE Standard shall provide minimum data element requirements for a Common Configuration Schema. *See Note 9.*

2. The Common Configuration Schema shall be provided as an XSD.

3. The Common Configuration Schema shall define configuration elements provided and required by each software component.

4. The Common Configuration Schema shall support definition of hierarchies of configuration information.

5. The Common Configuration Schema shall support definition of software component-specific configuration sub-schemas.

6. Each FACE-conformant software component shall provide schema information for required configuration data.

   — The required configuration data shall be validated against the minimum schema data elements defined in Section 3.12.

3.13 Health Monitoring and Fault Management

The purpose of the FACE Health Monitoring and Fault Management (HMFM) component is to provide standardized methods for detecting, reporting, and handling faults/failures within the scope of a single system/platform. The ultimate objective of HMFM is maintaining service

availability of FACE-conformant components when faults and errors are detected. The basic goals of HMFM include:

- Detecting and handling run-time faults and errors
- Identifying suspect compute resources associated with a fault
- Providing the flexibility to enable system designers to engage the appropriate level of HMFM capabilities given the desired design objectives

These faults and errors can arise from any FACE component. Since HMFM itself may be the source or victim of a fault, HMFM's own availability requirement is also imposed upon itself.

HMFM functionality may be provided as a component or group of components spanning the Operating System Segment, PSS Segment, and/or Portable Component Segment (as a Common Service). The specific HMFM components within a given system will depend on platform requirements.

3.13.1 Introduction

The essence of a typical HMFM capability can be framed around the fault management cycle state-machine depicted in Figure 32. The goal is to eliminate run-time service outage through system awareness by HMFM of the occurrence of run-time faults and failures. Upon detection of a run-time fault, the entire Fault Management (FM) cycle is engaged to direct the system back to a healthy state, by a series of pre-configured FM policies. Given a start state of a healthy system, we can follow the FM state machine diagram as follows:

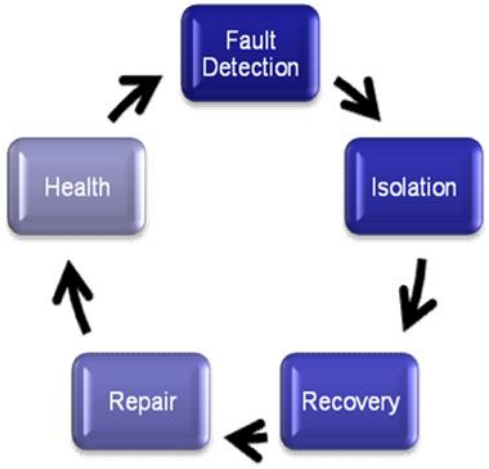


Figure 32: Fault Detection and Recovery

- Fault detection state – a fault is realized or declared by HMFM as some defined fault criteria has been met (e.g., through health monitoring or heartbeat rules) and thus promoted to "fault" status. Such faults can originate from any segment within the scope of a system.
- Fault isolation state – HMFM will determine the scope of a fault given the particular entity or location of the fault. *See Note 6.*

- Fault recovery state – HMFM will engage a fault recovery policy that drives the system to maintain a state of continued service availability. *See Note 6.*
- Fault repair state – if possible and so configured, HMFM will restore faulted resources back to a healthy state and then re-integrate those computing assets back into the system.
- Health – the optimal state of system functionality, with functional HMFM operational.

Parts or all of these HMFM supported fault management cycle capabilities may be relevant to a particular design.

3.13.2 HMFM Requirements

The following are high-level HMFM capabilities which address the service scope, functional capabilities, and configuration properties. These capabilities are required for the Safety and Security Profiles, and optional for the General Purpose Profile.

- Integrator shall define a set of fault criteria and fault management attributes. Consideration should be given to existing standards such as the Service Availability Forum (SAF) and ARINC 653 Health Monitoring.
- HMFM shall be configured at system start-up.
- HMFM shall provide active monitoring capabilities to detect faults and failures.
- HMFM shall execute response or recovery actions for detected faults and failures as configured.
- HMFM shall use the FACE Transport Interface or the I/O Interface transport mechanisms for communications with HMFM clients, but will use the Transport Services API data model definition to describe the data. *See Note 9.*
- HMFM configuration may be administratively viewable at run-time.
- HMFM may support one or more redundancy models as fault recovery and avoidance mechanisms.
- HMFM may support the “repair” option, such that HMFM attempts to resurrect failed or faulted resources.
- HMFM may monitor/manage the instantiation and termination of components.
- HMFM may generate alarms and notifications to indicate internal state transitions experienced by the components.

3.13.3 Operating System Segment Health Monitoring Requirements

The Operating System Segment will include Health Monitor (HM) capabilities at the module-level (i.e., computing platform level), partition-level, and process-level (i.e., thread-level). This includes capabilities to start and terminate the module, restart and idle partitions within the module, and halt and restart threads.

1. For ARINC 653 applications, health monitoring shall be based on capabilities and configuration information defined in ARINC 653 Part 1.

2. For POSIX applications, health monitoring shall be based on capabilities and configuration information (or a subset of capabilities and configuration information) defined in ARINC 653 Part 1. POSIX terminology is substituted in place of ARINC 653 terminology when dealing with threads and processes.

   — The data types and ARINC 653 health monitoring services will be available to a POSIX-based partition.

      - REPORT_APPLICATION_MESSAGE
      - CREATE_ERROR_HANDLER
      - GET_ERROR_STATUS
      - RAISE_APPLICATION_ERROR

   — For the GET_ERROR_STATUS service, the POSIX thread ID will be reported for the FAILED_PROCESS_ID.

Note Services such as REPORT_APPLICATION_MESSAGE may be achieved through Transport Services Interface messages.

3.14 Internet Standards Description

This section identifies the IP-based network standards and profiles required to promote interoperability between IP-based network applications. The goal of this section is to identify a minimum RFC set of network standards, consistent with existing DoD profiles, to ensure maximum interoperability between FACE applications. A detailed table of Internet Standards is described in Appendix D.

A common network infrastructure is defined, based on the following DoD standards and tailored to the FACE platform environment.

- DoD Joint Technical Architecture
- DoD IPv6 Standard Profiles for IPv6-Capable Products
- A Profile for IPv6 in the US Government

The DoD standards above define overlapping and somewhat inconsistent groups of mandatory requirements for network standards. The defined standards broadly divide into IPv4 requirements and IPv6 requirements which provides a natural separation of the requirements into two independent network profiles. IPv4 networks represent the majority of the existing embedded network platforms currently in service, while IPv6 networks represent the emerging standards for future platforms. Network standards associated with a particular profile are identified by a (✓) symbol in the IPv4 and/or IPv6 columns of the Inter-Network Standards Summary Table.

The decision to support an IPv4 only or an IPv6 only network is a program-specific decision; however, if the decision is made to support an IP network, then all mandatory FACE Standards associated with that capability must be supported.

In keeping with the goal to identify a minimum set of network standards, only mandatory DoD requirements were considered for inclusion in the FACE specification. Where multiple mandatory options or conflicting standards were specified for identical functions, two strategies were employed to choose a unique selection for the FACE Standard. When presented with multiple options, preference was given to Internet standards common between DoD standards. Similarly, when presented with multiple options, preference was given to Internet standards commonly supported by existing RTOS implementations.

Within each of the IPv4 and IPv6 network profiles there exists a set of mandatory core network and routing infrastructure standards which shall be supported to ensure proper network operation and interoperability at the socket interface layer defined by POSIX. The Inter-Network Standards Summary Table summarizes the network standards required for FACE conformance of network host or router devices operating in an Internet environment.

To reduce the obligation of including unnecessary network protocols, most application-level protocols are deprecated from mandatory to optional packages. These are identified within the network standards tables as optional requirements (O) with a unique named package name such as NTP or DHCP. Inclusion of an optional capability is a program-specific decision; however, if the decision is made to support a specific network capability, then all of the standards associated with that capability must be supported.

Finally, the network requirements can be further divided into the following categories representing the different roles network devices play within a network:

- Host: Any network node whose primary purpose is to support network protocols that are the source and/or destination of IP layer communication.
- Router: A network node that forwards network packets with the purpose of interconnecting distinct sub-networks. Routers are distinct from hosts in that they are normally not the destination of application data traffic.

Internet nodes acting in the capacity of a Host or Router need implement only those requirements marked in the Host or Router column of the Inter-Network Standards Summary Table.

4 Security

As the DoD network connectivity expands and the need to manage various classification levels of data within and between aircraft increases, security and Information Assurance (IA) become critical considerations for future avionics developments. Although information assurance is pervasive across avionics systems, the FACE Security Profile only applies to the specific software components that actively enforce or contribute to meeting the platform security requirements. The following sections provide basic security considerations respected by a FACE-conformant implementation.

4.1 Scope

Security in the context of FACE does:

- Recognize that Procuring Agencies and their designated approval authorities define user community-specific requirements, guidelines, and processes for addressing systems, security, and software lifecycle processes to achieve Authority to Operate
- Provide context for processes that capture best principles to be used in developing “high assurance” security software
- Address re-use of FACE software components developed to specific security levels
- Enable support for lower security levels
- Specify applicable API sets for each segment comprising the computing platform to facilitate certifiability to a target security level
- Provide recommendations for artifacts to be created, managed, and maintained for security-related components and associated system security architecture, facilitating low-cost re-use

Security in the context of FACE does not:

- Prescribe specific systems, security, and software lifecycle processes, such as DIACAP, NIACAP, or equivalent
- Assert that a component, segment, or application is certified to some evaluation assurance level (only a subset of the system security and design assurance requirements required to achieve Authority to Operate is addressed by the FACE Standard)
- Assert that a single vendor provide all security components that comprise the segments of the FACE computing environment

4.2 Guiding Concepts

The sections below provide guidance and recommendations on developing FACE security-relevant software components. Security-relevant software executes within or supports the FACE Security Profile framework. However, it differs from other FACE profiles in that it also contributes or influences the security policy of the FACE system. More specifically, a FACE software component that executes within the FACE Security Profile but has no mechanism to alter or otherwise impact the security policy is not considered security-relevant. The following sections include discussions on security function separation and common criteria. These guiding concepts are critical to reducing the time and cost to field secure solutions to the warfighter and are applicable to the entire software lifecycle and air platform stakeholders.

4.2.1 Separation and Partitioning of Security Functions

The goal of separation and partitioning is to minimize the recurring cost and schedule impacts of certifications and accreditations required for Authority to Operate of new systems through similarity and visibility of information architecture and documentation. To support this goal, FACE provides a Capability Environment, though not independently certified and accredited, that will support the process of acquiring an Authority to Operate through the use of sound security engineering practices, component type certification, and the clear representation of those information flows, and associated controls relevant to the environment. *See Note 1*.

This includes enforcing the following (NEAT) characteristics:

- (N) Non-bypassable: Security functions cannot be circumvented.
- (E) Evaluatable: Security functions are small enough and simple enough for mathematical verification.
- (A) Always Invoked: The policies of each security function are enforced each and every time.
- (T) Tamper-proof: Subversive entities cannot alter the security data or functions.

In order to support security-critical functions the infrastructure supports the following (TIME) characteristics:

- (T) Type-Safe: All messages, buffers, APIs, caches, pointers, URLs, SQL commands, and network inputs are checked for correct content and protected from buffer overflow, uninitialized content, and mode confusion.
- (I) Infiltration: Information is only accepted from an architecturally authorized and controlled source. Processing must not be affected by information from outside of the function's architecturally authorized operational environment. *See Note 1*.
- (M) Mediation: The effect of a subject's execution depends only on resources with which it is allowed to interact. A subject will only provide information to a destination object that is specially authorized to receive that information from its source. *See Note 1*.
- (E) Exfiltration: Information is only provided to an architecturally authorized and controlled destination.

Any security-relevant function shall be physically or virtually separated from all other applications and capabilities within the environment. *See Note 1*.

4.2.2 Common Criteria

The Common Criteria enable an objective evaluation to validate that a particular product or system satisfies a defined set of security requirements. Although the focus of the Common Criteria is evaluation, it presents a standard that is of interest to those who develop security requirements.

The Common Criteria (CC) was developed through a combined effort of six countries: the United States, Canada, France, Germany, the Netherlands, and the United Kingdom. This effort built on earlier standards, including Europe's Information Technology Security Evaluation Criteria (ITSEC), the United States Trusted Computer System Evaluation Criteria (TCSEC), and the Canadian Trusted Computer Product Evaluation Criteria (CTCPEC). The Common Criteria is an international standard (ISO/IEC 15408) for computer security. A Common Criteria evaluation allows an objective evaluation to validate that a particular product satisfies a defined set of security requirements. The focus of the Common Criteria is evaluation of a product or system, and less on development of requirements. Nevertheless, its evaluation role makes it of interest to those who develop security requirements. The Common Criteria allow for seven Evaluation Assurance Levels (EALs), which will be discussed further. *See Note 1*.

Functional and assurance security requirements are the basis for the Common Criteria. There are seven Evaluation Assurance Levels (EALs). The higher the level, the more confidence you can have that the security functional requirements have been met. The levels are as follows:

- **EAL1: Functionally Tested**: Applies when you require confidence in a product's correct operation, but do not view threats to security as serious. An evaluation at this level should provide evidence that the target of evaluation functions in a manner consistent with its documentation and that it provides useful protection against identified threats.
- **EAL2: Structurally Tested**: Applies when developers or users require low to moderate independently assured security but the complete development record is not readily available. This situation may arise when there is limited developer access or when there is an effort to secure legacy systems.
- **EAL3: Methodically Tested and Checked**: Applies when developers or users require a moderate level of independently assured security and require a thorough investigation of the target of evaluation and its development, without substantial re-engineering.
- **EAL4: Methodically Designed, Tested, and Reviewed**: Applies when developers or users require moderate to high independently assured security in conventional commodity products and are prepared to incur additional security-specific engineering costs.
- **EAL5: Semi-Formally Designed and Tested**: Applies when developers or users require high, independently assured security in a planned development and require a rigorous development approach that does not incur unreasonable costs from specialist security engineering techniques.

- **EAL6: Semi-Formally Verified Design and Tested**: Applies when developing security targets of evaluation for application in high-risk situations where the value of the protected assets justifies the additional costs.
- **EAL7: Formally Verified Design and Tested**: Applies to the development of security targets of evaluation for application in extremely high-risk situations, as well as when the high value of the assets justifies the higher costs.

Further discussion will be provided about the objective levels of Basic Robustness (EAL 2), Medium Robustness (EAL 4-5), and High Robustness (EAL 6-7) in the guidance documents. *See Note 1*.

4.3 FACE Security Profile

The FACE Security Profile as described in Section 2.7 is a constrained subset of the full FACE Standard that specifically addresses the requirements of security-relevant software. Specific details on the APIs and other restrictions imposed by the FACE Security Profile are provided in other sections of this document and are not repeated herein.

The FACE Security Profile also addresses functional and performance attributes that define a computing environment that can support multiple levels of security processing requirements. To be more specific, security considerations exhibit the following key operational environment attributes:

- Non-Interference between applications
- Non-Interference between applications and the operational environment
- Non-Interference between applications and external interfaces
- Security Function Isolation
- Security Function Recovery
- Security Function Safe Failure
- Trusted Information Flow between applications
- Trusted Information Flow between applications and the external interfaces
- Protection of Data in Processing
- Protection of Data in Transit
- Protection of Data at Rest

Since Confidentiality, Integrity, and Availability are system-level characteristics, they are beyond the scope of this Standard.

It is important to reiterate that the FACE Security Profile is defined for all the FACE Segments: Operating System, Device Driver, I/O Services, Transport Services, PSS, and Portable

Component Segment. For each of the segments, the Security Profile limits the set of APIs to those that are robust and necessary to develop security-related software components.

The FACE Security Profile includes:

- A set of APIs for each FACE Segment. This set is a subset of all APIs supported by the FACE General Purpose Profile.
- A set of attributes necessarily required for high integrity software (e.g., QoS, non-interference, isolation, robust behavior)
- Robust time and space partitioning requirements

4.4 FACE Security Boundary

Although information assurance applies to the entire data network, for the initial release of the FACE Standard the security boundary applies only to the FACE computing environment and applications encapsulated within a single air platform. This Standard specifies the software components required for creating portable secure applications. Any particular implementation of the security functions of the FACE environment is not specified in the initial release of the FACE Standard. Implementations that impact application portability may be specified in future editions of FACE. Furthermore, the FACE security boundary does not extend to the communication terminals or devices used to exchange information to a wider network. Figure 33 provides a notional high-level context diagram of a computing environment that processes one or more levels of secure data and is intended to clarify the FACE security boundary.

The colors of the Sensor and Processing blocks within this context diagram indicate different security categories: single-level if solid, and multi-level if blended. The blue boxes indicate boundary processing, and the yellow boxes at the bottom indicate aircraft communications equipment outside of the FACE context. The figure also shows several cyber defense functions that may or may not be included in a particular context, depending on the risk to the system. The labeling shown could be for internal use in a Multi-Level environment; for example, Commercial Internet Protocol Security Option (CIPSO) or Common Architecture Label IPv6 Security Option (CALIPSO). If the environment does not require labeling, then it would not have to be applied/removed as files or messages enter/leave the environment. Cryptographically bound labels to message or files for integrity checking or other purposes might also be needed.

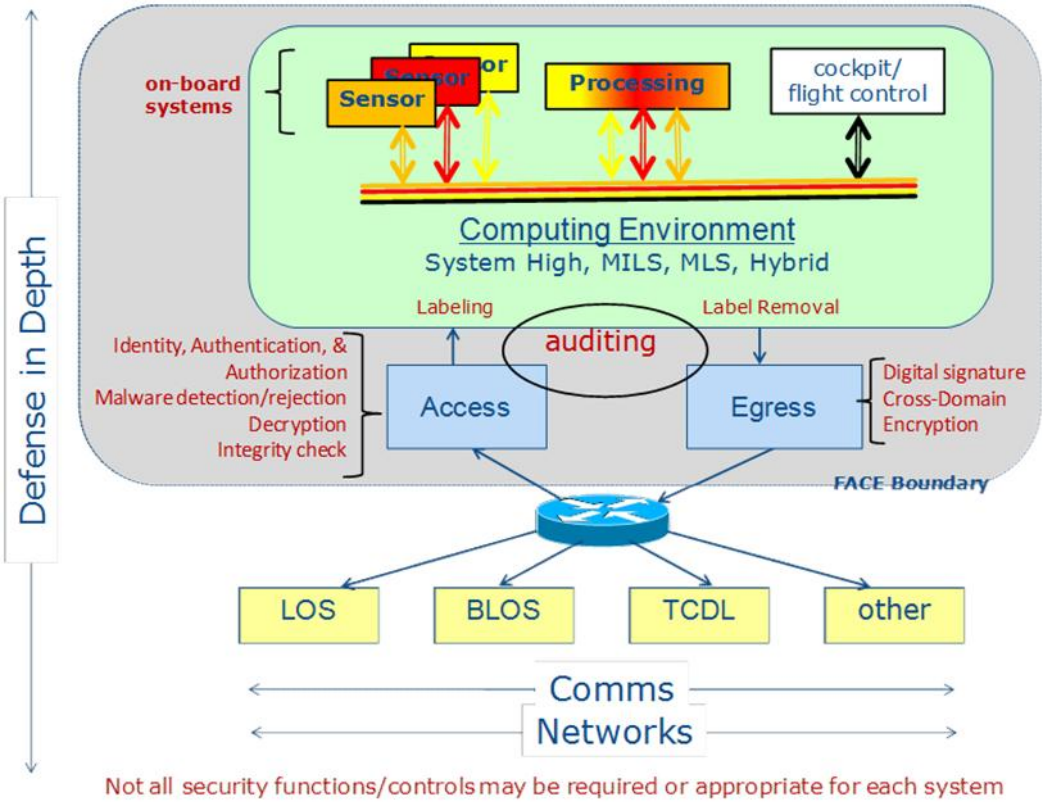


Figure 33: FACE Security Boundary

5 Safety Assurance

Software engineering has become a significant part of the overall engineering of modern safety-critical systems, ranging from weapons systems to avionics and other vehicle control systems. Many of these complex systems consist of combinations of developmental, legacy Government-Off-The-Shelf (GOTS), Commercial-Off-The-Shelf (COTS), open source, and other non-developmental software used to monitor and control physical processes, the failure of which could lead to injury, loss of human life, or other catastrophic mishap. The airborne systems fielded on military and civilian platforms need to meet rigorous airworthiness standards specified by the respective Defense and Federal airworthiness authorities. This is achieved through robust systems safety processes and design assurance rigor. For the purposes of this document, safety-relevant software components are defined as those that require the computing infrastructure of the Safety Profile. These components depend upon all layers of the FACE architecture from operating system to application.

5.1 Scope

Safety assurance in the context of FACE does:

- Recognize that Procuring Agencies and their designated airworthiness authorities define the user community-specific requirements, guidelines, and processes for addressing systems, safety, and software lifecycle processes to achieve airworthiness
- Use DO-178B design assurance principles for nomenclature purposes and to outline a candidate approach to safety assurance if DO-178B processes were to be mandated by the procuring agency
- Provide context for processes that capture best principles to be used in developing “high assurance” safety software
- Address re-use of FACE software components developed to a specific safety level
- Enable support for meeting any given safety level
- Specify applicable API sets through the Safety Profile for use by each segment comprising the computing platform to facilitate certifiabilty to a specific safety level
- Provide recommendations for artifacts to be created, managed, and maintained for safety-related components and associated system safety architecture
- Facilitate low-cost re-use

Safety assurance in the context of FACE does not:

- Prescribe specific systems, safety, and software lifecycle processes, such as DO-178B, ARP 4754A, ARP 4761 or equivalent

- Assert that a component, segment, or application is certified to some safety level (only a subset of the system safety and design assurance requirements required to achieve airworthiness is addressed by the FACE Standard)
- Assert that any single vendor provide all safety components that comprise the segments of the FACE computing environment

The following sections provide guiding concepts, requirements, and pertinent standards and references that are recommended for FACE safety-relevant software component development.

5.2 Guiding Concepts

Since the primary goal of FACE is to bring more capabilities to the warfighter in less time and at a lower cost, safety-relevant software also provides a mechanism for reducing the cost and schedule associated with integration and certification. While the following list is not exhaustive, software component separation and development processes are critical to enabling re-usable safety-relevant software. These guiding safety concepts are applicable to the entire software lifecycle and air platform stakeholders.

5.2.1 Separation and Partitioning of Safety Functions

FACE will provide a Capability Environment, though not independently certified and evaluated, which will support the process of acquiring an Airworthiness Release through the use of sound safety engineering practices, component qualification, integration testing, and the clear representation of the relationships and sources of information required by safety-critical operation. *See Note 1*.

To meet this objective, a FACE-conformant implementation shall be independent of the underlying safety infrastructure. This does not preclude support of safety-critical capabilities or safety-relevant functions operating as applications within the FACE implementation. *See Note 1*.

5.2.2 Process

Process descriptions and terminology within this section are based on DO-178B: Software Considerations in Airborne Systems and Equipment Certification, as published by RTCA, Inc. Development of DO-178B was a joint effort with EUROCAE who publish the document as ED-12B. When specified by the Technical Standard Order (TSO) for which design authorization is sought, the FAA references DO-178B as the document it uses for guidance to determine whether the software will perform reliably in an airborne environment. Along with DO-178B, process requirements and guidance are derived from ARP 4754A: Guidelines for Development of Civil Aircraft and Systems, and ARP 4761: Guidelines and Methods for Conducting the Safety Assessment Process on Civil Airborne Systems and Equipment. As noted earlier, the governing processes for any procurement will be specified by procuring agencies and their airworthiness authorities. *See Note 1*.

The Development Assurance Level (DAL), as defined by DO-178B, is determined from the safety assessment process and hazard analysis by examining the effects of a failure condition in the system. The failure conditions are categorized by their effects on the aircraft, crew, and passengers.

- **Catastrophic**: Failure may cause a crash.
- **Hazardous**: Failure has a large negative impact on safety or performance, or reduces the ability of the crew to operate the aircraft due to physical distress or a higher workload, or causes serious or fatal injuries among the passengers.
- **Major**: Failure is significant, but has a lesser impact than a Hazardous failure (for example, leads to passenger discomfort rather than injuries).
- **Minor**: Failure is noticeable, but has a lesser impact than a Major failure (for example, causing passenger inconvenience or a routine flight plan change).
- **No Effect**: Failure has no impact on safety, aircraft operation, or crew workload.

The implementation of safety attributes in a design may require achievement of qualification objectives beyond those described in DO-178B.

The number of objectives to be satisfied (some with independence) is determined by the software level. In this context, "independence" refers to execution of qualification tasks by an agency separate from the software development team. This can be achieved by an outside agency, an independent verification entity within a company, or, in some cases, through use of automated tools.

Safety-relevant software components shall be developed with a process aligned to a specific software DAL. While the portable components executing within the Safety Profile may be developed using the appropriate DAL, the software infrastructure components that establish the computing environment need to be designed using processes equivalent to or exceeding the highest DAL of the Portable Application components they service. *See Note 1*.

A re-usable software component will be certified as part of the certification of an encompassing system; however, it must have its own set of software lifecycle data. *See Note 1*.

Any components in a safety partition shall have artifacts for certification at the level of the partition. *See Note 1*.

Safety evaluation may require configuration management of all software lifecycle artifacts including requirements, design, code, test reports, and supporting documents. This requirement will be driven by the Certification Authority. *See Note 1*.

5.3 FACE Safety Profile

The FACE Safety Profile as described in Section 2.7 is a constrained subset of the full FACE Standard that specifically addresses the requirements of safety-critical software. Specific details on the APIs and other restrictions imposed by the FACE Safety Profile are provided in other sections of this document.

- Non-Interference between applications
- Non-Interference between applications and the operational environment
- Non-Interference between applications and external interfaces

- Software Fault Isolation
- Software Fault Recover
- Reliable Information Flow between applications
- Reliable Information Flow between applications and the external interfaces
- Integrated health monitoring and management
- Standards-based system configuration data

It is important to reiterate that the FACE Safety Profile is defined for all the FACE Segments: Operating System, Device Driver, I/O Services, Transport Services, PSS, and Portable Components. For each of the segments, the Safety Profile limits the set of APIs to those that are deterministic and necessary to develop safety-related software components.

The FACE Safety Profile includes:

- A set of APIs for each FACE Segment; this set is a subset of all APIs supported by the FACE General Purpose Profile
- A set of attributes required for high integrity software (e.g., QoS, non-interference, isolation, deterministic behavior)
- Robust time and space partitioning requirements

6 Conformance

Defining conformance and creating a Certification Policy and Program for this Technical Standard is the next step in establishing an effective FACE Standard. Without the associated conformance criteria and certification processes, there is no assurance that a supplier has developed or implemented products or solutions according to the approved Technical Standard.

Certification provides formal recognition of conformance to an industry Technical Standard, which allows:

- Suppliers and practitioners to make and substantiate clear claims of conformance to a Technical Standard
- Buyers to specify and successfully procure from vendors who conform to and provide solutions that conform to the Technical Standard

Following the approval of this Technical Standard, the FACE Consortium will work with The Open Group to establish conformance criteria and define an associated Certification Program for this Technical Standard. Conformance assessment is the act of determining the conformance of an implementation to a specification, or the adherence of a business operation to a best practice or process definition. There are many techniques for assessing such conformance, including the use of a standardized test method, quality assessment by industry experts, and vendors' claims of conformance made within a defined legal framework. The techniques to be used will be chosen during the process of defining the Certification Program. *See Note 1*.

Following implementation of the Certification Program, practitioners wishing to have their implementations certified as conformant to the Technical Standard will be able to apply for certification, at which time a conformance assessment will be performed. *See Note 1*.

A Operating System POSIX Interface

This normative appendix describes the FACE profiles based on the IEEE Std 1003.1-2008. The Operating System Segment and Operating System Interface are described in Section 3.1 and Section 3.9, respectively.

Explanation for POSIX Call Table Format

INC. Included in this Profile indicated at top of column.

EXC. Excluded from this Profile, but included in POSIX PSE51 and PSE52.

Blank Excluded from this Profile, but included in POSIX PSE53 and PSE54.

Note: Excluded and Blank items may be included in future editions of FACE.

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
posix_fadvise()					_POSIX_ADVISORY_INFO
posix_fallocate()					_POSIX_ADVISORY_INFO
posix_memalign()					_POSIX_ADVISORY_INFO
aio_cancel()				INCL	_POSIX_ASYNCHRONOUS_IO
aio_error()				INCL	_POSIX_ASYNCHRONOUS_IO
aio_fsync()				INCL	_POSIX_ASYNCHRONOUS_IO
aio_read()				INCL	_POSIX_ASYNCHRONOUS_IO
aio_return()				INCL	_POSIX_ASYNCHRONOUS_IO
aio_suspend()				INCL	_POSIX_ASYNCHRONOUS_IO
aio_write()				INCL	_POSIX_ASYNCHRONOUS_IO
lio_listio()				INCL	_POSIX_ASYNCHRONOUS_IO
pthread_barrier_destroy()					_POSIX_BARRIERS and _POSIX THREADS
pthread_barrier_init()					_POSIX_BARRIERS and _POSIX THREADS
pthread_barrier_wait()					_POSIX_BARRIERS and _POSIX THREADS

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
pthread_barrierattr_destroy()					_POSIX_BARRIERS and _POSIX THREADS
pthread_barrierattr_init()					_POSIX_BARRIERS and _POSIX THREADS
pthread_barrierattr_ getpshared()					_POSIX_BARRIERS, _POSIX_THREADS, AND _POSIX _THREAD_PROCESS_SHARED
pthread_barrierattr_setpshared()					_POSIX_BARRIERS, _POSIX_THREADS AND _POSIX_ THREAD_PROCESS_SHARED
clock_nanosleep()	EXCL	EXCL	EXCL	INCL	_POSIX_CLOCK_SELECTION
pthread_condattr_getclock()	INCL	INCL	INCL	INCL	_POSIX_CLOCK_SELECTION and _POSIX_THREADS
pthread_condattr_setclock()	INCL	INCL	INCL	INCL	_POSIX_CLOCK_SELECTION and _POSIX_THREADS
clock_getcpuclockid()				INCL	_POSIX_CPUTIME
fsync()	EXCL	INCL	INCL	INCL	_POSIX_FSYNC
msync()		EXCL	EXCL	INCL	_POSIX_MAPPED_FILES and _POSIX_SYNCHRONIZED_IO
mmap()	INCL	INCL	INCL	INCL	_POSIX_MAPPED_FILES or _POSIX_ SHARED_MEMORY_OBJECTS or _POSIX_TYPED_MEMORY_OBJECTS
munmap()	EXCL	EXCL	EXCL	INCL	_POSIX_MAPPED_FILES or _POSIX _SHARED_MEMORY_OBJECTS or _POSIX_TYPED_MEMORY_OBJECTS
mlockall()	EXCL	EXCL	EXCL	INCL	_POSIX_MEMLOCK
munlockall()					_POSIX_MEMLOCK
mlock()	EXCL	EXCL	EXCL	INCL	_POSIX_MEMLOCK_RANGE
munlock()	EXCL	EXCL	EXCL	INCL	_POSIX_MEMLOCK_RANGE
mprotect()				INCL	_POSIX_MEMORY_PROTECTION
mq_close()		EXCL	EXCL	INCL	_POSIX_MESSAGE_PASSING
mq_getattr()		INCL	INCL	INCL	_POSIX_MESSAGE_PASSING
mq_notify()		INCL	INCL	INCL	_POSIX_MESSAGE_PASSING
mq_open()		INCL	INCL	INCL	_POSIX_MESSAGE_PASSING
mq_receive()		INCL	INCL	INCL	_POSIX_MESSAGE_PASSING
mq_send()		INCL	INCL	INCL	_POSIX_MESSAGE_PASSING

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
mq_setattr()		INCL	INCL	INCL	_POSIX_MESSAGE_PASSING
mq_unlink()		EXCL	EXCL	INCL	_POSIX_MESSAGE_PASSING
mq_timedreceive()		INCL	INCL	INCL	_POSIX_MESSAGE_PASSING and _POSIX_TIMEOUTS
mq_timedsend()		INCL	INCL	INCL	_POSIX_MESSAGE_PASSING and _POSIX_TIMEOUTS
sched_getparam()		EXCL	INCL	INCL	_POSIX_PRIORITY_SCHEDULING
sched_getscheduler()		EXCL	INCL	INCL	_POSIX_PRIORITY_SCHEDULING
sched_setparam()		EXCL	INCL	INCL	_POSIX_PRIORITY_SCHEDULING
sched_setscheduler()		EXCL	INCL	INCL	_POSIX_PRIORITY_SCHEDULING
posix_spawnattr_ getschedparam()				INCL	_POSIX_PRIORITY_SCHEDULING and _POSIX_SPAWN
posix_spawnattr_ getschedpolicy()				INCL	_POSIX_PRIORITY_SCHEDULING and _POSIX_SPAWN
posix_spawnattr_ setschedparam()				INCL	_POSIX_PRIORITY_SCHEDULING and _POSIX_SPAWN
posix_spawnattr_ setschedpolicy()				INCL	_POSIX_PRIORITY_SCHEDULING and _POSIX_SPAWN
sched_yield()	INCL	INCL	INCL	INCL	_POSIX_PRIORITY_SCHEDULING or _POSIX_THREADS
sched_get_priority_max()	INCL	INCL	INCL	INCL	_POSIX_PRIORITY_SCHEDULING, _POSIX_THREADS and _POSIX _THREAD_PRIORITY_SCHEDULING
sched_get_priority_min()	INCL	INCL	INCL	INCL	_POSIX_PRIORITY_SCHEDULING, _POSIX_THREADS and _POSIX_ THREAD_PRIORITY_SCHEDULING
sched_rr_get_interval()	EXCL	EXCL	INCL	INCL	_POSIX_PRIORITY_SCHEDULING, _POSIX_THREADS and _POSIX_ THREAD_PRIORITY_SCHEDULING
sigqueue()	INCL	INCL	INCL	INCL	_POSIX_REALTIME_SIGNALS
sigtimedwait()	INCL	INCL	INCL	INCL	_POSIX_REALTIME_SIGNALS
sigwaitinfo()	INCL	INCL	INCL	INCL	_POSIX_REALTIME_SIGNALS
sem_close()	INCL	INCL	INCL	INCL	_POSIX_SEMAPHORES
sem_destroy()	EXCL	EXCL	INCL	INCL	_POSIX_SEMAPHORES
sem_getvalue()	INCL	INCL	INCL	INCL	_POSIX_SEMAPHORES
sem_init()	INCL	INCL	INCL	INCL	_POSIX_SEMAPHORES

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
sem_open()	INCL	INCL	INCL	INCL	_POSIX_SEMAPHORES
sem_post()	INCL	INCL	INCL	INCL	_POSIX_SEMAPHORES
sem_trywait()	INCL	INCL	INCL	INCL	_POSIX_SEMAPHORES
sem_unlink()	EXCL	EXCL	INCL	INCL	_POSIX_SEMAPHORES
sem_wait()	INCL	INCL	INCL	INCL	_POSIX_SEMAPHORES
sem_timedwait()	INCL	INCL	INCL	INCL	_POSIX_SEMAPHORES and _POSIX_TIMEOUTS
shm_open()	INCL	INCL	INCL	INCL	_POSIX_SHARED_MEMORY_ OBJECTS
shm_unlink()	EXCL	EXCL	EXCL	INCL	_POSIX_SHARED_MEMORY_ OBJECTS
posix_madvise()					_POSIX_SHARED_MEMORY_OBJECTS, _POSIX_MAPPED_FILES, AND POSIX_ADVISORY_INFO
posix_spawn()			INCL	INCL	_POSIX_SPAWN
posix_spawn_file_actions_ addclose()				INCL	_POSIX_SPAWN
posix_spawn_file_actions_ adddup2()				INCL	_POSIX_SPAWN
posix_spawn_file_actions_ addopen()				INCL	_POSIX_SPAWN
posix_spawn_file_actions_ destroy()				INCL	_POSIX_SPAWN
posix_spawn_file_actions_init()				INCL	_POSIX_SPAWN
posix_spawnattr_destroy()			INCL	INCL	_POSIX_SPAWN
posix_spawnattr_getflags()			INCL	INCL	_POSIX_SPAWN
posix_spawnattr_getpgroup()				INCL	_POSIX_SPAWN
posix_spawnattr_getsigdefault()			INCL	INCL	_POSIX_SPAWN
posix_spawnattr_getsigmask()			INCL	INCL	_POSIX_SPAWN
posix_spawnattr_init()			INCL	INCL	_POSIX_SPAWN
posix_spawnattr_setflags()			INCL	INCL	_POSIX_SPAWN
posix_spawnattr_setpgroup()				INCL	_POSIX_SPAWN
posix_spawnattr_setsigdefault()			INCL	INCL	_POSIX_SPAWN
posix_spawnattr_setsigmask()			INCL	INCL	_POSIX_SPAWN
posix_spawnp()				INCL	_POSIX_SPAWN

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
pthread_spin_destroy()					_POSIX_SPIN_LOCKS and _POSIX_THREADS
pthread_spin_init()					_POSIX_SPIN_LOCKS and _POSIX_THREADS
pthread_spin_lock()					_POSIX_SPIN_LOCKS and _POSIX_THREADS
pthread_spin_trylock()					_POSIX_SPIN_LOCKS and _POSIX_THREADS
pthread_spin_unlock()					_POSIX_SPIN_LOCKS and _POSIX_THREADS
fdatasync()	EXCL	EXCL	EXCL	INCL	_POSIX_SYNCHRONIZED_IO
pthread_attr_getstackaddr()	EXCL	EXCL	EXCL	INCL	_POSIX_THREAD_ATTR_ STACKADDR and _POSIX_THREADS
pthread_attr_setstackaddr()	EXCL	EXCL	EXCL	INCL	_POSIX_THREAD_ATTR_ STACKADDR and _POSIX_THREADS
pthread_attr_getstacksize()	EXCL	INCL	INCL	INCL	_POSIX_THREAD_ATTR_ STACKSIZE, _POSIX_THREADS
pthread_attr_setstacksize()	EXCL	INCL	INCL	INCL	_POSIX_THREAD_ATTR_ STACKSIZE, _POSIX_THREADS
pthread_attr_getstack()	INCL	INCL	INCL	INCL	_POSIX_THREAD_ATTR_ STACKSIZE, XSI_THREADS_EXT
pthread_attr_setstack()	INCL	INCL	INCL	INCL	_POSIX_THREAD_ATTR_ STACKSIZE, _POSIX_THREADS, and _POSIX_THREAD_ATTR_ STACKADDR, XSI_THREADS_EXT
pthread_getcpuclockid()	INCL	INCL	INCL	INCL	_POSIX_THREAD_CPUTIME and _POSIX_THREADS
pthread_mutex_getprioceiling()	EXCL	EXCL	EXCL	INCL	_POSIX_THREAD_PRIO_PROTECT and _POSIX_THREADS
pthread_mutex_setprioceiling()	EXCL	EXCL	EXCL	INCL	_POSIX_THREAD_PRIO_PROTECT and _POSIX_THREADS
pthread_mutexattr_ getprioceiling()	INCL	INCL	INCL	INCL	_POSIX_THREAD_PRIO_PROTECT and _POSIX_THREADS
pthread_mutexattr_ setprioceiling()	INCL	INCL	INCL	INCL	_POSIX_THREAD_PRIO_PROTECT and _POSIX_THREADS
pthread_attr_getinheritsched()	INCL	INCL	INCL	INCL	_POSIX_THREAD_PRIORITY_SCHEDULING and _POSIX_THREADS
pthread_attr_getschedpolicy()	INCL	INCL	INCL	INCL	_POSIX_THREAD_PRIORITY_SCHEDULING and _POSIX_THREADS

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
pthread_attr_getscope()	INCL	INCL	INCL	INCL	_POSIX_THREAD_PRIORITY_ SCHEDULING and _POSIX_THREADS
pthread_attr_setinheritsched()	INCL	INCL	INCL	INCL	_POSIX_THREAD_PRIORITY_ SCHEDULING and _POSIX_THREADS
pthread_attr_setschedpolicy()	INCL	INCL	INCL	INCL	_POSIX_THREAD_PRIORITY_ SCHEDULING and _POSIX_THREADS
pthread_attr_setscope()	INCL	INCL	INCL	INCL	_POSIX_THREAD_PRIORITY_SCHED ULING and _POSIX_THREADS
pthread_getschedparam()	INCL	INCL	INCL	INCL	_POSIX_THREAD_PRIORITY_ SCHEDULING and _POSIX_THREADS
pthread_setschedparam()	INCL	INCL	INCL	INCL	_POSIX_THREAD_PRIORITY_ SCHEDULING and _POSIX_THREADS
pthread_setschedprio()	INCL	INCL	INCL	INCL	_POSIX_THREAD_PRIORITY_ SCHEDULING and _POSIX_THREADS
pthread_condattr_getpshared()				INCL	_POSIX_THREAD_PROCESS_ SHARED and _POSIX_THREADS
pthread_condattr_setpshared()				INCL	_POSIX_THREAD_PROCESS_ SHARED and _POSIX_THREADS
pthread_mutexattr_getpshared()				INCL	_POSIX_THREAD_PROCESS_ SHARED and _POSIX_THREADS
pthread_mutexattr_setpshared()				INCL	_POSIX_THREAD_PROCESS_ SHARED and _POSIX_THREADS
pthread_mutexattr_ getprotocol()	INCL	INCL	INCL	INCL	_POSIX_THREADS and either _POSIX_THREAD_PRIO_INHERIT or _POSIX_THREAD_PRIO_PROTECT
pthread_mutexattr_setprotocol()	INCL	INCL	INCL	INCL	_POSIX_THREADS and either _POSIX_THREAD_PRIO_INHERIT or _POSIX_THREAD_PRIO_PROTECT
pthread_mutex_timedlock()					_POSIX_TIMEOUTS and _POSIX_THREADS
posix_trace_timedgetnext_ event()		EXCL	EXCL	INCL	_POSIX_TIMEOUTS and _POSIX_TRACE
clock_getres()	INCL	INCL	INCL	INCL	_POSIX_TIMERS
clock_gettime()	INCL	INCL	INCL	INCL	_POSIX_TIMERS
clock_settime()	INCL	INCL	INCL	INCL	_POSIX_TIMERS
nanosleep()	INCL	INCL	INCL	INCL	_POSIX_TIMERS
timer_create()	INCL	INCL	INCL	INCL	_POSIX_TIMERS
timer_delete()	EXCL	EXCL	INCL	INCL	_POSIX_TIMERS

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
timer_getoverrun()	INCL	INCL	INCL	INCL	_POSIX_TIMERS
timer_gettime()	INCL	INCL	INCL	INCL	_POSIX_TIMERS
timer_settime()	INCL	INCL	INCL	INCL	_POSIX_TIMERS
posix_trace_attr_destroy()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_attr_getclockres()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_attr_ getcreatetime()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_attr_ getgenversion()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_attr_ getmaxdatasize()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_attr_ getmaxsystemeventsize()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_attr_ getmaxusereventsize()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_attr_getname()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_attr_ getstreamfullpolicy()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_attr_ getstreamsize()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_attr_init()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_attr_ setmaxdatasize()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_attr_setname()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_attr_ setstreamfullpolicy()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_attr_setstreamsize()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_clear()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_create()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_event()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_eventid_equal()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_eventid_get_ name()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_eventid_open()		EXCL	EXCL	EXCL	_POSIX_TRACE

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
posix_trace_eventtypelist_getnext_id()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_eventtypelist_rewind()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_get_attr()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_get_status()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_getnext_event()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_shutdown()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_start()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_stop()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_trygetnext_event()		EXCL	EXCL	EXCL	_POSIX_TRACE
posix_trace_eventset_add()		EXCL	EXCL	EXCL	_POSIX_TRACE and _POSIX_TRACE_EVENT_FILTER
posix_trace_eventset_del()		EXCL	EXCL	EXCL	_POSIX_TRACE and _POSIX_TRACE_EVENT_FILTER
posix_trace_eventset_empty()		EXCL	EXCL	EXCL	_POSIX_TRACE and _POSIX_TRACE_EVENT_FILTER
posix_trace_eventset_fill()		EXCL	EXCL	EXCL	_POSIX_TRACE and _POSIX_TRACE_EVENT_FILTER
posix_trace_eventset_ismember()		EXCL	EXCL	EXCL	_POSIX_TRACE and _POSIX_TRACE_EVENT_FILTER
posix_trace_get_filter()		EXCL	EXCL	EXCL	_POSIX_TRACE and _POSIX_TRACE_EVENT_FILTER
posix_trace_set_filter()		EXCL	EXCL	EXCL	_POSIX_TRACE and _POSIX_TRACE_EVENT_FILTER
posix_trace_trid_eventid_open()		EXCL	EXCL	EXCL	_POSIX_TRACE and _POSIX_TRACE_EVENT_FILTER
posix_trace_attr_getinherited()				EXCL	_POSIX_TRACE and _POSIX_TRACE_INHERIT
posix_trace_attr_setinherited()				EXCL	_POSIX_TRACE and _POSIX_TRACE_INHERIT
posix_trace_attr_getlogfullpolicy()		EXCL	EXCL	EXCL	_POSIX_TRACE and _POSIX_TRACE_LOG
posix_trace_attr_getlogsize()		EXCL	EXCL	EXCL	_POSIX_TRACE and _POSIX_TRACE_LOG
posix_trace_attr_setlogfullpolicy()		EXCL	EXCL	EXCL	_POSIX_TRACE and _POSIX_TRACE_LOG

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
posix_trace_attr_setlogsize()		EXCL	EXCL	EXCL	_POSIX_TRACE and _POSIX_TRACE_LOG
posix_trace_close()		EXCL	EXCL	EXCL	_POSIX_TRACE and _POSIX_TRACE_LOG
posix_trace_create_withlog()		EXCL	EXCL	EXCL	_POSIX_TRACE and _POSIX_TRACE_LOG
posix_trace_flush()		EXCL	EXCL	EXCL	_POSIX_TRACE and _POSIX_TRACE_LOG
posix_trace_open()		EXCL	EXCL	EXCL	_POSIX_TRACE and _POSIX_TRACE_LOG
posix_trace_rewind()		EXCL	EXCL	EXCL	_POSIX_TRACE and _POSIX_TRACE_LOG
posix_mem_offset()					_POSIX_TYPED_MEMORY_OBJECTS
posix_typed_mem_get_info()					_POSIX_TYPED_MEMORY_OBJECTS
posix_typed_mem_open()					_POSIX_TYPED_MEMORY_OBJECTS
crypt()					_XOPEN_CRYPT
encrypt()					_XOPEN_CRYPT
setkey()					_XOPEN_CRYPT
bcmp()					_XOPEN_LEGACY
bcopy()					_XOPEN_LEGACY
bzero()					_XOPEN_LEGACY
ecvt()					_XOPEN_LEGACY
fcvt()					_XOPEN_LEGACY
ftime()					_XOPEN_LEGACY
gcvt()					_XOPEN_LEGACY
getwd()					_XOPEN_LEGACY
index()					_XOPEN_LEGACY
mktemp()					_XOPEN_LEGACY
rindex()					_XOPEN_LEGACY
utimes()					_XOPEN_LEGACY
wcswcs()					_XOPEN_LEGACY
fattach()					_XOPEN_STREAMS
fdetach()					_XOPEN_STREAMS
getmsg()					_XOPEN_STREAMS

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
getpmsg()					_XOPEN_STREAMS
ioctl()		INCL	INCL	INCL	_XOPEN_STREAMS
isastream()					_XOPEN_STREAMS
putmsg()					_XOPEN_STREAMS
putpmsg()					_XOPEN_STREAMS
getdate_err					XSI_C_LANG_SUPPORT
longjmp()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_JUMP
setjmp()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_JUMP
acos()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
acosf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
acosh()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
acoshf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
acoshl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
acosl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
asin()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
asinf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
asinh()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
asinhf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
asinhl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
asinl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
atan()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
atan2()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
atan2f()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
atan2l()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
atanf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
atanh()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
atanhf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
atanhl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
atanl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cabs()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cabsf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
cabsl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cacos()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cacosf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cacosh()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cacoshf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cacoshl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cacosl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
carg()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cargf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cargl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
casin()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
casinf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
casinh()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
casinhf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
casinhl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
casinl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
catan()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
catanf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
catanh()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
catanhf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
catanhl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
catanl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cbrt()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cbrtf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cbrtl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
ccos()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
ccosf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
ccosh()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
ccoshf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
ccoshl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
ccosl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
ceil()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
ceilf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
ceill()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cexp()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cexpf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cexpl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cimag()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cimagf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cimagl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
clog()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
clogf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
clogl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
conj()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
conjf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
conjl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
copysign()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
copysignf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
copysignl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cos()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
cosf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cosh()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
coshf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
coshl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cosl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cpow()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cpowf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cpowl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cproj()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cprojf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
cprojl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
creal()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
crealf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
creall()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
csin()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
csinf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
csinh()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
csinhf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
csinhl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
csinl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
csqrt()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
csqrtf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
csqrtl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
ctan()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
ctanf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
ctanh()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
ctanhf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
ctanhl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
ctanl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
erf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
erfc()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
erfcf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
erfcl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
erff()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
erfl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
exp()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
exp2()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
exp2f()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
exp2l()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
expf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
expl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
expm1()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
expm1f()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
expm1l()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
fabs()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
fabsf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
fabsl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
fdim()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
fdimf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
fdiml()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
floor()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
floorf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
floorl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
fma()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
fmaf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
fmal()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
fmax()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
fmaxf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
fmaxl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
fmin()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
fminf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
fminl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
fmod()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
fmodf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
fmodl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
fpclassify()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
frexp()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
frexpf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
frexpl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
hypot()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
hypotf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
hypotl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
ilogb()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
ilogbf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
ilogbl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
isfinite()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
isgreater()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
isgreaterequal()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
isinf()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
isless()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
islessequal()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
islessgreater()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
isnan()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
isnormal()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
isunordered()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
ldexp()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
ldexpf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
ldexpl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
lgamma()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
lgammaf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
lgammal()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
llrint()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
llrintf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
llrintl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
llround()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
llroundf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
llroundl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
log()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
log10()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
log10f()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
log10l()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
log1p()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
log1pf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
log1pl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
log2()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
log2f()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
log2l()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
logb()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
logbf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
logbl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
logf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
logl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
lrint()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
lrintf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
lrintl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
lround()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
lroundf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
lroundl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
modf()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
modff()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
modfl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
nan()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
nanf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
nanl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
nearbyint()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
nearbyintf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
nearbyintl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
nextafter()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
nextafterf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
nextafterl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
nexttoward()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
nexttowardf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
nexttowardl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
pow()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
powf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
powl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
remainder()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
remainderf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
remainderl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
remquo()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
remquof()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
remquol()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
rint()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
rintf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
rintl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
round()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
roundf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
roundl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
scalb()					XSI_MATH
scalbln()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
scalblnf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
scalblnl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
scalbn()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
scalbnf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
scalbnl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
signbit()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
sin()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
sinf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
sinh()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
sinhf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
sinhl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
sinl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
sqrt()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
sqrtf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
sqrtl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
tan()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
tanf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
tanh()	INCL	INCL	INCL	INCL	POSIX_C_LANG_MATH
tanhf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
tanhl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
tanl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
tgamma()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
tgammaf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
tgammal()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
trunc()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
truncf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
truncl()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_MATH
abs()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
atof()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
atoi()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
atol()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
atoll()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
bsearch()	EXCL	EXCL	INCL	INCL	POSIX_C_LANG_SUPPORT
calloc()	EXCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
ctime()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
difftime()	EXCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
div()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
feclearexcept()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
fegetenv()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
fegetexceptflag()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
fegetround()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
feholdexcept()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
feraiseexcept()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
fesetenv()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
fesetexceptflag()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
fesetround()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
fetestexcept()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
feupdateenv()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
free()	EXCL	EXCL	INCL	INCL	POSIX_C_LANG_SUPPORT
gmtime()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
imaxabs()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
imaxdiv()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
isalnum()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
isalpha()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
isblank()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
iscntrl()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
isdigit()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
isgraph()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
islower()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
isprint()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
ispunct()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
isspace()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
isupper()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
isxdigit()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
labs()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
ldiv()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
llabs()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
lldiv()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
localeconv()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
localtime()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
malloc()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
memchr()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
memcmp()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
memcpy()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
memmove()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
memset()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
mktime()	EXCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
qsort()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
rand()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
realloc()	EXCL	EXCL	INCL	INCL	POSIX_C_LANG_SUPPORT
setlocale()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
snprintf()	EXCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
sprintf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
srand()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
sscanf()	EXCL	EXCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strcat()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strchr()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strcmp()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strcoll()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
strcpy()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strcspn()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strerror()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
strftime()	EXCL	EXCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strlen()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strncat()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strncmp()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strncpy()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strpbrk()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strrchr()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strspn()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strstr()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strtod()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strtof()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
strtoimax()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
strtok()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
strtol()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strtold()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
strtoll()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
strtoul()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
strtoull()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
strtoumax()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
strxfrm()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
time()	EXCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
tolower()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
toupper()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
tzname	EXCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
tzset()	EXCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT
va_arg()	EXCL	EXCL	INCL	INCL	POSIX_C_LANG_SUPPORT
va_copy()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
va_end()	EXCL	EXCL	INCL	INCL	POSIX_C_LANG_SUPPORT
va_start()	EXCL	EXCL	INCL	INCL	POSIX_C_LANG_SUPPORT
vsnprintf()	EXCL	EXCL	INCL	INCL	POSIX_C_LANG_SUPPORT
vsprintf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
vsscanf()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
asctime()	EXCL	EXCL	EXCL	INCL	POSIX_C_LANG_SUPPORT
asctime_r()	EXCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT, _POSIX_THREAD_SAFE_FUNCTIONS
ctime_r()	EXCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT, _POSIX_THREAD_SAFE_FUNCTIONS
gmtime_r()	EXCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT, _POSIX_THREAD_SAFE_FUNCTIONS
localtime_r()	EXCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT, _POSIX_THREAD_SAFE_FUNCTIONS
rand_r()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT, _POSIX_THREAD_SAFE_FUNCTIONS
strerror_r()	EXCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT, _POSIX_THREAD_SAFE_FUNCTIONS
strtok_r()	INCL	INCL	INCL	INCL	POSIX_C_LANG_SUPPORT, _POSIX_THREAD_SAFE_FUNCTIONS
btowc()				INCL	POSIX_C_LANG_WIDE_CHAR
iswalnum()				INCL	POSIX_C_LANG_WIDE_CHAR
iswalpha()				INCL	POSIX_C_LANG_WIDE_CHAR
iswblank()				INCL	POSIX_C_LANG_WIDE_CHAR
iswcntrl()				INCL	POSIX_C_LANG_WIDE_CHAR

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
iswctype()				INCL	POSIX_C_LANG_WIDE_CHAR
iswdigit()				INCL	POSIX_C_LANG_WIDE_CHAR
iswgraph()				INCL	POSIX_C_LANG_WIDE_CHAR
iswlower()				INCL	POSIX_C_LANG_WIDE_CHAR
iswprint()				INCL	POSIX_C_LANG_WIDE_CHAR
iswpunct()				INCL	POSIX_C_LANG_WIDE_CHAR
iswspace()				INCL	POSIX_C_LANG_WIDE_CHAR
iswupper()				INCL	POSIX_C_LANG_WIDE_CHAR
iswxdigit()				INCL	POSIX_C_LANG_WIDE_CHAR
mblen()				INCL	POSIX_C_LANG_WIDE_CHAR
mbrlen()				INCL	POSIX_C_LANG_WIDE_CHAR
mbrtowc()				INCL	POSIX_C_LANG_WIDE_CHAR
mbsinit()				INCL	POSIX_C_LANG_WIDE_CHAR
mbsrtowcs()				INCL	POSIX_C_LANG_WIDE_CHAR
mbstowcs()				INCL	POSIX_C_LANG_WIDE_CHAR
mbtowc()				INCL	POSIX_C_LANG_WIDE_CHAR
swprintf()				INCL	POSIX_C_LANG_WIDE_CHAR
swscanf()				INCL	POSIX_C_LANG_WIDE_CHAR
towctrans()				INCL	POSIX_C_LANG_WIDE_CHAR
towlower()				INCL	POSIX_C_LANG_WIDE_CHAR
towupper()				INCL	POSIX_C_LANG_WIDE_CHAR
vswprintf()				INCL	POSIX_C_LANG_WIDE_CHAR
vswscanf()				INCL	POSIX_C_LANG_WIDE_CHAR
wcrtomb()				INCL	POSIX_C_LANG_WIDE_CHAR
wcscat()				INCL	POSIX_C_LANG_WIDE_CHAR
wcschr()				INCL	POSIX_C_LANG_WIDE_CHAR
wcscmp()				INCL	POSIX_C_LANG_WIDE_CHAR
wcscoll()				INCL	POSIX_C_LANG_WIDE_CHAR
wcscpy()				INCL	POSIX_C_LANG_WIDE_CHAR
wcscspn()				INCL	POSIX_C_LANG_WIDE_CHAR
wcsftime()				INCL	POSIX_C_LANG_WIDE_CHAR

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
wcslen()				INCL	POSIX_C_LANG_WIDE_CHAR
wcsncat()				INCL	POSIX_C_LANG_WIDE_CHAR
wcsncmp()				INCL	POSIX_C_LANG_WIDE_CHAR
wcsncpy()				INCL	POSIX_C_LANG_WIDE_CHAR
wcspbrk()				INCL	POSIX_C_LANG_WIDE_CHAR
wcsrchr()				INCL	POSIX_C_LANG_WIDE_CHAR
wcsrtombs()				INCL	POSIX_C_LANG_WIDE_CHAR
wcsspn()				INCL	POSIX_C_LANG_WIDE_CHAR
wcsstr()				INCL	POSIX_C_LANG_WIDE_CHAR
wcstod()				INCL	POSIX_C_LANG_WIDE_CHAR
wcstof()				INCL	POSIX_C_LANG_WIDE_CHAR
wcstoimax()				INCL	POSIX_C_LANG_WIDE_CHAR
wcstok()				INCL	POSIX_C_LANG_WIDE_CHAR
wcstol()				INCL	POSIX_C_LANG_WIDE_CHAR
wcstold()				INCL	POSIX_C_LANG_WIDE_CHAR
wcstoll()				INCL	POSIX_C_LANG_WIDE_CHAR
wcstombs()				INCL	POSIX_C_LANG_WIDE_CHAR
wcstoul()				INCL	POSIX_C_LANG_WIDE_CHAR
wcstoull()				INCL	POSIX_C_LANG_WIDE_CHAR
wcstoumax()				INCL	POSIX_C_LANG_WIDE_CHAR
wcsxfrm()				INCL	POSIX_C_LANG_WIDE_CHAR
wctob()				INCL	POSIX_C_LANG_WIDE_CHAR
wctomb()				INCL	POSIX_C_LANG_WIDE_CHAR
wctrans()				INCL	POSIX_C_LANG_WIDE_CHAR
wctype()				INCL	POSIX_C_LANG_WIDE_CHAR
wmemchr()				INCL	POSIX_C_LANG_WIDE_CHAR
wmemcmp()				INCL	POSIX_C_LANG_WIDE_CHAR
wmemcpy()				INCL	POSIX_C_LANG_WIDE_CHAR
wmemmove()				INCL	POSIX_C_LANG_WIDE_CHAR
wmemset()				INCL	POSIX_C_LANG_WIDE_CHAR
clearerr()	EXCL	INCL	INCL	INCL	POSIX_DEVICE_IO

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
close()	EXCL	INCL	INCL	INCL	POSIX_DEVICE_IO
fclose()	EXCL	INCL	INCL	INCL	POSIX_DEVICE_IO
fdopen()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
feof()	EXCL	INCL	INCL	INCL	POSIX_DEVICE_IO
ferror()	EXCL	INCL	INCL	INCL	POSIX_DEVICE_IO
fflush()	EXCL	INCL	INCL	INCL	POSIX_DEVICE_IO
fgetc()	EXCL	INCL	INCL	INCL	POSIX_DEVICE_IO
fgets()	EXCL	INCl	INCL	INCL	POSIX_DEVICE_IO
fileno()	EXCL	INCL	INCL	INCL	POSIX_DEVICE_IO
fopen()	EXCL	INCL	INCL	INCL	POSIX_DEVICE_IO
fprintf()	EXCL	INCL	INCL	INCL	POSIX_DEVICE_IO
fputc()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
fputs()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
fread()	EXCL	INCL	INCL	INCL	POSIX_DEVICE_IO
freopen()	EXCL	INCL	INCL	INCL	POSIX_DEVICE_IO
fscanf()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
fwrite()	EXCL	INCL	INCL	INCL	POSIX_DEVICE_IO
getc()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
getchar()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
gets()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
open()	EXCL	INCL	INCL	INCL	POSIX_DEVICE_IO
perror()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
printf()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
putc()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
putchar()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
puts()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
read()	EXCL	INCL	INCL	INCL	POSIX_DEVICE_IO
scanf()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
setbuf()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
setvbuf()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
stderr	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
stdin	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
stdout	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
ungetc()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
vfprintf()	EXCL	EXCL	INCL	INCL	POSIX_DEVICE_IO
vfscanf()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
vprintf()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
vscanf()	EXCL	EXCL	EXCL	INCL	POSIX_DEVICE_IO
write()	EXCL	INCL	INCL	INCL	POSIX_DEVICE_IO
cfgetispeed()					POSIX_DEVICE_SPECIFIC
cfgetospeed()					POSIX_DEVICE_SPECIFIC
cfsetispeed()					POSIX_DEVICE_SPECIFIC
cfsetospeed()					POSIX_DEVICE_SPECIFIC
ctermid()					POSIX_DEVICE_SPECIFIC
isatty()					POSIX_DEVICE_SPECIFIC
tcdrain()					POSIX_DEVICE_SPECIFIC
tcflow()					POSIX_DEVICE_SPECIFIC
tcflush()					POSIX_DEVICE_SPECIFIC
tcgetattr()					POSIX_DEVICE_SPECIFIC
tcsendbreak()					POSIX_DEVICE_SPECIFIC
tcsetattr()					POSIX_DEVICE_SPECIFIC
ttyname()					POSIX_DEVICE_SPECIFIC
ttyname_r()					POSIX_DEVICE_SPECIFIC, _POSIX_THREAD_SAFE_FUNCTIONS
FD_CLR()				INCL	POSIX_EVENT_MGMT
FD_ISSET()				INCL	POSIX_EVENT_MGMT
FD_SET()				INCL	POSIX_EVENT_MGMT
FD_ZERO()				INCL	POSIX_EVENT_MGMT
pselect()				INCL	POSIX_EVENT_MGMT
select()			INCL	INCL	POSIX_EVENT_MGMT
dup()		EXCL	EXCL	INCL	POSIX_FD_MGMT
dup2()		EXCL	INCL	INCL	POSIX_FD_MGMT
fcntl()		EXCL	INCL	INCL	POSIX_FD_MGMT

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
fgetpos()		EXCL	EXCL	INCL	POSIX_FD_MGMT
fseek()		INCL	INCL	INCL	POSIX_FD_MGMT
fseeko()		INCL	INCL	INCL	POSIX_FD_MGMT
fsetpos()		EXCL	EXCL	INCL	POSIX_FD_MGMT
ftell()		INCL	INCL	INCL	POSIX_FD_MGMT
ftello()		INCL	INCL	INCL	POSIX_FD_MGMT
ftruncate()	INCL	INCL	INCL	INCL	POSIX_FD_MGMT
lseek()		INCL	INCL	INCL	POSIX_FD_MGMT
rewind()		EXCL	EXCL	INCL	POSIX_FD_MGMT
mkfifo()			INCL	INCL	POSIX_FIFO
chmod()			INCL	INCL	POSIX_FILE_ATTRIBUTES
chown()			INCL	INCL	POSIX_FILE_ATTRIBUTES
fchmod()				INCL	POSIX_FILE_ATTRIBUTES
fchown()				INCL	POSIX_FILE_ATTRIBUTES
umask()		INCL	INCL	INCL	POSIX_FILE_ATTRIBUTES
flockfile()	EXCL	EXCL	INCL	INCL	POSIX_FILE_LOCKING, _POSIX_ THREAD_SAFE_FUNCTIONS
ftrylockfile()	EXCL	EXCL	INCL	INCL	POSIX_FILE_LOCKING, _POSIX_ THREAD_SAFE_FUNCTIONS
funlockfile()	EXCL	EXCL	INCL	INCL	POSIX_FILE_LOCKING, _POSIX_ THREAD_SAFE_FUNCTIONS
getc_unlocked()	EXCL	EXCL	EXCL	INCL	POSIX_FILE_LOCKING, _POSIX_ THREAD_SAFE_FUNCTIONS
getchar_unlocked()	EXCL	EXCL	EXCL	INCL	POSIX_FILE_LOCKING, _POSIX_ THREAD_SAFE_FUNCTIONS
putc_unlocked()	EXCL	EXCL	EXCL	INCL	POSIX_FILE_LOCKING, _POSIX_ THREAD_SAFE_FUNCTIONS
putchar_unlocked()	EXCL	EXCL	EXCL	INCL	POSIX_FILE_LOCKING, _POSIX_ THREAD_SAFE_FUNCTIONS
access()		INCL	INCL	INCL	POSIX_FILE_SYSTEM
chdir()		INCL	INCL	INCL	POSIX_FILE_SYSTEM
closedir()		INCL	INCL	INCL	POSIX_FILE_SYSTEM
creat()		INCL	INCL	INCL	POSIX_FILE_SYSTEM
fpathconf()		EXCL	EXCL	INCL	POSIX_FILE_SYSTEM

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
fstat()		INCL	INCL	INCL	POSIX_FILE_SYSTEM
getcwd()		INCL	INCL	INCL	POSIX_FILE_SYSTEM
link()		INCL	INCL	INCL	POSIX_FILE_SYSTEM
mkdir()		INCL	INCL	INCL	POSIX_FILE_SYSTEM
opendir()		INCL	INCL	INCL	POSIX_FILE_SYSTEM
pathconf()		EXCL	EXCL	INCL	POSIX_FILE_SYSTEM
readdir()		INCL	INCL	INCL	POSIX_FILE_SYSTEM
remove()		INCL	INCL	INCL	POSIX_FILE_SYSTEM
rename()		INCL	INCL	INCL	POSIX_FILE_SYSTEM
rewinddir()		INCL	INCL	INCL	POSIX_FILE_SYSTEM
rmdir()		INCL	INCL	INCL	POSIX_FILE_SYSTEM
stat()	INCL	INCL	INCL	INCL	POSIX_FILE_SYSTEM
tmpfile()		EXCL	EXCL	INCL	POSIX_FILE_SYSTEM
tmpnam()		EXCL	EXCL	INCL	POSIX_FILE_SYSTEM
unlink()		INCL	INCL	INCL	POSIX_FILE_SYSTEM
utime()		EXCL	EXCL	INCL	POSIX_FILE_SYSTEM
readdir_r()		INCL	INCL	INCL	POSIX_FILE_SYSTEM, _POSIX_ THREAD_SAFE_FUNCTIONS
glob()					POSIX_FILE_SYSTEM_EXT
globfree()					POSIX_FILE_SYSTEM_EXT
setpgid()					POSIX_JOB_CONTROL
tcgetpgrp()					POSIX_JOB_CONTROL
tcsetpgrp()					POSIX_JOB_CONTROL
_Exit()			INCL	INCL	POSIX_MULTI_PROCESS
_exit()			INCL	INCL	POSIX_MULTI_PROCESS
assert()				INCL	POSIX_MULTI_PROCESS
atexit()			INCL	INCL	POSIX_MULTI_PROCESS
clock()			INCL	INCL	POSIX_MULTI_PROCESS
execl()			INCL	INCL	POSIX_MULTI_PROCESS
execle()			INCL	INCL	POSIX_MULTI_PROCESS
execlp()					POSIX_MULTI_PROCESS
execv()			INCL	INCL	POSIX_MULTI_PROCESS

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
execve()			INCL	INCL	POSIX_MULTI_PROCESS
execvp()					POSIX_MULTI_PROCESS
exit()			INCL	INCL	POSIX_MULTI_PROCESS
fork()			INCL	INCL	POSIX_MULTI_PROCESS
getpgrp()			INCL	INCL	POSIX_MULTI_PROCESS
getpid()			INCL	INCL	POSIX_MULTI_PROCESS
getppid()			INCL	INCL	POSIX_MULTI_PROCESS
setsid()			EXCL	INCL	POSIX_MULTI_PROCESS
sleep()			INCL	INCL	POSIX_MULTI_PROCESS
times()			INCL	INCL	POSIX_MULTI_PROCESS
wait()				INCL	POSIX_MULTI_PROCESS
waitpid()			INCL	INCL	POSIX_MULTI_PROCESS
accept()	EXCL	EXCL	INCL	INCL	POSIX_NETWORKING
bind()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
connect()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
endhostent()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
endnetent()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
endprotoent()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
endservent()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
freeaddrinfo()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
gai_strerror()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
getaddrinfo()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
gethostbyaddr()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
gethostbyname()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
gethostent()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
gethostname()	EXCL	INCL	INCL	INCL	POSIX_NETWORKING
getnameinfo()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
getnetbyaddr()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
getnetbyname()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
getnetent()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
getpeername()	INCL	INCL	INCL	INCL	POSIX_NETWORKING

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
getprotobyname()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
getprotobynumber()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
getprotoent()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
getservbyname()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
getservbyport()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
getservent()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
getsockname()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
getsockopt()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
h_errno	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
htonl()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
htons()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
if_freenameindex()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
if_indextoname()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
if_nameindex()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
if_nametoindex()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
inet_addr()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
inet_ntoa()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
inet_ntop()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
inet_pton()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
listen()	EXCL	EXCL	INCL	INCL	POSIX_NETWORKING
ntohl()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
ntohs()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
recv()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
recvfrom()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
recvmsg()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
send()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
sendmsg()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
sendto()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
sethostent()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
setnetent()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
setprotoent()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
setservent()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
setsockopt()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
shutdown()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
sockatmark()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
socket()	INCL	INCL	INCL	INCL	POSIX_NETWORKING
socketpair()	EXCL	EXCL	EXCL	INCL	POSIX_NETWORKING
pipe()			INCL	INCL	POSIX_PIPE
regcomp()					POSIX_REGEXP
regerror()					POSIX_REGEXP
regexec()					POSIX_REGEXP
regfree()					POSIX_REGEXP
pthread_rwlock_destroy()					POSIX_RW_LOCKS, _POSIX_THREADS
pthread_rwlock_init()					POSIX_RW_LOCKS, _POSIX_THREADS
pthread_rwlock_rdlock()					POSIX_RW_LOCKS, _POSIX_THREADS
pthread_rwlock_tryrdlock()					POSIX_RW_LOCKS, _POSIX_THREADS
pthread_rwlock_trywrlock()					POSIX_RW_LOCKS, _POSIX_THREADS
pthread_rwlock_unlock()					POSIX_RW_LOCKS, _POSIX_THREADS
pthread_rwlock_wrlock()					POSIX_RW_LOCKS, _POSIX_THREADS
pthread_rwlockattr_destroy()					POSIX_RW_LOCKS, _POSIX_THREADS
pthread_rwlockattr_init()					POSIX_RW_LOCKS, _POSIX_THREADS
pthread_rwlockattr_ getpshared()					POSIX_RW_LOCKS, _POSIX_THREADS, and POSIX_THREAD_PROCESS_SHARED
pthread_rwlockattr_ setpshared()					POSIX_RW_LOCKS, _POSIX_THREADS, and POSIX_THREAD_PROCESS_SHARED

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
pthread_rwlock_timedrdlock()					POSIX_RW_LOCKS, _POSIX_THREADS, and _POSIX_TIMEOUTS
pthread_rwlock_timedwrlock()					POSIX_RW_LOCKS, _POSIX_THREADS, and _POSIX_TIMEOUTS
pclose()					POSIX_SHELL_FUNC
popen()					POSIX_SHELL_FUNC
system()					POSIX_SHELL_FUNC
wordexp()					POSIX_SHELL_FUNC
wordfree()					POSIX_SHELL_FUNC
siglongjmp()			INCL	INCL	POSIX_SIGNAL_JUMP
sigsetjmp()			INCL	INCL	POSIX_SIGNAL_JUMP
abort()	EXCL	EXCL	INCL	INCL	POSIX_SIGNALS
alarm()	INCL	INCL	INCL	INCL	POSIX_SIGNALS
kill()	EXCL	EXCL	INCL	INCL	POSIX_SIGNALS
pause()	INCL	INCL	INCL	INCL	POSIX_SIGNALS
raise()	EXCL	EXCL	INCL	INCL	POSIX_SIGNALS
sigaction()	INCL	INCL	INCL	INCL	POSIX_SIGNALS
sigaddset()	INCL	INCL	INCL	INCL	POSIX_SIGNALS
sigdelset()	INCL	INCL	INCL	INCL	POSIX_SIGNALS
sigemptyset()	INCL	INCL	INCL	INCL	POSIX_SIGNALS
sigfillset()	INCL	INCL	INCL	INCL	POSIX_SIGNALS
sigismember()	INCL	INCL	INCL	INCL	POSIX_SIGNALS
signal()	EXCL	EXCL	EXCL	INCL	POSIX_SIGNALS
sigpending()	INCL	INCL	INCL	INCL	POSIX_SIGNALS
sigprocmask()	EXCL	EXCL	EXCL	INCL	POSIX_SIGNALS
sigsuspend()	INCL	INCL	INCL	INCL	POSIX_SIGNALS
sigwait()	INCL	INCL	INCL	INCL	POSIX_SIGNALS
confstr()	EXCL	EXCL	EXCL	INCL	POSIX_SINGLE_PROCESS
environ	EXCL	EXCL	INCL	INCL	POSIX_SINGLE_PROCESS
errno	INCL	INCL	INCL	INCL	POSIX_SINGLE_PROCESS
getenv()	EXCL	EXCL	INCL	INCL	POSIX_SINGLE_PROCESS

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
setenv()	EXCL	EXCL	EXCL	INCL	POSIX_SINGLE_PROCESS
sysconf()	EXCL	EXCL	INCL	INCL	POSIX_SINGLE_PROCESS
uname()	EXCL	EXCL	INCL	INCL	POSIX_SINGLE_PROCESS
unsetenv()	EXCL	EXCL	EXCL	INCL	POSIX_SINGLE_PROCESS
fnmatch()					POSIX_STRING_MATCHING
getopt()					POSIX_STRING_MATCHING
optarg					POSIX_STRING_MATCHING
opterr					POSIX_STRING_MATCHING
optind					POSIX_STRING_MATCHING
optopt					POSIX_STRING_MATCHING
lstat()			INCL	INCL	POSIX_SYMBOLIC_LINKS
readlink()					POSIX_SYMBOLIC_LINKS
symlink()					POSIX_SYMBOLIC_LINKS
getgrgid()					POSIX_SYSTEM_DATABASE
getgrnam()					POSIX_SYSTEM_DATABASE
getpwnam()					POSIX_SYSTEM_DATABASE
getpwuid()					POSIX_SYSTEM_DATABASE
getgrgid_r()					POSIX_SYSTEM_DATABASE, _POSIX_THREAD_SAFE_ FUNCTIONS
getgrnam_r()					POSIX_SYSTEM_DATABASE, _POSIX_THREAD_SAFE_ FUNCTIONS
getpwnam_r()					POSIX_SYSTEM_DATABASE, _POSIX_THREAD_SAFE_ FUNCTIONS
getpwuid_r()					POSIX_SYSTEM_DATABASE, _POSIX_THREAD_SAFE_ FUNCTIONS
pthread_atfork()	EXCL	EXCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_attr_destroy()	INCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_attr_getdetachstate()	EXCL	EXCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
pthread_attr_getschedparam()	INCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_attr_init()	INCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_attr_setdetachstate()	EXCL	EXCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_attr_setschedparam()	INCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_cancel()	EXCL	EXCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_cleanup_pop()	EXCL	EXCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_cleanup_push()	EXCL	EXCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_cond_broadcast()	EXCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_cond_destroy()	EXCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_cond_init()	EXCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_cond_signal()	EXCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_cond_timedwait()	EXCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_cond_wait()	EXCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_condattr_destroy()	EXCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_condattr_init()	EXCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_create()	INCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_detach()	EXCL	EXCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_equal()	INCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_exit()	EXCL	EXCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
pthread_getspecific()	EXCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_join()	EXCL	EXCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_key_create()	EXCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_key_delete()	EXCL	EXCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_kill()	EXCL	EXCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_mutex_destroy()	EXCL	EXCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_mutex_init()	INCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_mutex_lock()	INCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_mutex_trylock()	INCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_mutex_unlock()	INCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_mutexattr_destroy()	INCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_mutexattr_init()	INCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_once()	INCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_self()	INCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_setcancelstate()	EXCL	EXCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_setcanceltype()	EXCL	EXCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_setspecific()	EXCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_sigmask()	INCL	INCL	INCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
pthread_testcancel()	EXCL	EXCL	EXCL	INCL	POSIX_THREADS_BASE, _POSIX_THREADS
getegid()			INCL	INCL	POSIX_USER_GROUPS

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
geteuid()			INCL	INCL	POSIX_USER_GROUPS
getgid()			INCL	INCL	POSIX_USER_GROUPS
getgroups()			INCL	INCL	POSIX_USER_GROUPS
getlogin()			EXCL	INCL	POSIX_USER_GROUPS
getuid()			INCL	INCL	POSIX_USER_GROUPS
setegid()			INCL	INCL	POSIX_USER_GROUPS
seteuid()			INCL	INCL	POSIX_USER_GROUPS
setgid()			INCL	INCL	POSIX_USER_GROUPS
setuid()			INCL	INCL	POSIX_USER_GROUPS
getlogin_r()				INCL	POSIX_USER_GROUPS, _POSIX_ THREAD_SAFE_FUNCTIONS
fgetwc()				INCL	POSIX_WIDE_CHAR_IO
fgetws()				INCL	POSIX_WIDE_CHAR_IO
fputwc()				INCL	POSIX_WIDE_CHAR_IO
fputws()				INCL	POSIX_WIDE_CHAR_IO
fwide()				INCL	POSIX_WIDE_CHAR_IO
fwprintf()				INCL	POSIX_WIDE_CHAR_IO
fwscanf()				INCL	POSIX_WIDE_CHAR_IO
getwc()				INCL	POSIX_WIDE_CHAR_IO
getwchar()				INCL	POSIX_WIDE_CHAR_IO
putwc()				INCL	POSIX_WIDE_CHAR_IO
putwchar()				INCL	POSIX_WIDE_CHAR_IO
ungetwc()				INCL	POSIX_WIDE_CHAR_IO
vfwprintf()				INCL	POSIX_WIDE_CHAR_IO
vfwscanf()				INCL	POSIX_WIDE_CHAR_IO
vwprintf()				INCL	POSIX_WIDE_CHAR_IO
vwscanf()				INCL	POSIX_WIDE_CHAR_IO
wprintf()				INCL	POSIX_WIDE_CHAR_IO
wscanf()				INCL	POSIX_WIDE_CHAR_IO
_tolower()					XSI_C_LANG_SUPPORT
_toupper()					XSI_C_LANG_SUPPORT
a64l()					XSI_C_LANG_SUPPORT

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
drand48()					XSI_C_LANG_SUPPORT
erand48()					XSI_C_LANG_SUPPORT
ffs()					XSI_C_LANG_SUPPORT
getcontext()					XSI_C_LANG_SUPPORT
getdate()					XSI_C_LANG_SUPPORT
getsubopt()					XSI_C_LANG_SUPPORT
hcreate()					XSI_C_LANG_SUPPORT
hdestroy()					XSI_C_LANG_SUPPORT
hsearch()					XSI_C_LANG_SUPPORT
iconv()					XSI_C_LANG_SUPPORT
iconv_close()					XSI_C_LANG_SUPPORT
iconv_open()					XSI_C_LANG_SUPPORT
initstate()					XSI_C_LANG_SUPPORT
insque()					XSI_C_LANG_SUPPORT
isascii()					XSI_C_LANG_SUPPORT
jrand48()					XSI_C_LANG_SUPPORT
l64a()					XSI_C_LANG_SUPPORT
lcong48()					XSI_C_LANG_SUPPORT
lfind()					XSI_C_LANG_SUPPORT
lrand48()					XSI_C_LANG_SUPPORT
lsearch()					XSI_C_LANG_SUPPORT
makecontext()					XSI_C_LANG_SUPPORT
memccpy()					XSI_C_LANG_SUPPORT
mrand48()					XSI_C_LANG_SUPPORT
nrand48()					XSI_C_LANG_SUPPORT
random()					XSI_C_LANG_SUPPORT
remque()					XSI_C_LANG_SUPPORT
seed48()					XSI_C_LANG_SUPPORT
setcontext()					XSI_C_LANG_SUPPORT
setstate()					XSI_C_LANG_SUPPORT
signgam()					XSI_C_LANG_SUPPORT

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
srand48()					XSI_C_LANG_SUPPORT
srandom()					XSI_C_LANG_SUPPORT
strcasecmp()					XSI_C_LANG_SUPPORT
strdup()					XSI_C_LANG_SUPPORT
strfmon()					XSI_C_LANG_SUPPORT
strncasecmp()					XSI_C_LANG_SUPPORT
strptime()					XSI_C_LANG_SUPPORT
swab()					XSI_C_LANG_SUPPORT
swapcontext()					XSI_C_LANG_SUPPORT
tdelete()					XSI_C_LANG_SUPPORT
tfind()					XSI_C_LANG_SUPPORT
timezone()					XSI_C_LANG_SUPPORT
toascii()					XSI_C_LANG_SUPPORT
tsearch()					XSI_C_LANG_SUPPORT
twalk()					XSI_C_LANG_SUPPORT
daylight()					XSI_DBM
dbm_clearerr()					XSI_DBM
dbm_close()					XSI_DBM
dbm_delete()					XSI_DBM
dbm_error()					XSI_DBM
dbm_fetch()					XSI_DBM
dbm_firstkey()					XSI_DBM
dbm_nextkey()					XSI_DBM
dbm_open()					XSI_DBM
dbm_store()					XSI_DBM
fmtmsg()					XSI_DEVICE_IO
poll()					XSI_DEVICE_IO
pread()					XSI_DEVICE_IO
pwrite()					XSI_DEVICE_IO
readv()					XSI_DEVICE_IO
writev()					XSI_DEVICE_IO

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
grantpt()					XSI_DEVICE_SPECIFIC
posix_openpt()					XSI_DEVICE_SPECIFIC
ptsname()					XSI_DEVICE_SPECIFIC
unlockpt()					XSI_DEVICE_SPECIFIC
dlclose()					XSI_DYNAMIC_LINKING
dlerror()					XSI_DYNAMIC_LINKING
dlopen()					XSI_DYNAMIC_LINKING
dlsym()					XSI_DYNAMIC_LINKING
truncate()					XSI_FD_MGMT
basename()					XSI_FILE_SYSTEM
dirname()					XSI_FILE_SYSTEM
fchdir()					XSI_FILE_SYSTEM
fstatvfs()					XSI_FILE_SYSTEM
ftw()					XSI_FILE_SYSTEM
lchown()					XSI_FILE_SYSTEM
lockf()					XSI_FILE_SYSTEM
mknod()					XSI_FILE_SYSTEM
mkstemp()					XSI_FILE_SYSTEM
nftw()					XSI_FILE_SYSTEM
realpath()					XSI_FILE_SYSTEM
seekdir()					XSI_FILE_SYSTEM
statvfs()					XSI_FILE_SYSTEM
sync()					XSI_FILE_SYSTEM
telldir()					XSI_FILE_SYSTEM
tempnam()					XSI_FILE_SYSTEM
catclose()					XSI_I18N
catgets()					XSI_I18N
catopen()					XSI_I18N
msgctl()					XSI_I18N
msgget()					XSI_I18N
msgrcv()					XSI_I18N

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
msgsnd()					XSI_I18N
nl_langinfo()					XSI_I18N
semctl()					XSI_I18N
semget()					XSI_I18N
semop()					XSI_I18N
shmat()					XSI_I18N
shmctl()					XSI_I18N
shmdt()					XSI_I18N
shmget()					XSI_I18N
ftok()					XSI_IPC
tcgetsid()					XSI_JOB_CONTROL
_longjmp()					XSI_JUMP
_setjmp()					XSI_JUMP
j0()					XSI_MATH
j1()					XSI_MATH
jn()					XSI_MATH
y0()					XSI_MATH
y1()					XSI_MATH
yn()					XSI_MATH
getpgid()					XSI_MULTI_PROCESS
getpriority()					XSI_MULTI_PROCESS
getrlimit()					XSI_MULTI_PROCESS
getrusage()					XSI_MULTI_PROCESS
getsid()					XSI_MULTI_PROCESS
nice()					XSI_MULTI_PROCESS
setpgrp()					XSI_MULTI_PROCESS
setpriority()					XSI_MULTI_PROCESS
setrlimit()					XSI_MULTI_PROCESS
ulimit()					XSI_MULTI_PROCESS
usleep()					XSI_MULTI_PROCESS
vfork()					XSI_MULTI_PROCESS

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
waitid()				INCL	XSI_MULTI_PROCESS
bsd_signal()					XSI_SIGNALS
killpg()					XSI_SIGNALS
sigaltstack()					XSI_SIGNALS
sighold()					XSI_SIGNALS
sigignore()					XSI_SIGNALS
siginterrupt()					XSI_SIGNALS
sigpause()					XSI_SIGNALS
sigrelse()					XSI_SIGNALS
sigset()					XSI_SIGNALS
ualarm()					XSI_SIGNALS
gethostid()					XSI_SINGLE_PROCESS
gettimeofday()					XSI_SINGLE_PROCESS
putenv()					XSI_SINGLE_PROCESS
endpwent()					XSI_SYSTEM_DATABASE
getpwent()					XSI_SYSTEM_DATABASE
setpwent()					XSI_SYSTEM_DATABASE
closelog()					XSI_SYSTEM_LOGGING
openlog()					XSI_SYSTEM_LOGGING
setlogmask()					XSI_SYSTEM_LOGGING
syslog()					XSI_SYSTEM_LOGGING
pthread_mutexattr_gettype()	EXCL	EXCL	EXCL	INCL	XSI_THREAD_MUTEX_EXT
pthread_mutexattr_settype()	EXCL	EXCL	EXCL	INCL	XSI_THREAD_MUTEX_EXT
pthread_attr_getguardsize()	EXCL	EXCL	INCL	INCL	XSI_THREADS_EXT
pthread_attr_setguardsize()	EXCL	EXCL	INCL	INCL	XSI_THREADS_EXT
pthread_getconcurrency()	EXCL	INCL	INCL	INCL	XSI_THREADS_EXT
pthread_setconcurrency()	EXCL	INCL	INCL	INCL	XSI_THREADS_EXT
getitimer()					XSI_TIMERS
setitimer()					XSI_TIMERS
endgrent()					XSI_USER_GROUPS
endutxent()					XSI_USER_GROUPS

IEEE Std 1003.1-2008 API	Security	Safety Base	Safety Extended	General Purpose	POSIX Functionality Categories
getgrent()					XSI_USER_GROUPS
getutxent()					XSI_USER_GROUPS
getutxid()					XSI_USER_GROUPS
getutxline()					XSI_USER_GROUPS
pututxline()					XSI_USER_GROUPS
setgrent()					XSI_USER_GROUPS
setregid()					XSI_USER_GROUPS
setreuid()					XSI_USER_GROUPS
setutxent()					XSI_USER_GROUPS
wcswidth()					XSI_WIDE_CHAR
wcwidth()					XSI_WIDE_CHAR
1123	163	240	330	807	Totals

Notes:

1. For the Safety Base Profile, message queue support is within a partition only (i.e., is not an inter-partition communications mechanism).
2. For the Security and Safety Profiles, mutex support (e.g., *pthread_mutexattr_setprotocol*() API) is _POSIX_THREAD_PRIO_PROTECT only.
3. For the Security and Safety Profiles, memory mapping support (e.g., *mmap*() API) is _POSIX_SHARED_MEMORY_OBJECTS only.

A.1 POSIX API Rules

Multi-Process

If *fork*() is called within the Safety Profile, it must be followed immediately to a call to *exec*().

POSIX Safety Profile Substitutions

In order to limit the amount of APIs that are certified within the Safety Profile API, substitutions were made. The substituted APIs will perform an equivalent operation. The following list details which APIs were substituted – *see Note 22*:

- *ctime*() will be substituted by *ctime_r*(). *See Note 22.*

- *gmtime*() will be substituted by *gmtime_r*(). *See Note 22.*
- *asctime*() will be substituted by *asctime_r*(). *See Note 22.*
- *localtime*() will be substituted by *local_time_r*(). *See Note 22.*
- *bcopy*() will be substituted by *memmove*(). *See Note 22.*
- *bzero*() will be substituted by *memset*(). *See Note 22.*
- *index*() will be substituted by *strchr*(). *See Note 22.*
- *rand*() will be substituted by *rand_r*(). *See Note 22.*
- *srand*() will be substituted by *rand_r*(). *See Note 22.*
- *sprintf*() will be substituted by *snprintf*(). *See Note 22.*
- *strerror*() will be substituted by *strerror_r*(). *See Note 22.*
- *rewind*() will be substituted by *fseek*(). *See Note 22.*
- *strtok*() will be substituted by *strtok_r*(). *See Note 22.*
- *setjmp*() will be substituted by *sigsetjmp*(). *See Note 22.*
- *longjmp*() will be substituted by *siglongjmp*(). *See Note 22.*
- *pthread_attr_getstackaddr*() will be substituted by *pthread_attr_getstack*(). *See Note 22.*
- *pthread_attr_setstackaddr*() will be substituted by *pthread_attr_setstack*(). *See Note 22.*
- *sigprocmask*() will be substituted by *pthread_sigmask*(). *See Note 22.*
- *fdatasync*() will be substituted by *fsync*(). *See Note 22.*
- *vsprintf*() will be substituted by *vsnprintf*(). *See Note 22.*
- *printf*() will be substituted by *fprintf*(). *See Note 22.*
- *signal*() will be substituted by *sigaction*(). *See Note 22.*

B I/O Interface API

B.1 Introduction

The I/O Services API will provide a normalized interface for Platform-Specific Components to access the I/O Services Segment. Each instance of the Platform-Specific Services (PSS) Segment and I/O Services Segment will also contain an instance of the I/O Service Interface library. The I/O Services API and the associated library will abstract the components of the PSS Segment and the I/O Services Segment from the specific transport mechanism needed to access platform device data, and to facilitate a distributed, modular platform device interface. The I/O Interface is described in Section 3.4. *See Note 6.*

In cases where the IDL differs from the example source code, the IDL takes precedence.

Note: All IDLs have an identical accompanying source code file that can be used for compilation. The code in this document is formatted to align within Microsoft Word® document formatting constraints.

B.2 I/O Services API and Message Definitions

```
module FACE {

    typedef string<30> INTERFACE_NAME_TYPE;
    typedef long MAX_MSG_SIZE_TYPE; /* maximum message length in bytes */
    typedef long MESSAGE_RANGE_TYPE;  /* maximum number of messages */
    typedef long MESSAGE_LENGTH_TYPE; /*for 1553 = # of words, for A429 = 32
bits, serial = message length in bytes */
    typedef long long SYSTEM_TIME_TYPE; /* 64-bit signed integer with a 1
nanosecond LSB */
    const long long INF_TIME_VALUE = -1;

    enum  ADDR_LEN
    {
        IPV4,
        IPV6
    };

    /* Defined by the I/O Services configuration file */
    enum PORT_DIRECTION_TYPE
    {
        SOURCE_PORT,  /*maps to O_WRONLY (ARINC 653 and POSIX)*/
        DESTINATION_PORT,  /*maps to O_RDONLY (ARINC 653 and POSIX)*/
        BI_DIRECTIONAL_PORT,  /*maps to O_RDWR (POSIX ONLY)*/
        PORT_NOT_DEFINED
    };

    typedef SYSTEM_TIME_TYPE TIMEOUT_TYPE;  /*millisecond precision*/
```

```
typedef string<40> IP_ADDRESS_TYPE;

struct SELECTION_TYPE
{
    long channel_number;    /*Used for A429, Serial Ports*/
    long terminal_address;  /*Used for 1553*/
    long subaddress;        /*Used for 1553*/
    IP_ADDRESS_TYPE ip_address;
    ADDR_LEN ip_protocol;
};

enum BUS_TYPE
{
    MIL_STD_1553,
    ARINC_429,
    SERIAL,
    DISCRETE,
    ANALOG,
    SYNCHRO
};

enum MODE_TYPE
{
    BC,
    RT,
    BM,
    TX,
    RX
};

struct CONFIGURATION_TYPE
{
    SELECTION_TYPE selection;
    BUS_TYPE bus;
    MODE_TYPE mode;
    PORT_DIRECTION_TYPE direction;
    MAX_MSG_SIZE_TYPE size;
    MESSAGE_RANGE_TYPE num_messages;  /* Queuing ports */
};

enum IO_CONNECTION_STATUS_TYPE
{
    GO,
    NOGO,
    IDLE,
    DEGRADED
};

struct STATUS_TYPE
{
    SYSTEM_TIME_TYPE last_message_time;
    boolean message_available;
    IO_CONNECTION_STATUS_TYPE connection_status;
};

enum RETURN_CODE_TYPE
{
    NO_ERROR,  /* request valid and operation performed */
    NO_ACTION,
    NOT_AVAILABLE,  /* no message was available */
```

```
        ADDR_IN_USE,  /* address currently in use */
        INVALID_PARAM,
        INVALID_CONFIG, /* parameter incompatible with configuration */
        PERMISSION_DENIED, /* no permission to send or connecting to wrong
partition */
        INVALID_MODE,  /* request incompatible with current mode */
        TIMED_OUT, /* the time expired before the request could be filled */
        MESSAGE_STALE, /* current time – timestamp exceeds configured limits */
        CONNECTION_IN_PROGRESS, /* asynchronous connection in progress */
        CONNECTION_CLOSED
    };

    enum MESSAGE_TYPE
    {
        COMMAND,
        STATUS,
        DATA
    };

    enum RELIABILITY_TYPE
    {
        RELIABLE,
        NON_RELIABLE
    };

    enum READ_WRITE_BEHAVIOR_TYPE
    {
        QUEUING,
        SAMPLING
    };

    enum COMMAND_TYPE
    {
        OPEN,
        REGISTER,
        READ,
        WRITE,
        CLOSE
    };

    enum A429_STATUS_TYPE
    {
        A429_IO_HW_FIFO_OVERFLOW,
        A429_IO_SW_CIRCULAR_BUFF_OVERFLOW,
        A429_IO_HW_ADDRESS_ERROR,
        A429_IO_HW_SEQUENCE_ERROR,
        A429_IO_HW_PARITY_ERROR,
        A429_IO_CLOCK_LOSS,
        A429_IO_UNKNOWN_ERROR
    };

    enum M1553_STATUS_TYPE
    {
        M1553_BC_IO_NO_RESPONSE,
        M1553_BC_IO_LOOP_TEST_FAIL,
        M1553_BC_IO_MSG_RETRIED,
        M1553_BC_IO_BAD_DATA_BLOCK,
        M1553_BC_IO_ADDRESS_ERROR,
        M1553_BC_IO_WORD_COUNT_ERROR,
        M1553_BC_IO_SYNC_ERROR,
```

```
    M1553_BC_IO_INVALID_WORD,
    M1553_RT_IO_TERMINAL_FLAG,
    M1553_RT_IO_SUBSYSTEM_FLAG,
    M1553_RT_IO_SERVICE_REQUEST,
    M1553_RT_IO_BUSY,
    M1553_RT_IO_DYNAMIC_BC,
    M1553_RT_IO_NO_RESPONSE,
    M1553_RT_IO_LOOP_TEST_FAIL,
    M1553_RT_IO_ILLEGAL_COMMAND_WORD,
    M1553_RT_IO_WORD_COUNT_ERROR,
    M1553_RT_IO_SYNC_ERROR,
    M1553_RT_IO_INVALID_WORD,
    M1553_RT_IO_RT_RT_GAP_SYNC_ADDR_ERROR,
    M1553_RT_IO_RT_RT_2ND_CMD_ERROR,
    M1553_RT_IO_COMMAND_WORD_ERROR,
    M1553_BM_IO_NO_RESPONSE,
    M1553_BM_IO_WORD_COUNT_ERROR,
    M1553_BM_IO_SYNC_ERROR,
    M1553_BM_IO_INVALID_WORD,
    M1553_BM_IO_RT_RT_GAP_SYNC_ADDR_ERROR,
    M1553_BM_IO_RT_RT_2ND_CMD_ERROR,
    M1553_BM_IO_COMMAND_WORD_ERROR,
    M1553_BM_IO_BAD_DATA_BLOCK,
    M1553_BM_IO_MESSAGE_ERROR,
    M1553_BM_IO_INSTRUMENTATION,
    M1553_BM_IO_SERVICE_REQUEST,
    M1553_BM_IO_RESERVED_BITS,
    M1553_BM_IO_BROADCAST_RCVD,
    M1553_BM_IO_BUSY,
    M1553_BM_IO_SF,
    M1553_BM_IO_DYNAMIC_BC,
    M1553_BM_IO_TF
};

enum SERIAL_STATUS_TYPE
{
    SERIAL_OVERRRUN_ERROR,
    SERIAL_PARITY_ERROR,
    SERIAL_FRAMING_ERROR,
    SERIAL_BREAK_ERROR
};

const short DISCRETE_ERROR = 1;

// OMG IDL DOES NOT SUPPORT POINTERS, FOR NOW WE ARE
// SPECIFYING POINTER TYPES AS STRINGS and list their
// language-specific implementation in sections following
// the definitions.

// typedef void* MESSAGE_ADDR_TYPE;
// typedef void* INTERFACE_HANDLE_TYPE;
//    typedef void (*CALLBACK_ADDRESS_TYPE)
//        (in MAX_MSG_SIZE_TYPE, in MESSAGE_ADDR_TYPE);
typedef string MESSAGE_ADDR_TYPE;
typedef string INTERFACE_HANDLE_TYPE;
typedef string CALLBACK_ADDRESS_TYPE;

interface IO {
    void Open(
        in INTERFACE_NAME_TYPE name,
```

```
            in CONFIGURATION_TYPE config_info,
            in TIMEOUT_TYPE timeout,
            out INTERFACE_HANDLE_TYPE handle,
            out RETURN_CODE_TYPE return_code);

        void Register (
            in INTERFACE_HANDLE_TYPE handle,
            in CALLBACK_ADDRESS_TYPE callback_address,
            out RETURN_CODE_TYPE return_code);

        void Read (
            in INTERFACE_HANDLE_TYPE handle,
            in TIMEOUT_TYPE timeout,
            out MESSAGE_LENGTH_TYPE message_length,
            out MESSAGE_ADDR_TYPE data_buffer,
            out RETURN_CODE_TYPE return_code);

        void Write (
            in INTERFACE_HANDLE_TYPE handle,
            in TIMEOUT_TYPE timeout,
            in MESSAGE_LENGTH_TYPE message_length,
            in MESSAGE_ADDR_TYPE data_buffer,
            out RETURN_CODE_TYPE return_code);

        void Get_Status (
            in INTERFACE_HANDLE_TYPE handle,
            out STATUS_TYPE status,
            out RETURN_CODE_TYPE return_code);

        void Close (
            in INTERFACE_HANDLE_TYPE handle,
            out RETURN_CODE_TYPE return_code);
    };
};
```

B.3 Language-Specific Data Type Definitions

C & C++ Definitions

```
typedef void* FACE_MESSAGE_ADDR_TYPE;
typedef void* FACE_INTERFACE_HANDLE_TYPE;
typedef void (*FACE_CALLBACK_ADDRESS_TYPE)
    (/* in */ MAX_MSG_SIZE_TYPE, /* in */ MESSAGE_ADDR_TYPE);
```

Example C Style Data Type Declarations

(OMG IDL C Language Mapping Specification v1.0)

```
typedef char FACE_INTERFACE_NAME_TYPE [30];
enum  FACE_ADDR_LEN {
    FACE_IPV4,
    FACE_IPV6
};
struct FACE_SELECTION_TYPE {
    long channel_number;    /*Used for A429, Serial Ports*/
    long terminal_address;  /*Used for 1553*/
    ...
```

```
};
```

Example C++ Style Data Type Declarations

(OMG IDL C++ Language Mapping Specification v1.2)

```
namespace FACE {
    typedef char INTERFACE_NAME_TYPE [30];
    enum ADDR_LEN {
        IPV4,
        IPV6
    };
    struct SELECTION_TYPE {
        long channel_number;    /*Used for A429, Serial Ports*/
        long terminal_address;  /*Used for 1553*/
    ...
    };
};
```

B.4 Open Function

The *FACE_IO_Open*() function call shall allow for the PSS component to create a connection through a POSIX socket, ARINC queuing port, or ARINC sampling port in a distributed system or, if in the same partition, may be a direct function call into the I/O Service component.

```
/* C style declaration */
void FACE_IO_Open (/*IN*/  FACE_INTERFACE_NAME_TYPE name,
                   /*IN*/  FACE_CONFIGURATION_TYPE config_info,
                   /*IN*/  FACE_TIMEOUT_TYPE timeout,
                   /*OUT*/ FACE_INTERFACE_HANDLE_TYPE *handle,
                   /*OUT*/ FACE_RETURN_CODE_TYPE *return_code);

/* C++ style declaration */
void FACE::IO::Open (/*IN*/  FACE::INTERFACE_NAME_TYPE name,
                     /*IN*/  FACE::CONFIGURATION_TYPE config_info,
                     /*IN*/  FACE::TIMEOUT_TYPE timeout,
                     /*OUT*/ FACE::INTERFACE_HANDLE_TYPE &handle,
                     /*OUT*/ FACE::RETURN_CODE_TYPE &return_code);
```

The *FACE_IO_OPEN*() call gets access to a port that was created during initialization by the I/O Interface support libraries using the configuration file information. The *config_info* parameter is for verification only and will return a CONFIG_ERROR if there is a mismatch from the parameters used in port creation. *FACE_IO_OPEN*() will be non-blocking regardless of the underlying transport mechanism.

B.5 Register Function

The *FACE_IO_Register*() function call shall allow for the PSS component to register for a data receiving connection. *FACE_IO_Register*() shall register for a separate thread or process that will poll the connection for data and callback the appropriate PSS component when data is available on the appropriate POSIX socket, ARINC queuing port, or ARINC sampling port in a distributed system or if in the same partition, a direct function call into the I/O service

component. This function call will allow for the PSS component to subscribe to a specific connection and be notified when data is available.

```
/* C style declaration */
void FACE_IO_Register (/*IN*/  FACE_INTERFACE_HANDLE_TYPE handle,
                       /*IN*/  FACE_CALLBACK_ADDRESS_TYPE callback_address,
                       /*OUT*/ FACE_RETURN_CODE_TYPE *return_code);

/* C++ style declaration */
void FACE::IO::Register (/*IN*/  FACE::INTERFACE_HANDLE_TYPE handle,
                         /*IN*/  FACE::CALLBACK_ADDRESS_TYPE callback_address,
                         /*OUT*/ FACE::RETURN_CODE_TYPE &return_code);
```

The function is not necessary but offers a mechanism to avoid polling ports unnecessarily. When a new message is written to the port, the *callback_address* is called with the message data and message length.

B.6 Read Function

The *FACE_IO_Read*() function call shall allow for the PSS component to poll a data receiving connection. *FACE_IO_Read*() shall read data for a connection addressed by *handle* on the appropriate POSIX socket, ARINC queuing port, or ARINC sampling port in a distributed system or, if in the same partition, a direct function call into the I/O Service component.

```
/* C style declaration */
void FACE_IO_Read (/*IN*/  FACE_INTERFACE_HANDLE_TYPE handle,
                   /*IN*/  FACE_TIMEOUT_TYPE timeout,
                   /*OUT*/ FACE_MESSAGE_LENGTH_TYPE *message_length,
                   /*OUT*/ FACE_MESSAGE_ADDR_TYPE *data_buffer,
                   /*OUT*/ FACE_RETURN_CODE_TYPE *return_code);

/* C++ style declaration */
void FACE::IO::Read (/*IN*/  FACE::INTERFACE_HANDLE_TYPE handle,
                     /*IN*/  FACE::TIMEOUT_TYPE timeout,
                     /*OUT*/ FACE::MESSAGE_LENGTH_TYPE &message_length,
                     /*OUT*/ FACE::MESSAGE_ADDR_TYPE &data_buffer,
                     /*OUT*/ FACE::RETURN_CODE_TYPE &return_code);
```

The *FACE_IO_READ*() call will write the number of bytes contained in the message to the location given by *data_buffer* and copy the number of bytes to *message_length*. The timestamp from the message is checked and if it is too old, *return_code* is set to MESSAGE_STALE. The call will block until a message is available or timeout has expired. The timestamp is recorded for use in the status call.

B.7 Write Function

The *FACE_IO_Write*() function call shall allow for the PSS component to send data to a specified data connection. *FACE_IO_Write*() shall write data for a connection addressed by *handle* on the appropriate POSIX socket, ARINC queuing port, or ARINC sampling port in a distributed system or if in the same partition, a direct function call into the I/O Service component.

```
/* C style declaration */
void FACE_IO_Write (/*IN*/  FACE_INTERFACE_HANDLE_TYPE handle,
```

```
                    /*IN*/  FACE_TIMEOUT_TYPE timeout,
                    /*IN*/  FACE_MESSAGE_LENGTH_TYPE message_length,
                    /*IN*/  FACE_MESSAGE_ADDR_TYPE data_buffer,
                    /*OUT*/ FACE_RETURN_CODE_TYPE *return_code);

/* C++ style declaration */
void FACE::IO::Write (/*IN*/  FACE::INTERFACE_HANDLE_TYPE handle,
                      /*IN*/  FACE::TIMEOUT_TYPE timeout,
                      /*IN*/  FACE::MESSAGE_LENGTH_TYPE message_length,
                      /*IN*/  FACE::MESSAGE_ADDR_TYPE data_buffer,
                      /*OUT*/ FACE::RETURN_CODE_TYPE &return_code);
```

The *FACE_IO_WRITE*() call will write the current time to the port indicated by *handle* followed by *message_length* bytes from the location given by *data_buffer*. If the port is a queuing type port and it is full, the call will block until space is available or timeout has expired.

B.8 Get Status Function

The *FACE_IO_Get_Status*() function call shall allow for the PSS component to query a connection for status information. *FACE_IO_Get_Status*() shall receive status data for a connection addressed by *handle* on the appropriate POSIX socket, ARINC queuing port, or ARINC sampling port in a distributed system or, if in the same partition, a direct function call into the I/O Service component.

```
/* C style declaration */
void FACE_IO_Get_Status (/*IN*/  FACE_INTERFACE_HANDLE_TYPE handle,
                         /*OUT*/ FACE_STATUS_TYPE *status,
                         /*OUT*/ FACE_RETURN_CODE_TYPE *return_code)

/* C++ style declaration */
void FACE::IO::Get_Status (/*IN*/  FACE::INTERFACE_HANDLE_TYPE handle,
                           /*OUT*/ FACE::STATUS_TYPE &status,
                           /*OUT*/ FACE::RETURN_CODE_TYPE &return_code)
```

Copy the current status into the address specified in *status*.

B.9 Close Function

The *FACE_IO_Close*() function call shall allow for the PSS component to close a connection and release the addressed *handle*.

```
/* C style declaration */
void FACE_IO_Close (/*IN*/  FACE_INTERFACE_HANDLE_TYPE handle,
                    /*OUT*/ FACE_RETURN_CODE_TYPE *return_code)

/* C++ style declaration */
void FACE::IO::Close (/*IN*/  FACE::INTERFACE_HANDLE_TYPE handle,
                      /*OUT*/ FACE::RETURN_CODE_TYPE &return_code)
```

Release the connection to the port and erase any information being stored about the connection.

B.10 I/O Interface API Message Set

I/O Service Message Format				
31	23	15	7	0
BUS_TYPE (8 bits)	PAYLOAD_LENGTH (24 bits)			
MESSAGE_TYPE (16 bits)		CM_ST (14 bits)	IC	TS
PAYLOAD_DATA (N Bytes align on 32-bit boundary) . . . PAYLOAD_DATA (N Bytes align on 32-bit boundary)				
TIMESTAMP_MSW (32 bits)				
TIMESTAMP_LSW (32 bits)				
CHECKSUM (32 bits includes header and payload)				

Figure 34: I/O Service Interface Message Definitions

Detailed Message Field Descriptions:

1. **BUS_TYPE**: 8 bits defining the type of I/O Bus the message is related to:
   - — MILS_STD_1553
   - — ARINC_429
   - — SERIAL
   - — DISCRETE
   - — ANALOG
   - — SYNCHRO
2. **LENGTH**: 24-bit value indicating the Payload Length in bytes
3. **MESSAGE_TYPE**: 16-bit value indicating the type of the message:
   - — COMMAND
   - — STATUS
   - — DATA
4. **CM/ST FIELD**: 14 bits representing:
5. **COMMAND**: to be processed if MESSAGE_TYPE = COMMAND:
   - — OPEN

— REGISTER

— READ

— WRITE

— CLOSE

6. **STATUS**: Returned if MESSAGE_TYPE = STATUS:

— ARINC-429 STATUS if BUS_TYPE = ARINC_429

- A429_IO_HW_FIFO_OVERFLOW
- A429_IO_SW_CIRCULAR_BUFF_OVERFLOW
- A429_IO_HW_ADDRESS_ERROR
- A429_IO_HW_SEQUENCE_ERROR
- A429_IO_HW_PARITY_ERROR
- A429_IO_CLOCK_LOSS
- A429_IO_UNKNOWN_ERROR

— MIL-STD-1553 STATUS if BUS_TYPE = MIL_STD_1553:

- M1553_BC_IO_NO_RESPONSE
- M1553_BC_IO_LOOP_TEST_FAIL
- M1553_BC_IO_MSG_RETRIED
- M1553_BC_IO_BAD_DATA_BLOCK
- M1553_BC_IO_ADDRESS_ERROR
- M1553_BC_IO_WORD_COUNT_ERROR
- M1553_BC_IO_SYNC_ERROR
- M1553_BC_IO_INVALID_WORD
- M1553_RT_IO_TERMINAL_FLAG
- M1553_RT_IO_SUBSYSTEM_FLAG
- M1553_RT_IO_SERVICE_REQUEST
- M1553_RT_IO_BUSY
- M1553_RT_IO_DYNAMIC_BC
- M1553_RT_IO_NO_RESPONSE
- M1553_RT_IO_LOOP_TEST_FAIL
- M1553_RT_IO_ILLEGAL_COMMAND_WORD

- M1553_RT_IO_WORD_COUNT_ERROR
- M1553_RT_IO_SYNC_ERROR
- M1553_RT_IO_INVALID_WORD
- M1553_RT_IO_RT_RT_GAP_SYNC_ADDR_ERROR
- M1553_RT_IO_RT_RT_2ND_CMD_ERROR
- M1553_RT_IO_COMMAND_WORD_ERROR
- M1553_BM_IO_NO_RESPONSE
- M1553_BM_IO_WORD_COUNT_ERROR
- M1553_BM_IO_SYNC_ERROR
- M1553_BM_IO_INVALID_WORD
- M1553_BM_IO_RT_RT_GAP_SYNC_ADDR_ERROR
- M1553_BM_IO_RT_RT_2ND_CMD_ERROR
- M1553_BM_IO_COMMAND_WORD_ERROR
- M1553_BM_IO_BAD_DATA_BLOCK
- M1553_BM_IO_MESSAGE_ERROR
- M1553_BM_IO_INSTRUMENTATION
- M1553_BM_IO_SERVICE_REQUEST
- M1553_BM_IO_RESERVED_BITS
- M1553_BM_IO_BROADCAST_RCVD
- M1553_BM_IO_BUSY
- M1553_BM_IO_SF
- M1553_BM_IO_DYNAMIC_BC
- M1553_BM_IO_TF

— Serial Status Values if BUS_TYPE = SERIAL:

- SERIAL_OVERRRUN_ERROR
- SERIAL_PARITY_ERROR
- SERIAL_FRAMING_ERROR
- SERIAL_BREAK_ERROR

— Discrete Values if BUS_TYPE = DISCRETE:

- DISCRETE_ERROR

— All 0s if MESSAGE_TYPE = DATA

7. **IC**: 1-bit indication whether or not to implement an integrity check. If set to 1 the receiving entity will calculate the checksum on the header and data and compare to the message CHECKSUM field. If set to 0 then the CHECKSUM field can be ignored.

8. **TS**: 1 bit indicating that a timestamp is required.

9. **Payload Data**: N Number of bytes representing the Payload for the appropriate message type. For DATA messages it will be the data that will be sent to or received from the interface defined by IO_TYPE. For example:

   — A typical Payload for a MIL-STD-1553 message would include the RT number, SA value, the *x* number of 16-bit messages, and a buffer containing the *x* number of 16-bit messages.

   — For a command message the Payload would contain the parameters needed to execute the command.

10. **TIMESTAMP**: Signed 64-bit value representing the time (in nanoseconds) that Payload was received from the I/O device or was packed by the PSS Segment. All interfaces may use the system Up-Time for time stamping. TIMESTAMP should only be required for MESSAGE_TYPE = DATA.

11. **CHECKSUM**: The checksum is the algebraic sum of the message byte values. Sum all message bytes excluding the CHECKSUM field itself. Append the result as a 32-bit value (ignore overflow).

MIL-STD-1553 Payload Message Format

1. **BUS_ID**: Identify the MIL-STD-1553 BUS.

2. **RT_NUMBER**: Remote Terminal Address (0 to 31).

3. **SA_NUMBER**: Sub Address Number (0 to 31).

4. **NUM_MSG**: Number of 16-bit messages in the Data Buffer (0 - 31).

5. **DATA_BUFFER**: NUM_MSG 16-bit messages.

MIL-STD-1553 Message Payload Format			
31 23	15	7	0
BUS_ID (8 bits)	RT_NUMBER (8 bits)	SA_NUMBER (8 bits)	NUM_MSG (8 bits)
DATA_BUFFER (NUM_MSG 16-bit buffers)			
.			
.			
.			
DATA_BUFFER (NUM_MSG 16-bit buffers)			

Figure 35: MIL-STD-1553 Message Payload Format

ARINC 429 Payload Message Format

1. **CHANNEL**: The ARINC 429 Channel that the message is transmitted or received on (1 to *x*).
2. **NUM_LABELS**: The number of labels included in the ARINC 429 Label Payload.
3. **ARINC 429 Label Payload**: The Data Payload of 1-to-many ARINC 429 Labels.

ARINC 429 Message Payload Format				
31	23	15	7	0
CHANNEL (16 bits)		NUM_LABELS (16 bits)		
ARINC_429_LABEL (1)				
.				
.				
.				
ARINC_429_LABEL (NUM_LABELS)				

Figure 36: ARINC 429 Message Payload Format

Asynchronous Serial Payload Message Format

1. **CHANNEL**: Serial channel on which the message is transmitted or received.
2. **BYTE_COUNT**: Number of bytes in the data buffer.
3. **DATA_BUFFER**: BYTE_COUNT bytes of serial data.

Serial Message Payload Format				
31	23	15	7	0
CHANNEL (8 bits)	BYTE_COUNT (16 bits)		DATA_BUFFER	
DATA_BUFFER				
.				
.				
DATA_BUFFER (BYTE_COUNT size)				

Figure 37: Serial Message Payload Format

Discrete Payload Message Format

Discrete Message Payload Format				
31	23	15	7	0
ST	DISCRETE_CHAN_NUM (30 bits)			

Figure 38: Discrete Message Payload

1. **ST**: STATE of the discrete (0 – 2):

   — 0 = Off

   — 1 = On

   — 2 = Tristate

2. **DISCRETE CHANNEL NUMBER**: Defines the number of the discrete to be set to the value in ST.

B.11 I/O Services Segment Configuration File

The following information describes the parameters that make up the I/O Services Segment Configuration file:

Connection Name	Name of the connection referenced.
Connection Type	Type of connection (e.g., sampling, queuing, socket, direct API).
Connection Direction	Direction of the connection.
Create Connection	Flag to tell I/O Services if it needs to create the connection. Queuing Ports and Sampling Ports are pre-created by the OS.
Port Name	Name of Queuing port or Sampling port.
Message Size	Size of the data.
Message Range	Number of message a lot for in the connection.
Refresh Period	Max allowable age of valid message
Reliability	Reliable or Non-Reliable.
ReadWriteBehavior	Buffering or Sampling.
Queue Discipline	Priority or FIFO.
Connection Domain	POSIX Socket Domain.
Socket Type	POSIX Socket Type.
Receive Flag	Receive Flag for POSIX Sockets.
Send Flag	Send Flag for POSIX Sockets.
Source Address	Source IP address.
Destination Address	Destination IP address.
Source Port	Source Port.

Destination Port	Destination Port.
IO Type	IO type to associate additional config parameters (e.g., 429, 1553, serial, etc.)

Serial Port Configuration Data

SERIAL_IO_CHAN_NUM	Physical Serial Port Number
SERIAL_IO_MODE	Serial Port operational mode (e.g., RS-232, RS-422, RS-485)
SERIAL_IO_BAUD_RATE	Baud Rate for serial port
SERIAL_IO_DATA_BITS	Number of data bits to be configured
SERIAL_IO_STOP_BITS	Number of Stop Bits for serial port
SERIAL_IO_PARITY	Parity for serial port

ARINC 429 Configuration Data

A429_CHANNEL_NUM	Channel Number
A429_IO_DIRECTION	Channel Direction (TX or RX)
A429_IO_PARITY	Parity for channel
A429_IO_CHAN_SPEED	Channel speed (High or Low)

MIL-STD-1553 Configuration Data

M1553_CHANNEL	Channel Number each channel is dual redundant
M1553_MODE	Channel Mode (BC, RT, or BM)

Discrete I/O Configuration Data

DISCRETE_IO_MAX_INPUTS	Maximum number of discrete inputs
DISCRETE_IO_MAX_OUTPUTS	Maximum number of discrete outputs
DISCRETE_IO_CHANNEL_NUM	Discrete channel number
DISCRETE_IO_DIRECTION	Channel Direction (In or Out)
DISCRETE_IO_INITIAL_OUTPUT_VALUE	Initial Value (True or False)

C Transport Interface API

C.1 Introduction

The Transport Interface is defined by an abstraction interface allowing portable components to access transport mechanisms used by the Transport Services Segment library. These mechanisms include queues, sockets, sampling ports, etc. The goal of the Transport Interface is to enhance portability by abstracting multiple standards-based transport mechanism interfaces from the portable component. The Transport Interface and Transport Services Segment are described in Section 3.6 and Section 3.7, respectively.

In cases where the IDL differs from the example source code, the IDL takes precedence.

Note: All IDLs have an identical accompanying source code file that can be used for compilation. The code in this document is formatted to align within Microsoft Word® document formatting constraints.

C.2 Data Types

```
module FACE {

    typedef string<64> CONNECTION_NAME_TYPE;
    typedef long MESSAGE_SIZE_TYPE;

    /* Defined by the Transport Services configuration file */
    enum CONNECTION_DIRECTION_TYPE {
        SOURCE,             /*maps to O_WRONLY (ARINC 653 and POSIX)*/
        DESTINATION,        /*maps to O_RDONLY (ARINC 653 and POSIX)*/
        BI_DIRECTIONAL,     /*maps to O_RDWR (POSIX ONLY)*/
        ONE_WAY_REQUEST_SOURCE,
        ONE_WAY_REQUEST_DESTINATION,
        TWO_WAY_REQUEST_SYNCHRONOUS_SOURCE,
        TWO_WAY_REQUEST_SYNCHRONOUS_DESTINATION,
        TWO_WAY_REQUEST_REPLY_ASYNCHRONOUS_SOURCE,
        TWO_WAY_REQUEST_REPLY_ASYNCHRONOUS_DESTINATION,
        NOT_DEFINED_CONNECTION_DIRECTION_TYPE
    };

    /* SYSTEM_TIME_TYPE ALSO DEFINED IN IO SECTION */
    typedef long long SYSTEM_TIME_TYPE; /* 64-bit signed integer with a 1
nanosecond LSB */
    typedef long CONNECTION_ID_TYPE; /* two connection IDs for socketpair */
    typedef long long SEQUENCE_ID_TYPE; /* used to tie together
request/response messages */

    /* Defined by the Transport Services configuration file */
    enum CONNECTION_TYPE {
        SAMPLING_PORT,   /* ARINC 653 sampling port */
        QUEUING_PORT,    /* ARINC 653 queuing port */
```

```
        SOCKET,          /* POSIX socket */
        MQ,              /* POSIX queue */
        SHM,             /* POSIX shared memory */
        CORBA,           /* CORBA */
        DDS              /* Data Distribution Services */
    };

    /* RETURN_CODE_TYPE Also defined in IO Datatypes */
    enum RETURN_CODE_TYPE {
        NO_ERROR,       /* request valid and operation performed */
        NO_ACTION,      /* status of system unaffected by request */
        NOT_AVAILABLE,  /* resource required by request unavailable */
        ADDR_IN_USE,    /* address currently in use */
        INVALID_PARAM,  /* invalid parameter specified in request */
        INVALID_CONFIG, /* parameter incompatible with configuration */
        PERMISSION_DENIED, /* no permission to send or connecting to wrong
partition */
        INVALID_MODE,   /* request incompatible with current mode */
        TIMED_OUT,      /* time-out tied up with request has expired */
        MESSAGE_STALE,  /*current timestamp exceeds configured limits*/
        CONNECTION_IN_PROGRESS, /* asynchronous connection in progress */
        CONNECTION_CLOSED
    }; /* POSIX ERRONO mapped to these codes by transport services */

    // MESSAGE_RANGE_TYPE ALSO DEFINED IN IO SECTION
    typedef long MESSAGE_RANGE_TYPE;

    typedef boolean WAITSET_TYPE[32];

    struct FACE_SPECIFIED_TYPE {
/* The value below is a placeholder for the actual structure definition of
Pub/Sub data. When used in practice, "FACE_SPECIFIED_STRUCT" will be replaced
by either the actual name of the data structure or “void *” as dictated by the
connection’s configuration. */
        octet structure_placeholder;
    };

    interface FACE_SPECIFIED_INTERFACE {
/* The method below is a placeholder for the actual interface methods. For the
actual interface, "FACE_SPECIFIED_INTERFACE" will be replaced by the actual
name of the service. */
        void placeholder();
    };

    interface Read_Callback {
        void send_event( in FACE_SPECIFIED_TYPE message,
        in MESSAGE_SIZE_TYPE message_size,
        in SEQUENCE_ID_TYPE sequence_id,
        in WAITSET_TYPE waitset);
    };

    /* Defined by the Transport Services configuration file */
    enum QUEUING_DISCIPLINE_TYPE {
        FIFO,
        PRIORITY,
        NOT_DEFINED_QUEUING_DISCIPLINE_TYPE
    };

    /* Defined by the Transport Services configuration file */
    enum CONNECTION_DOMAIN_TYPE {
```

```
    UNIX,     /* AF_UNIX */
    INET,     /* AF_INET */
    NOT_DEFINED_CONNECTION_DOMAIN_TYPE
};

/* Defined by the Transport Services configuration file */
enum SOCKET_TYPE {
    STREAM,      /* SOCK_STREAM */
    DGRAM,       /* SOCK_DGRAM */
    SEQPACKET,   /* SOCK_SEQPACKET */
    NOT_DEFINED_SOCKET_TYPE
};

/* Defined by the Transport Services configuration file */
enum RECEIVE_FLAG_TYPE {
    PEEK,                   /* SOCK_STREAM */
    OOB_RECEIVE_FLAG_TYPE,  /* SOCK_DGRAM */
    WAITALL,                /* SOCK_SEQPACKET */
    NOT_DEFINED_RECEIVE_FLAG_TYPE
};

/* Defined by the Transport Services configuration file */
enum SEND_FLAG_TYPE {
    EOR,                /* SOCK_STREAM*/
    OOB_SEND_FLAG_TYPE,  /* SOCK_DGRAM*/
    NOSIGNAL,           /* SOCK_SEQPACKET*/
    NOT_DEFINED_SEND_FLAG_TYPE
};

enum VALIDITY_TYPE {
    INVALID,     /* Also “empty” and “down” */
    VALID        /* Also “occupied” and “UP” */
};

/* Addressed in Validity_Type */
/* typedef enum { EMPTY = 0, OCCUPIED = 1} EMPTY_INDICATOR_TYPE; */
/* typedef enum { DOWN = 0, UP = 1} EVENT_STATE_TYPE; */

/* Support publish/subscribe and request/response */
enum MESSAGING_PATTERN_TYPE {
    PUB_SUB,
    CLIENT,
    SERVER
};

/* Connection provides reliable or non-reliable transport */
enum RELIABILITY_TYPE {
    RELIABLE,
    NON_RELIABLE
};

/* Connection provides queuing or sampling behavior */
enum READ_WRITE_BEHAVIOR_TYPE {
    QUEUING,
    SAMPLING
};

typedef long WAITING_RANGE_TYPE;
typedef string FILTER_SPECIFICATION_TYPE;
typedef long FILTER_VALUE_TYPE[64];
```

```
    /* Elements defined by the Transport Services configuration file */
    struct TRANSPORT_CONNECTION_STATUS_TYPE {
        MESSAGE_RANGE_TYPE MESSAGE;
        MESSAGE_RANGE_TYPE MAX_MESSAGE;
        MESSAGE_SIZE_TYPE MAX_MESSAGE_SIZE;
        CONNECTION_DIRECTION_TYPE CONNECTION_DIRECTION;
        WAITING_RANGE_TYPE WAITING_PROCESSES_OR_MESSAGES;
        SYSTEM_TIME_TYPE REFRESH_PERIOD; /* Maximum age of a valid message */
        VALIDITY_TYPE LAST_MSG_VALIDITY;
    };

    /* Defined by the Transport Services configuration file */
    /* Struct for TSS to store connection configuration Data */
    struct CONNECTION_ATTR_STRUCT
    {
        CONNECTION_NAME_TYPE connection_name;
        CONNECTION_TYPE connection;
        CONNECTION_DOMAIN_TYPE connection_domain;
        SOCKET_TYPE socket;
        MESSAGE_SIZE_TYPE message_size;
        MESSAGE_RANGE_TYPE message_range;
        CONNECTION_DIRECTION_TYPE connection_direction;
        MESSAGING_PATTERN_TYPE messaging_pattern;
        RELIABILITY_TYPE reliability;
        READ_WRITE_BEHAVIOR_TYPE read_write_behavior;
        SYSTEM_TIME_TYPE refresh_period;
        QUEUING_DISCIPLINE_TYPE connection_discipline;
    };

    interface TS {
        void create_connection(
            in  CONNECTION_NAME_TYPE connection_name,
            in  MESSAGING_PATTERN_TYPE pattern,
            in  FILTER_SPECIFICATION_TYPE filter_spec,
            out CONNECTION_ID_TYPE connection_id,
            out CONNECTION_DIRECTION_TYPE connection_direction,
            out MESSAGE_SIZE_TYPE message_size,
            inout FACE_SPECIFIED_INTERFACE server, /* only used for attaching
to a server */
            out RETURN_CODE_TYPE return_code);

        void destroy_connection(
            in CONNECTION_ID_TYPE connection_id,
            out RETURN_CODE_TYPE return_code );

        void receive_message (
            in  CONNECTION_ID_TYPE connection_id,
            in  SYSTEM_TIME_TYPE timeout,
            in  FILTER_VALUE_TYPE filter_vals,
            inout SEQUENCE_ID_TYPE sequence_id,
            out FACE_SPECIFIED_TYPE message,
            out MESSAGE_SIZE_TYPE message_size,
            out RETURN_CODE_TYPE return_code);

        void send_message (
            in  CONNECTION_ID_TYPE connection_id,
            in  FACE_SPECIFIED_TYPE message,
            in  MESSAGE_SIZE_TYPE message_size, /* if set to 0, TSS will use
the size of the data type */
```

```
            in SYSTEM_TIME_TYPE timeout,
            inout SEQUENCE_ID_TYPE sequence_id,
            out RETURN_CODE_TYPE return_code);

        void register_read_callback (
            in  CONNECTION_ID_TYPE connection_id,
            in  WAITSET_TYPE waitset,
            in  Read_Callback data_callback,
            in MESSAGE_SIZE_TYPE max_message_size,
            out RETURN_CODE_TYPE return_code);

        void get_connection_parameters (
            inout CONNECTION_NAME_TYPE connection_name,
            inout CONNECTION_ID_TYPE connection_id,
            out   TRANSPORT_CONNECTION_STATUS_TYPE connection_status,
            out   RETURN_CODE_TYPE return_code);
    };
};
```

C.3 C Data Types

```
typedef char FACE_CONNECTION_NAME_TYPE [64];
typedef long FACE_MESSAGE_SIZE_TYPE;

/* All Transport Services configuration file connections are defined by the
CONNECTION_NAME key */
/* Defined by the Transport Services configuration file */
    enum FACE_CONNECTION_DIRECTION_TYPE {
        /* 0 */ SOURCE,      /*maps to O_WRONLY (ARINC 653 and POSIX)*/
        /* 1 */ DESTINATION, /*maps to O_RDONLY (ARINC 653 and POSIX)*/
        /* 2 */ BI_DIRECTIONAL,  /*maps to O_RDWR (POSIX ONLY)*/
        /* 3 */ ONE_WAY_REQUEST_SOURCE,
        /* 4 */ ONE_WAY_REQUEST_DESTINATION,
        /* 5 */ TWO_WAY_REQUEST_SYNCHRONOUS_SOURCE,
        /* 6 */ TWO_WAY_REQUEST_SYNCHRONOUS_DESTINATION,
        /* 7 */ TWO_WAY_REQUEST_REPLY_ASYNCHRONOUS_SOURCE,
        /* 8 */ TWO_WAY_REQUEST_REPLY_ASYNCHRONOUS_DESTINATION,
        /* 9 */ NOT_DEFINED_CONNECTION_DIRECTION_TYPE
    };

    typedef long long FACE_SYSTEM_TIME_TYPE; /* 64-bit signed integer with a 1
nanosecond LSB */
    typedef long CONNECTION_ID_TYPE; /* two connection IDs for socketpair */
    typedef long long FACE_SEQUENCE_ID_TYPE; /* used to tie together
request/response messages */

/* Defined by the Transport Services configuration file */
    enum FACE_CONNECTION_TYPE {
        /* 0 */ SAMPLING_PORT,   /* ARINC 653 sampling port */
        /* 1 */ QUEUING_PORT,    /* ARINC 653 queuing port */
        /* 2 */ SOCKET,          /* POSIX socket */
        /* 3 */ MQ,              /* POSIX queue */
        /* 4 */ SHM,             /* POSIX shared memory */
        /* 5 */ CORBA,           /* CORBA */
        /* 6 */ DDS              /* Data Distribution Services */
    };

    enum FACE_RETURN_CODE_TYPE {
        /* 0 */ NO_ERROR,  /* request valid and operation performed */
```

```
        /* 1 */ NO_ACTION, /* status of system unaffected by request */
        /* 2 */ NOT_AVAILABLE, /* resource required by request unavailable */
        /* 3 */ ADDR_IN_USE,   /* address currently in use */
        /* 4 */ INVALID_PARAM, /* invalid parameter specified in request */
        /* 5 */ INVALID_CONFIG, /* parameter incompatible with configuration */
        /* 6 */ PERMISSION_DENIED, /* no permission to send or connecting to
wrong partition */
        /* 7 */ INVALID_MODE, /* request incompatible with current mode */
        /* 8 */ TIMED_OUT,    /* time-out tied up with request has expired */
        /* 9 */ MESSAGE_STALE, /*current timestamp exceeds configured limits*/
        /*10*/ CONNECTION_IN_PROGRESS,/* asynchronous connection in progress */
        /* 11 */ CONNECTION_CLOSED
    };  /* POSIX ERRONO mapped to these codes by transport services */

    typedef long FACE_MESSAGE_RANGE_TYPE;
    typedef boolean FACE_WAITSET_TYPE[32];

    typedef struct FACE_SPECIFIED_STRUCT {
/* The below value is a placeholder for the actual structure definition of
Pub/Sub data. When used in practice, "FACE_SPECIFIED_STRUCT" will be replaced
by either the actual name of the data structure or “void *” as dictated by the
connection’s configuration. */
        sequence <octet> value;
    } FACE_SPECIFIED_TYPE;

    struct FACE_SPECIFIED_INTERFACE {
/* The function pointer below is a placeholder for the actual interrface
methods. For the actual interface, "FACE_SPECIFIED_INTERFACE" will be replaced
by the actual name of the service. */
        void placeholder();
    };

    struct Read_Callback {
        void* send_event(FACE_SPECIFIED_TYPE message,
            MESSAGE_SIZE_TYPE message_size,
            SEQUENCE_ID_TYPE sequence_id,
            WAITSET_TYPE waitset);
    };

    /* Defined by the Transport Services configuration file */
    enum FACE_QUEUING_DISCIPLINE_TYPE {
        /* 0 */ FIFO,
        /* 1 */ PRIORITY,
        /* 2 */ NOT_DEFINED_QUEUING_DISCIPLINE_TYPE
    };

    /* Defined by the Transport Services configuration file */
    enum FACE_CONNECTION_DOMAIN_TYPE {
        /* 0 */ UNIX,      /* AF_UNIX */
        /* 1 */ INET,      /* AF_INET */
        /* 2 */ NOT_DEFINED_CONNECTION_DOMAIN_TYPE
    };

    /* Defined by the Transport Services configuration file */
    enum FACE_SOCKET_TYPE {
        /* 0 */ STREAM,     /* SOCK_STREAM */
        /* 1 */ DGRAM,      /* SOCK_DGRAM */
        /* 2 */ SEQPACKET,  /* SOCK_SEQPACKET */
        /* 3 */ NOT_DEFINED_SOCKET_TYPE
    };
```

```
    /* Defined by the Transport Services configuration file */
    enum FACE_RECEIVE_FLAG_TYPE {
        /* 0 */ PEEK,                   /* SOCK_STREAM */
        /* 1 */ OOB_RECEIVE_FLAG_TYPE,  /* SOCK_DGRAM */
        /* 2 */ WAITALL,                /* SOCK_SEQPACKET */
        /* 3 */ NOT_DEFINED_RECEIVE_FLAG_TYPE
    };

    /* Defined by the Transport Services configuration file */
    enum FACE_SEND_FLAG_TYPE {
        /* 0 */ EOR,                 /* SOCK_STREAM*/
        /* 1 */ OOB_SEND_FLAG_TYPE,  /* SOCK_DGRAM*/
        /* 2 */ NOSIGNAL,            /* SOCK_SEQPACKET*/
        /* 3 */ NOT_DEFINED_SEND_FLAG_TYPE
    };

    enum FACE_VALIDITY_TYPE {
        /* 0 */ INVALID,        /* Also “empty” and “down” */
        /* 0 */ VALID           /* Also “occupied” and “UP” */
    };

    /* Addressed in Validity_Type */
    /* typedef enum { EMPTY = 0, OCCUPIED = 1} FACE_EMPTY_INDICATOR_TYPE; */
    /* typedef enum { DOWN = 0, UP = 1} FACE_EVENT_STATE_TYPE; */

    /* Support publish/subscribe and request/response */
    enum FACE_MESSAGING_PATTERN_TYPE {
        /* 0 */ PUB_SUB,
        /* 1 */ CLIENT,
        /* 2 */ SERVER
    };

    /* Connection provides reliable or non-reliable transport */
    enum RELIABILITY_TYPE {
        /* 0 */ RELIABLE,
        /* 1 */ NON_RELIABLE
    };

    /* Connection provides queuing or sampling behavior */
    enum READ_WRITE_BEHAVIOR_TYPE {
        /* 0 */ QUEUING,
        /* 1 */ SAMPLING
    };

    typedef long FACE_WAITING_RANGE_TYPE;
    typedef string FACE_FILTER_SPECIFICATION_TYPE;
    typedef long FACE_FILTER_VALUE_TYPE[64];

    /* Elements defined by the Transport Services configuration file */
    typedef struct connection_status_struct {
        FACE_MESSAGE_RANGE_TYPE MESSAGE;
        FACE_MESSAGE_RANGE_TYPE MAX_MESSAGE;
        FACE_MESSAGE_SIZE_TYPE MAX_MESSAGE_SIZE;
        FACE_CONNECTION_DIRECTION_TYPE CONNECTION_DIRECTION;
        FACE_WAITING_RANGE_TYPE WAITING_PROCESSES_OR_MESSAGES;
        FACE_SYSTEM_TIME_TYPE REFRESH_PERIOD;  /* Maximum age of a valid
message */
        FACE_VALIDITY_TYPE LAST_MSG_VALIDITY;
    } FACE_CONNECTION_STATUS_TYPE;
```

```
    /* Defined by the Transport Services configuration file */
    /* Struct for TSS to store connection configuration Data */
    typedef struct connection_attr_struct
    {
        FACE_CONNECTION_NAME_TYPE connection_name;
        FACE_CONNECTION_TYPE connection_type;
        FACE_CONNECTION_DOMAIN_TYPE connection_domain;
        FACE_SOCKET_TYPE socket_type;
        FACE_MESSAGE_SIZE_TYPE message_size;
        FACE_MESSAGE_RANGE_TYPE message_range;
        FACE_CONNECTION_DIRECTION_TYPE connection_direction;
        FACE_MESSAGING_PATTERN_TYPE messaging_pattern;
        FACE_RELIABILITY_TYPE reliability;
        FACE_READ_WRITE_BEHAVIOR_TYPE read_write_behavior;
        FACE_SYSTEM_TIME_TYPE refresh_period;
        FACE_QUEUING_DISCIPLINE_TYPE connection_discipline;
    } FACE_CONNECTION_ATTR_STRUCT;
```

C.4 C++ Data Types

```
namespace FACE
{
    typedef char CONNECTION_NAME_TYPE [64];
    typedef long MESSAGE_SIZE_TYPE;

/* All Transport Services configuration file connections are defined by the
CONNECTION_NAME key */
    /* Defined by the Transport Services configuration file */
    enum CONNECTION_DIRECTION_TYPE {
        /* 0 */ SOURCE,      /*maps to O_WRONLY (ARINC 653 and POSIX)*/
        /* 1 */ DESTINATION, /*maps to O_RDONLY (ARINC 653 and POSIX)*/
        /* 2 */ BI_DIRECTIONAL,  /*maps to O_RDWR (POSIX ONLY)*/
        /* 3 */ ONE_WAY_REQUEST_SOURCE,
        /* 4 */ ONE_WAY_REQUEST_DESTINATION,
        /* 5 */ TWO_WAY_REQUEST_SYNCHRONOUS_SOURCE,
        /* 6 */ TWO_WAY_REQUEST_SYNCHRONOUS_DESTINATION,
        /* 7 */ TWO_WAY_REQUEST_REPLY_ASYNCHRONOUS_SOURCE,
        /* 8 */ TWO_WAY_REQUEST_REPLY_ASYNCHRONOUS_DESTINATION,
        /* 9 */ NOT_DEFINED_CONNECTION_DIRECTION_TYPE
    };

    typedef long long SYSTEM_TIME_TYPE; /* 64-bit signed integer with a 1
nanosecond LSB */
    typedef long CONNECTION_ID_TYPE; /* two connection IDs for socketpair */
    typedef long long SEQUENCE_ID_TYPE; /* used to tie together
request/response messages */

/* Defined by the Transport Services configuration file */
    enum CONNECTION_TYPE {
        /* 0 */ SAMPLING_PORT,   /* ARINC 653 sampling port */
        /* 1 */ QUEUING_PORT,    /* ARINC 653 queuing port */
        /* 2 */ SOCKET,          /* POSIX socket */
        /* 3 */ MQ,              /* POSIX queue */
        /* 4*/ SHM,              /* POSIX shared memory */
        /* 5 */ CORBA,           /* CORBA */
        /* 6 */ DDS              /* Data Distribution Services */
    };
```

```
    enum RETURN_CODE_TYPE {
        /* 0 */ NO_ERROR,  /* request valid and operation performed */
        /* 1 */ NO_ACTION, /* status of system unaffected by request */
        /* 2 */ NOT_AVAILABLE, /* resource required by request unavailable */
        /* 3 */ INVALID_PARAM, /* invalid parameter specified in request */
        /* 4 */ INVALID_CONFIG, /* parameter incompatible with configuration */
        /* 5 */ INVALID_MODE, /* request incompatible with current mode */
        /* 6 */ TIMED_OUT, /* time-out tied up with request has expired */
        /* 7 */ MESSAGE_STALE, /*current timestamp exceeds configured limits*/
        /* 8 */ CONNECTION_IN_PROGRESS,   /* asynchronous connection in
progress */
        /* 9 */ CONNECTION_CLOSED
    }; /* POSIX ERRONO mapped to these codes by transport services */

    typedef long MESSAGE_RANGE_TYPE;
    typedef boolean WAITSET_TYPE[32];

    typedef struct FACE_SPECIFIED_STRUCT {
/* The below value is a placeholder for the actual structure definition of
Pub/Sub data. When used in practice, "FACE_SPECIFIED_STRUCT" will be replaced
by either the actual name of the data structure or “void *” as dictated by the
connection’s configuration. */
        sequence <octet> value;
    } FACE_SPECIFIED_TYPE;

    class FACE_SPECIFIED_INTERFACE {
/* The method below is a placeholder for the actual interrface methods. For the
actual interface, "FACE_SPECIFIED_INTERFACE" will be replaced by the actual
name of the service. */
        void placeholder();
    };

    class Read_Callback {
        void send_event(FACE_SPECIFIED_TYPE message,
            MESSAGE_SIZE_TYPE message_size,
            SEQUENCE_ID_TYPE sequence_id,
            WAITSET_TYPE waitset);
    };
    enum QUEUING_DISCIPLINE_TYPE {
        /* 0 */ FIFO,
        /* 1 */ PRIORITY,
        /* 2 */ NOT_DEFINED_QUEUING_DISCIPLINE_TYPE
    };

    enum VALIDITY_TYPE {
        /* 0 */ INVALID,   /* Also “empty” and “down” */
        /* 0 */ VALID      /* Also “occupied” and “UP” */
    };

/* Addressed in Validity_Type */
/* typedef enum {EMPTY = 0, OCCUPIED = 1} FACE_EMPTY_INDICATOR_TYPE; */
/* typedef enum { DOWN = 0, UP = 1} FACE_EVENT_STATE_TYPE; */

    /* Support publish/subscribe and request/response */
    enum MESSAGING_PATTERN_TYPE {
        /* 0 */ PUB_SUB,
        /* 1 */ CLIENT,
        /* 2 */ SERVER
    };
```

```
    /* Connection provides reliable or non-reliable transport */
    enum RELIABILITY_TYPE {
        /* 0 */ RELIABLE,
        /* 1 */ NON_RELIABLE
    };

    /* Connection provides queuing or sampling behavior */
    enum READ_WRITE_BEHAVIOR_TYPE {
        /* 0 */ QUEUING,
        /* 1 */ SAMPLING
    };

    typedef long WAITING_RANGE_TYPE;
    typedef string FILTER_SPECIFICATION_TYPE;
    typedef long FILTER_VALUE_TYPE[64];

    /* Elements defined by the Transport Services configuration file */
    typedef struct connection_status_struct {
        MESSAGE_RANGE_TYPE MESSAGE;
        MESSAGE_RANGE_TYPE MAX_MESSAGE;
        MESSAGE_SIZE_TYPE MAX_MESSAGE_SIZE;
        CONNECTION_DIRECTION_TYPE CONNECTION_DIRECTION;
        WAITING_RANGE_TYPE WAITING_PROCESSES_OR_MESSAGES;
        SYSTEM_TIME_TYPE REFRESH_PERIOD; /* Maximum age of a valid message */
        VALIDITY_TYPE LAST_MSG_VALIDITY;
    } CONNECTION_STATUS_TYPE;

    typedef struct connection_attr_struct
    {
        CONNECTION_NAME_TYPE connection_name;
        CONNECTION_TYPE connection_type;
        CONNECTION_DOMAIN_TYPE connection_domain;
        SOCKET_TYPE socket_type;
        MESSAGE_SIZE_TYPE message_size;
        MESSAGE_RANGE_TYPE message_range;
        CONNECTION_DIRECTION_TYPE connection_direction;
        MESSAGING_PATTERN_TYPE messaging_pattern;
        RELIABILITY_TYPE reliability;
        READ_WRITE_BEHAVIOR_TYPE read_write_behavior;
        SYSTEM_TIME_TYPE refresh_period;
        QUEUING_DISCIPLINE_TYPE connection_discipline;

    } CONNECTION_ATTR_STRUCT;

} // end namespace FACE
```

C.5 Create Connection

The Transport Services Segment provides an interface to create a connection. This interface allows the use of DDS, CORBA, ARINC 653, and POSIX connections. The Transport Services Segment implements the underlying DDS, CORBA, ARINC 653, and POSIX function calls, and passes the parameters and data of the application segment through these standardized function calls. The parameters for the individual connections are determined through the Transport Services Segment configuration capability. The Transport Services Segment manages the system-level connection identifiers based on DDS, CORBA, ARINC 653, and POSIX connections for routing purposes. Certain communication APIs may be called natively without using the Transport Interface. For restrictions see Section C.11.

Note If the Transport Services Segment is creating a POSIX socket, then *bind*(), *connect*(), or *listen*() will be called with the appropriate parameters immediately after *socket*(). *create_connection*() will be non-blocking regardless of the underlying transport mechanism.

```
/* C style declaration */
void FACE_TS_create_connection(
    /* in */ FACE_CONNECTION_NAME_TYPE connection_name,
    /* in */ FACE_MESSAGING_PATTERN_TYPE pattern,
    /* in */ FACE_FILTER_SPECIFICATION_TYPE filter_spec,
    /* out */ FACE_CONNECTION_ID_TYPE *connection_id,
    /* out */ FACE_CONNECTION_DIRECTION_TYPE *connection_direction,
    /* out */ FACE_MESSAGE_SIZE_TYPE *message_size,
    /* inout */ void *server, /* only used for attaching to a server */
    /* out */ FACE_RETURN_CODE_TYPE *return_code);

/* C style declaration */
void FACE_TS_SPECIFIED_INTERFACE_create_connection(
    /* in */ FACE_CONNECTION_NAME_TYPE connection_name,
    /* in */ FACE_MESSAGING_PATTERN_TYPE pattern,
    /* in */ FACE_FILTER_SPECIFICATION_TYPE filter_spec,
    /* out */ FACE_CONNECTION_ID_TYPE *connection_id,
    /* out */ FACE_CONNECTION_DIRECTION_TYPE *connection_direction,
    /* out */ FACE_MESSAGE_SIZE_TYPE *message_size,
    /* inout */ FACE_SPECIFIED_INTERFACE *server, /* only used for attaching to
a server */
    /* out */ FACE_RETURN_CODE_TYPE *return_code);

/* C++ style declaration */
void FACE::TS::create_connection(
    /* in */ FACE::CONNECTION_NAME_TYPE connection_name,
    /* in */ FACE::MESSAGING_PATTERN_TYPE pattern,
    /* in */ FACE::FILTER_SPECIFICATION_TYPE filter_spec,
    /* out */ FACE::CONNECTION_ID_TYPE &connection_id,
    /* out */ FACE::CONNECTION_DIRECTION_TYPE &connection_direction,
    /* out */ FACE::MESSAGE_SIZE_TYPE &message_size,
    /* inout */ FACE::SPECIFIED_INTERFACE &server, /* only used for attaching
to a server */
    /* out */ FACE::RETURN_CODE_TYPE &return_code);
```

ARINC 653 Sampling Port

When a sampling port is instantiated through *create_connection*() there are certain parameters that must have input and others associated with POSIX connections that will be NULL due to no operation for this particular transport mechanism. *See Note 9.*

```
/* NOTE:  THESE ARE THE CALLS MADE BY TRANSPORT SERVICES */
void CREATE_SAMPLING_PORT (
/*in */ SAMPLING_PORT_NAME_TYPE SAMPLING_PORT_NAME, /* connection_name */
/*in */ MESSAGE_SIZE_TYPE MAX_MESSAGE_SIZE,  /* message_size */
/*in */ PORT_DIRECTION_TYPE PORT_DIRECTION,  /* connection_direction */
/*in */ SYSTEM_TIME_TYPE REFRESH_PERIOD,     /* refresh_period */
/*out*/ SAMPLING_PORT_ID_TYPE *SAMPLING_PORT_ID,  /* connection_id */
/*out*/ RETURN_CODE_TYPE *RETURN_CODE);      /* return_code */
```

ARINC 653 Queuing Port

When a queuing port is instantiated through *create_connection*() there are certain parameters that must have input and others associated with POSIX connections that will be NULL due to no operation for this particular transport mechanism. *See Note 9.*

```
/* NOTE:  THESE ARE THE CALLS MADE BY TRANSPORT SERVICES */
void CREATE_QUEUING_PORT (
/*in */ QUEUING_PORT_NAME_TYPE QUEUING_PORT_NAME, /* connection_name */
/*in */ MESSAGE_SIZE_TYPE MAX_MESSAGE_SIZE,  /* message_size */
/*in */ MESSAGE_RANGE_TYPE MAX_NB_MESSAGE,   /* message_range */
/*in */ PORT_DIRECTION_TYPE PORT_DIRECTION,  /* connection_direction */
/*in */ QUEUING_DISCIPLINE_TYPE QUEUING_DISCIPLINE, /* connection_discipline */
/*out*/ QUEUING_PORT_ID_TYPE *QUEUING_PORT_ID, /* connection_id */
/*out*/ RETURN_CODE_TYPE *RETURN_CODE);      /* return_code */
```

POSIX Socket

When a socket is instantiated through *create_connection*() there are certain parameters that must have input and others associated with ARINC 653 connections that will be NULL due to no operation for this particular transport mechanism. *See Note 9.*

The following POSIX functions will be called in this order by Transport Services when the POSIX socket connection is instantiated by *create_connection*().

```
/* NOTE:  THESE ARE THE CALLS MADE BY TRANSPORT SERVICES */
1.  int socket(int domain,     /* connection_domain */
               int type,       /* socket_type */
               int protocol);  /* chosen by the TSS */
2.  int bind(int socket,                    /* connection_id */
             const struct sockaddr *address, /* provided by TSS */
             socklen_t address_len);         /* provided by TSS */
             OR
             int connect( int socket,        /* connection_id */
             const struct sockaddr *address, /* provided by TSS */
             socklen_t address_len);         /* provided by TSS */
3.  int listen(int socket,       /* connection_id */
               int backlog);     /* provided by TSS */
4.  int accept(int socket,
               struct sockaddr *restrict address,
               socklen_t *restrict address_len);
/* The returned “int” will map to the connection_id */
```

POSIX Queue

When a queue is instantiated through *create_connection*() there are certain parameters that must have input and others associated with ARINC connections that will be NULL due to no operation for this particular transport mechanism. The Transport Services Segment will complete the parameter list of the *mq_open*() function call.

```
/* NOTE: THESE ARE THE CALLS MADE BY TRANSPORT SERVICES */
mqd_t mq_open(const char *name,   /* connection_name */
              int oflag);         /* connection_direction */
/* mqd_t is mapped to connection_id through TSS */
```

POSIX Shared Memory

When shared memory is instantiated through *create_connection*() there are certain parameters that must have input and others associated with ARINC connections that will be NULL due to no operation for this particular transport mechanism. The Transport Services Segment will complete the parameter list of the *shm_open*() function call.

The following POSIX functions will be called in this order by Transport Services when the POSIX shared memory connection is instantiated by *create_connection*().

```
/* NOTE:  THESE ARE THE CALLS MADE BY TRANSPORT SERVICES */
1.  int shm_open(const char *name,  /* connection_name */
                 int oflag,         /* chosen by TSS */
                 mode_t mode);      /* chosen by TSS */
/* The returned “int” will map to the connection_id */
2.  void *mmap(void *addr,          /* connection_id */
               size_t len,          /* message_size */
               int prot,            /* chosen by TSS */
               int flags,           /* chosen by TSS */
               int fildes,          /* chosen by TSS */
               off_t off);          /* chosen by TSS */
```

CORBA

The following CORBA methods are called in this order by Transport Services when a CORBA Object Request Broker (ORB) is instantiated by *create_connection*().

Since CORBA is distributed object middleware, a CORBA object will be created for applications receiving data. Any application sending data will retrieve a reference to the CORBA object to which it wants to send data. The configuration file for an application will determine the unique name(s) for the CORBA object(s) used. *See Note 9*.

```
/* NOTE: THESE ARE THE CALLS MADE BY TRANSPORT SERVICES FOR THE APPLICATION
SENDING DATA */
/* NOTE: These calls assume a C++ mapping from the CORBA IDL, and should be
performed in the order specified below */
/* NOTE: These calls assume an IDL FACE module with a TransportObject
interface. The FACE module name and the TransportObject interface name shall be
verbatim including case – see Note 26 */
1.  CORBA::ORB_ptr CORBA::ORB_init (int arg_count, // count of ORB arguments
    char** arg_vector); // vector of ORB arguments
2.  CORBA::ORB_ptr CORBA::ORB:resolve_initial_references (char * objectId);
    // the objectId shall be set to "NameService" to retrieve the CORBA Naming
Service. See Note 26.
3.  CosNaming::NamingContext_ptr CosNaming::NamingContext::_narrow
    (CORBA::Object_ptr); // Narrow the CORBA object to the Naming Service
4.  bool CORBA::is_nil (CORBA::Object_ptr); // Check that the object reference
was narrowed correctly to a NamingService.
5.  CORBA::ORB_ptr CosNaming::NamingContext:resolve (CosNaming::Name
receiver_name); // Retrieve the object to receive the data
6.  bool CORBA::is_nil (CORBA::Object_ptr); // Check that the Naming Service
was able to resolve the name.
7.  CosNaming::NamingContext_ptr FACE::TransportObject::_narrow
(CORBA::Object_ptr); // Narrow the CORBA object to the one and only Transport
object
8.  bool CORBA::is_nil (CORBA::Object_ptr); // Check that the object reference
was narrowed correctly to a Transport object.
```

```
/* NOTE: THESE ARE THE CALLS MADE BY TRANSPORT SERVICES FOR THE APPLICATION
RECEIVING DATA */
/* NOTE: These calls assume a C++ mapping from the CORBA IDL and should be
performed in the order specified below. */
/* NOTE: These calls also assume a CORBA IDL FACE module with a TransportObject
interface. The “FACE” module name and the “TransportObject” interface name
shall be used verbatim including case. See Note 26. */
1.  CORBA::ORB_ptr CORBA::ORB_init (int arg_count, // count of ORB arguments
    char** arg_vector); // vector of ORB arguments
2.  CORBA::ORB_ptr CORBA::ORB:resolve_initial_references (char * objectId);
    // the objectId shall be set to "RootPOAManager" to retrieve the root
Portable Object Adapter (POA) manager. See Note 26.
3.  PortableServer::POAManager_ptr PortableServer::POAManager::_narrow
(CORBA::Object_ptr); // Narrow the CORBA object to the root POA manager.
4.  bool CORBA::is_nil (CORBA::Object_ptr); // Check that the object reference
was narrowed correctly to the root POA manager.
5.  CORBA::ORB_ptr CORBA::ORB:resolve_initial_references (char * objectId);
    // the objectId shall be set to "NameService" to retrieve the CORBA Naming
Service. See Note 26.
6.  CosNaming::NamingContext_ptr CosNaming::NamingContext::_narrow
(CORBA::Object_ptr); // Narrow the CORBA object to the Naming Service
7.  bool CORBA::is_nil (CORBA::Object_ptr); // Check that the object reference
was narrowed correctly to a NamingService.
8.  TransportObject * new TransportObject (); // Create the object to receive
data whose interface is defined by CORBA IDL.
9.  FACE::TransportObject_ptr FACE::TransportObject:_this ();
    // Register the object with the ORB.
10. CosNaming::NamingContext:rebind (CosNaming::Name receiver_name,
    FACE::TransportObject_ptr);
11. PortableServer::POAManager:activate (); // Activate the POA manager.
12. CORBA::ORB:run (); // Run the ORB to make the TransportObject available.
```

Data Distribution Service

When DDS is instantiated through *create_connection*() there are certain parameters that must have input and others associated with POSIX and ARINC connections that will be NULL. *See Note 9*.

The *connection_id* will be generated by the Transport Services Segment. For the following, the data namespace is “SPACE” and the data type used is “Foo”. Any call involving the QoS of a domain entity can be modified based upon the needs of the connection.

```
1.DDS_DomainParticipantFactory DDS_DomainParticipantFactory_get_instance(void);
2.DDS_DomainParticipant DDS_DomainParticipantFactory_create_participant(
    DDS_DomainParticipantFactory factory, /* Provided by 1 */
    const DomainID_t domainId,            /* Empty variable */
    const DDS_DomainParticipantQos *qos,  /* chosen/configured by TSS */
    const struct DDS_DomainParticipantListener *a_listener, /* chosen by TSS */
    const DDS_StatusMask mask);           /* chosen by TSS */
3.DDS_TypeSupport SPACE_FooTypeSupport__alloc(void);
4.DDS_string SPACE_FooTypeSupport_get_type_name (DDS_TypeSupport type);
5.DDS_ReturnCode_SPACE_FooTypeSupport_register_type (
    DDS_TypeSupport type,                /* Provided by 3 */
    DDS_DomainParticipant participant,   /* Provided by 2 */
    DDS_string type_name);               /* Provided by 4 */
6.DDS_TopicQos* DDS_TopicQos__alloc(void);
7.DDS_ReturnCode_t DDS_DomainParticipant_get_default_topic_qos (
```

```
    DDS_DomainParticipant participant,   /* Provided by 2 */
    DDS_TopicQos* qos);                  /* Provided by 6 */
8.Adjust the QoS as configured/required by modifying the DDS_TopicQos
structure.
9.DDS_Topic DDS_DomainParticipant_create_topic (
    DDS_DomainParticipant participant,   /* Provided by 2 */
    const DDS_char* topic_name,          /* Parsed connection_name */
    const DDS_char* type_name,           /* Provided by 4 */
    const DDS_TopicQos* qos,             /* Provided by 6 */
    const struct DDS_TopicListener* a_listener, /* Chosen by TSS */
    const DDS_StatusMask mask);          /* Chosen by TSS */
10.If connection_direction is:
    a.  SOURCE
    i.  DDS_PublisherQos* DDS_PublisherQos__alloc(void);
    ii. DDS_ReturnCode_t DDS_DomainParticipant_get_default_publisher_qos(
        DDS_DomainParticipant participant,    /* Provided by 2 */
        DDS_PublisherQos* qos);               /* Provided by 10.a.i */
    iii.Adjust partition name in the DDS_PublisherQos structure.
        Partition name provided by the connection_name attribute.
    iv. DDS_Publisher DDS_DomainParticipant_create_publisher (
        DDS_DomainParticipant participant,    /* Provided by 2 */
        const DDS_PublisherQos* qos,          /* Provided by 10.a.i */
        const struct DDS_PublisherListener* a_listener, /* Chosen by TSS */
        const DDS_StatusMask mask);           /* Chosen by TSS */
    v.  DDS_DataWriterQos* DDS_DataWriterQos__alloc(void);
    vi. DDS_ReturnCode_t DDS_Publisher_get_default_datawriter_qos (
        DDS_Publisher publisher,      /* Provided by 10.a.iv */
        DDS_DataWriterQos* qos);      /* Provided by 10.a.v */
    vii.DDS_ReturnCode_t DDS_Publisher_copy_from_topic_qos (
        DDS_Publisher publisher,      /* Provided by 10.a.iv */
        DDS_DataWriterQos* dw_qos,    /* Provided by 10.a.v */
        const DDS_TopicQos* topic_qos);  /* Provided by 6 */
    viii.DDS_DataWriter DDS_Publisher_create_datawriter (
        DDS_Publisher publisher,      /* Provided by 10.a.iv */
        const DDS_Topic a_topic,      /* Provided by 9 */
        const DDS_DataWriterQos* qos, /* Provided by 10.a.v */
        const struct DDS_DataWriterListener* listener, /* Chosen by TSS */
        const DDS_StatusMask mask);   /* Chosen by TSS */
    b.  DESTINATION
    i.  DDS_SubscriberQos* DDS_SubscriberQos__alloc(void);
    ii. DDS_ReturnCode_t DDS_DomainParticipant_get_default_subscriber_qos (
        DDS_DomainParticipant participant, /* Provided by 2 */
        DDS_SubscriberQos* qos);           /* Provided by 10.b.i */
    iii.Adjust partition name in the DDS_SubscriberQos structure.
        Partition name provided by the connection_name attribute.
    iv. DDS_Subscriber_DDS_DomainParticipant_create_subscriber (
        DDS_DomainParticipant participant,     /* Provided by 2 */
        const DDS_SubscriberQos* qos,          /* Provided 10.b.i */
        const struct DDS_SubscriberListener* a_listener, /* Chosen by TSS */
        const DDS_StatusMask mask);            /* Chosen by TSS */
    v.  DDS_DataReaderQos* DDS_DataReaderQos__alloc(void);
    vi. DDS_ReturnCode_t DDS_Subscriber_get_default_datareader_qos (
        DDS_Subscriber subscriber,             /* Provided by 10.b.iv */
        DDS_DataReaderQos* qos);               /* Provided by 10.b.v */
    vii.DDS_ReturnCode_t DDS_Subscriber_copy_from_topic_qos (
        DDS_Subscriber subscriber,             /* Provided by 10.b.iv */
        DDS_DataReaderQos* dw_qos,             /* Provided by 10.b.v */
        const DDS_TopicQos* topic_qos);        /* Provided by 6 */
    viii.DDS_DataReader DDS_Subscriber_create_datareader (
        DDS_Subscriber subscriber,             /* Provided by 10.b.iv */
```

```
const DDS_Topic a_topic,                /* Provided by 6 */
const DDS_DataReaderQos* qos,           /*Provided by 10.b.v*/
const struct DDS_DataReaderListener* listener, /*Chosen by TSS*/
const DDS_StatusMask mask);             /*Chosen by TSS*/
```

C.6 Destroy Connection

1. Transport Services will keep track of the *mqd_t* descriptors and have them searchable by connection name.
2. Transport Services will keep track of the address and length of the shared memory referenced by the connection name.
3. *int mq_unlink*(*const char *name*); will be called after *mq_close*() in Transport Services.
4. *int shm_unlink*(*const char *name*); will be called after *munmap*() in Transport Services.
5. For DDS, Transport Services will keep track of the *DDS_DomainParticipant* and *DDS_Topic* variables for deletion.
   - For SOURCE *connection_direction*, the *DDS_Publisher* and *DDS_DataWriter* will be kept track of for deletion.
   - For DESTINATION *connection_direction*, the *DDS_Subscriber* and *DDS_DataReader* will be kept track of for deletion.

```
/* C style declaration */
void FACE_TS_destroy_connection(
/ in  */ FACE_CONNECTION_ID_TYPE connection_id,
/*out */ FACE_RETURN_CODE_TYPE *return_code );

/* C++ style declaration */
void FACE::TS::destroy_connection(
/* in  */ FACE::CONNECTION_ID_TYPE connection_id,
/* out */ FACE::RETURN_CODE_TYPE &return_code );
```

CORBA

The following CORBA method will be called by Transport Services when CORBA is shutdown by *destroy_connection*().

```
/* NOTE: THIS IS THE CALL MADE BY TRANSPORT SERVICES */
CORBA::ORB:destroy ();  // Destroy the ORB.
```

C.7 Receive Message

Receive message will be used to call the following ARINC 653 function calls – *see Note 9*:

Sampling Port READ_SAMPLING_MESSAGE

Queuing Port RECEIVE_QUEUING_MESSAGE

Receive message will be used to call the following POSIX function calls – *see Note 9*:

Sockets	*recv*(), *recvfrom*(), *recvmsg*()
Message Queue	*mq_receive*(), *mq_timedreceive*()
Shared Memory	None.

The Transport Services Segment shall manage the source information from the *recvfrom*() and *recvmsg*() function calls for routing purposes.

Certain communication APIs may be called natively without using the Transport Interface. For restrictions, see Section C.11.

Receive Message will be used to call the following DDS function calls – *see Note 9*:

- SPACE_FooDataReader_take
- SPACE_FooDataReader_read

The form of this function is as follows:

```
/* C style declaration */
    void FACE_TS_receive_message (
        /* in */ FACE_CONNECTION_ID_TYPE connection_id,
        /* in */ FACE_SYSTEM_TIME_TYPE timeout,
        /* in */ FACE_FILTER_VALUE_TYPE filter_vals,
        /* inout */ FACE_SEQUENCE_ID_TYPE *sequence_id,
        /* out */ void *message,
        /* out */ FACE_MESSAGE_SIZE_TYPE *message_size,
        /* out */ FACE_RETURN_CODE_TYPE *return_code);

/* C style declaration */
    void FACE_TS_SPECIFIED_TYPE_receive_message (
        /* in */ FACE_CONNECTION_ID_TYPE connection_id,
        /* in */ FACE_SYSTEM_TIME_TYPE timeout,
        /* in */ FACE_FILTER_VALUE_TYPE filter_vals,
        /* inout */ FACE_SEQUENCE_ID_TYPE *sequence_id,
        /* out */ FACE_SPECIFIED_TYPE *message,
        /* out */ FACE_MESSAGE_SIZE_TYPE *message_size,
        /* out */ FACE_RETURN_CODE_TYPE *return_code);

/* C++ style declaration */
    void FACE::TS::receive_message (
        /* in */ FACE::CONNECTION_ID_TYPE connection_id,
        /* in */ FACE::SYSTEM_TIME_TYPE timeout,
        /* in */ FACE::FILTER_VALUE_TYPE filter_vals,
        /* inout */ FACE::SEQUENCE_ID_TYPE &sequence_id,
        /* out */ FACE::SPECIFIED_TYPE &message,
        /* out */ FACE::MESSAGE_SIZE_TYPE &message_size,
        /* out */ FACE::RETURN_CODE_TYPE &return_code);
```

When the connection is set up as a CLIENT, the *sequence_id* is an “in” parameter. In this case, *receive_message*() will block until TSS receives a response from the server matching that *sequence_id*.

When the connection is set up as a SERVER, the *sequence_id* is an “out” parameter. In this case, *receive_message*() will return the *sequence_id* of the incoming message from the client. The

server application must then use this *sequence_id* when sending a response using *send_message*().

CORBA

The following CORBA method will be called by Transport Services when receiving data using CORBA. *See Note 9.*

```
/* NOTE: THIS IS THE CALL MADE BY TRANSPORT SERVICES */
/* NOTE: These calls use a CORBA IDL FACE module with a TransportObject
interface. The TransportObject interface has two methods: get_data() that
returns an array of bytes and store_data() which stores an array of bytes. The
FACE module name, the TransportObject interface name, and the method names of
get_data and store_data shall be used verbatim including case. */
FACE::TransportObject:get_data (); // Get the data stored in the CORBA object.
See Note 26.
```

C.8 Send Message

Send message will be used to call the following ARINC 653 function calls – *see Note 9*:

Sampling Port WRITE_SAMPLING_MESSAGE

Queuing Port SEND_QUEUING_MESSAGE

Send message will be used to call the following POSIX function calls – *see Note 9*:

Sockets *send*(), *sendmsg*(), *sendto*()

Message Queue *mq_receive*(), *mq_timedreceive*()

Shared Memory None.

The Transport Services Segment shall manage the destination information from the *sendto*() and *sendmsg*() function calls for routing purposes.

Certain communication APIs may be called natively without using the Transport Interface. For restrictions see Section C.11.

Send message will be used to call the following DDS function call – *see Note 9*:

- *SPACE_FooDataWriter_write*()

```
/* C style declaration */
void FACE_TS_send_message (
    /* in */ FACE_CONNECTION_ID_TYPE connection_id,
    /* in */ void *message,
    /* in */ FACE_MESSAGE_SIZE_TYPE message_size, /* if 0, TSS will use the
size of the data type – see Note 9 */
    /* in */ FACE_SYSTEM_TIME_TYPE timeout,
    /* inout */ FACE_SEQUENCE_ID_TYPE *sequence_id,
    /* out */ FACE_RETURN_CODE_TYPE *return_code);
```

The form of *send_message*() is as follows for the type FACE_SPECIFIED_TYPE:

```
/* C style declaration */
void FACE_TS_SPECIFIED_TYPE_send_message (
    /* in */ FACE_CONNECTION_ID_TYPE connection_id,
    /* in */ FACE_SPECIFIED_TYPE *message,
    /* in */ FACE_MESSAGE_SIZE_TYPE message_size, /* if 0, TSS will use the
size of the data type – see Note 9 */
    /* in */ FACE_SYSTEM_TIME_TYPE timeout,
    /* inout */ FACE_SEQUENCE_ID_TYPE *sequence_id,
    /* out */ FACE_RETURN_CODE_TYPE *return_code);

/* C++ style declaration */
void FACE::TS::send_message (
    /* in */ FACE::CONNECTION_ID_TYPE connection_id,
    /* in */ const FACE::SPECIFIED_TYPE &message,
    /* in */ FACE::MESSAGE_SIZE_TYPE message_size, /* if 0, TSS will use the
size of the data type */
    /* in */ FACE::SYSTEM_TIME_TYPE timeout,
    /* inout */ FACE::SEQUENCE_ID_TYPE &sequence_id,
    /* out */ FACE::RETURN_CODE_TYPE &return_code);
```

When the connection is set up as a CLIENT, the *sequence_id* is an “out” parameter. In this case, TSS will generate a unique *sequence_id* and return it. Subsequently, the client application will pass this to *receive_message* when waiting on the response from the server.

When the connection is set up as a SERVER, the *sequence_id* is an “in” parameter. In this case, *send_message* will transmit the *sequence_id* to the client. This identifies the request to which the server is responding.

CORBA

The following CORBA method will be called by the Transport Services Segment when sending data using CORBA. *See Note 9.*

```
/* NOTE: THIS IS THE CALL MADE BY TRANSPORT SERVICES */
/* NOTE: These calls use a CORBA IDL FACE module with a TransportObject
interface. The TransportObject interface has two methods: get_data() that
returns an array of bytes and store_data() which stores an array of bytes. The
FACE module name, the TransportObject interface name, and the method names of
get_data and store_data shall be used verbatim including case */
FACE::TransportObject:store_data(); // Store the data in the CORBA object. See
Note 26.
```

C.9 Register Read Callback

The purpose of *register_read_callback*() is to provide a mechanism to read data without polling. This will be used for publish/subscribe transportation mechanisms. Certain communication APIs may be called natively without using the Transport Interface. For restrictions see Section C.11.

Register read callback will be used to address periodic ARINC 653 function calls without having to poll for data:

Queuing Port RECEIVE_QUEUING_MESSAGE

The *register_read_callback*() call will be used to address periodic POSIX function calls without having to poll for data:

Sockets	*recv*(), *recvfrom*(), *recvmsg*()
Message Queue	*mq_receive*(), *mq_timedreceive*()
Shared Memory	None.

The Transport Services Segment shall manage the source information from the *recvfrom*() and *recvmsg*() function calls for routing purposes.

The form of this function is as follows:

```
void FACE_TS_register_read_callback (
    /* in */  FACE_CONNECTION_ID_TYPE connection_id,
    /* in */  FACE_WAITSET_TYPE waitset,
    /* in */  FACE_Read_Callback data_callback,
    /* in */  FACE_MESSAGE_SIZE_TYPE max_message_size,
    /* out */ FACE_RETURN_CODE_TYPE *return_code);

void FACE_TS_SPECIFIED_TYPE_register_read_callback (
    /* in */  FACE_CONNECTION_ID_TYPE connection_id,
    /* in */  FACE_WAITSET_TYPE waitset,
    /* in */  FACE_Read_Callback data_callback,
    /* in */  FACE_MESSAGE_SIZE_TYPE max_message_size,
    /* out */ FACE_RETURN_CODE_TYPE *return_code);

/* C++ style declaration */
void FACE::TS::register_read_callback (
    /* in */  FACE::CONNECTION_ID_TYPE connection_id,
    /* in */  FACE::WAITSET_TYPE waitset,
    /* in */  FACE::Read_Callback data_callback,
    /* in */  FACE::MESSAGE_SIZE_TYPE max_message_size,
    /* out */ FACE::RETURN_CODE_TYPE &return_code);
```

C.10 Get Connection Parameters

The *Get_Connection_Parameters*() call will be used to call the following ARINC 653 function calls – *see Note 9*:

Sampling Port	GET_SAMPLING_PORT_ID GET_SAMPLING_PORT_STATUS
Queuing Port	GET_QUEUING_PORT_ID GET_QUEUING_PORT_STATUS

POSIX calls:

Sockets	*getsockname*()
Message Queue	*mq_getattr*()
Shared Memory	None.

Certain communication APIs may be called natively without using the Transport Interface. For restrictions see Section C.11.

DDS does not need to call anything to determine the status of the connection. The most recent *DDS_ReturnCode_t* or the validity of DataReader/Writer handles can be used to determine the connection status.

```
/* C style declaration */
void FACE_TS_get_connection_parameters (
    /* inout */ FACE_CONNECTION_NAME_TYPE *connection_name,
    /* inout */ FACE_CONNECTION_ID_TYPE *connection_id,
    /* out */ FACE_CONNECTION_STATUS_TYPE *connection_status,
    /* out */ FACE_RETURN_CODE_TYPE *return_code);

/* C++ style declaration */
void FACE::TS::get_connection_parameters (
    /* inout */ FACE::CONNECTION_NAME_TYPE &connection_name,
    /* inout */ FACE::CONNECTION_ID_TYPE &connection_id,
    /* out */ FACE::CONNECTION_STATUS_TYPE &connection_status,
    /* out */ FACE::RETURN_CODE_TYPE &return_code);
```

C.11 Function Calls Restricted to the Transport Services Segment

Inter-Process Communication API calls must be used internally to the Transport Interface and I/O Interface libraries unless within a single FACE Unit of Portability (UoP). The following APIs must not be used in the Portable Component Segment or to the Platform-Specific Services Segment that have been developed without the use of a separate run-time. These include POSIX and ARINC 653 data transport-specific API calls and all calls to Transport mechanisms (e.g., CORBA and DDS). There is an exception for programming language run-time environments (e.g., Java EE 6, Ada run-times) that use these APIs natively. COTS libraries that implement networking RFCs will be handled in one of the following ways:

- The COTS networking libraries must use the networking APIs directly if the Portable Components or Platform-Specific Services Segment uses the Transport Services Segment and the Transport Interface to abstract the COTS library from the FACE component (left half of the diagram below).
- A COTS networking library must use the transport services interface if the library is called directly by the Portable Components or Platform-Specific Services. The COTS library shall be delivered with the Application as a UoP (right half of the diagram below).

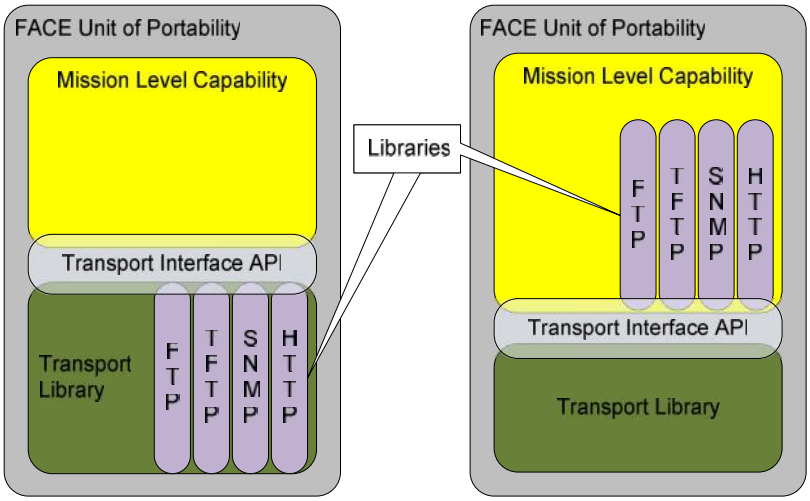


Figure 39: Transport Networking API Configuration Options

Transport and I/O Interface Utilized POSIX APIs

- *accept*()
- *bind*()
- *connect*()
- *endhostent*()
- *endnetent*()
- *endprotoent*()
- *endservent*()
- *freeaddrinfo*()
- *gai_strerror*()
- *getaddrinfo*()
- *gethostbyaddr*()
- *gethostbyname*()
- *gethostent*()
- *gethostname*()
- *getnameinfo*()
- *getnetbyaddr*()
- *getnetbyname*()

- *getnetent*()
- *getpeername*()
- *getprotobyname*()
- *getprotobynumber*()
- *getprotoent*()
- *getservbyname*()
- *getservbyport*()
- *getservent*()
- *getsockname*()
- *getsockopt*()
- *h_errno*
- *if_freenameindex*()
- *if_indextoname*()
- *if_nameindex*()
- *if_nametoindex*()
- *inet_addr*()
- *inet_ntoa*()
- *inet_ntop*()
- *inet_pton*()
- *listen*()
- *recv*()
- *recvfrom*()
- *recvmsg*()
- *send*()
- *sendmsg*()
- *sendto*()
- *sethostent*()
- *setnetent*()
- *setprotoent*()

- *setservent*()
- *setsockopt*()
- *shutdown*()
- *socket*()
- *sockatmark*()
- *mq_close*()
- *mq_getattr*()
- *mq_notify*()
- *mq_open*()
- *mq_receive*()
- *mq_send*()
- *mq_setattr*()
- *mq_unlink*()
- *shm_open*()
- *shm_unlink*()

Transport and I/O Interface Utilized ARINC 653 APIs

- CREATE_SAMPLING_PORT
- CREATE_QUEUING_PORT
- READ_SAMPLING_PORT
- RECEIVE_QUEUING_MESSAGE
- WRITE_SAMPLING_MESSAGE
- SEND_QUEUING_MESAGE
- GET_SAMPLING_PORT_ID
- GET_SAMPLING_PORT_STATUS
- GET_QUEUING_PORT_ID
- GET_QUEUING_PORT_STATUS

Transport APIs within a FACE Unit of Portability

Intra-process communications mechanisms (e.g., semaphores, signals, events, blackboard, buffers, message queues) shall be available natively to software components of the Portable Component Segment or the Platform-Specific Services Segment for communications within the FACE UoP. Software components must use the communications mechanisms supported by the Transport Services Segment if communicating outside the FACE UoP.

Transformation Service

The Transport Services Segment will provide a Transformation Function that will accept two message formats (the source message format and the destination format) and perform both data marshaling and linear conversions from the source message to the destination message format. This function should only be called from within the TSS library on the receiving side of the connection by *receive_data*() or the receive callback. The transformation service shall be built as a library that can be modified for different transformations without modifying the re-usable Transport Services Segment library.

An example of the transformation function interface is as follows:

```
error transform_data (
    FACE_MESSAGE_TYPE source,       //Source Message
    FACE_MESSAGE_TYPE &destination  //Destination Message
);
```

C.12 Transport Services Segment Configuration File

The following information describes the parameters that make up the Transport Services Segment Configuration file:

Connection Name	Name of the connection referenced.
Connection Type	Type of connection; i.e., sampling, queuing, socket, etc.
Connection Direction	Direction of the connection.
Connection Domain	POSIX Socket Domain.
Socket Type	POSIX Socket Type.
Max Message Size	Maximum size of the data allowed on the connection.
Message Range	Number of messages allotted for in the connection.
Messages Associated	Array of message GUIDs associated with the connection.
Data Transform Required	Boolean indicating that a data transform is required.
Refresh Period	Maximum allowable age of valid message.
Reliability	Reliable or Non-Reliable.
ReadWriteBehavior	Buffering or Sampling.
Queue Discipline	Priority or FIFO.
Current Value	Value of a semaphore.
Max Value	Max value of a semaphore.
Receive Flag	Receive Flag for POSIX Sockets.
Send Flag	Send Flag for POSIX Sockets.
Source Address	Source IP address.
Destination Address	Destination IP address.
Source Port	Source Port.
Destination Port	Destination Port

D Internet Standards

D.1 Network Protocol Requirements Summary

The following table summarizes the network standards required for FACE conformance of network host or router devices operating in an Internet environment. As a guide a (✓) symbol in the IPv4 and/or IPv6 columns indicates the applicability of a specific protocol to an IPv4 or IPv6 network. Internet Standards are described in Section 3.14.

Support is designated as mandatory or optional using the (●) or (O) symbol in the HOST and ROUTER columns in the table. Optional capabilities are grouped into packages in the Config Pkg column. Supporting an optional capability requires the inclusion of all associated standards identified in the Config Pkg. Within these packages requirements may only be applicable to HOST or ROUTER implementations. For instance, including support for the Domain Name Service (DNS) in an IPv4 network requires support for RFC 1034, 1035, and 2136. Including support for the Domain Name Service (DNS) in an IPv6 network requires support for RFC 1034, 1035, 1886, and 2136.[1]

Finally, the Source column indicates the original source of the DoD mandatory requirement with (J) representing the DoD Joint Technical Architecture, (D) representing the DoD IPv6 Standard Profiles for IPv6-Capable Products, and N representing the NIST SP500-267 Standard.

Table 8: Internetwork Standards Summary

Standard	Description	Host	Router	IPv4	IPv6	Config Pkg	Source
Core Network Infrastructure – All Internet nodes							
RFC 1122	Requirements for Internet Hosts – Communication Layers	●	●	✓		CORE	J
RFC 0919	Broadcasting Internet Datagrams	●	●	✓		CORE	J
RFC 0922	Broadcasting Internet Datagrams in the Presence of Subnets	●	●	✓		CORE	J
RFC 0950	Internet Standard Subnetting Procedure	●	●	✓		CORE	J
RFC 0792	Internet Control Message Protocol (ICMP)	●	●	✓		CORE	J
RFC 0791	Internet Protocol (IP)	●	●	✓		CORE	J
RFC 2236	Internet Group Management Protocol, Version 2	●	●	✓		CORE	J

[1] All IETF RFCs are available for reference at www.ietf.org.

Standard	Description	Host	Router	IPv4	IPv6	Config Pkg	Source
RFC 0768	User Datagram Protocol	●	●	✓	✓	CORE	J
RFC 0793	Transmission Control Protocol	●	●	✓	✓	CORE	J
RFC 2581	TCP Congestion Control	●		✓	✓	CORE	J
RFC 2460	IPv6 Specification	●	●		✓	CORE	J,D,N
RFC 2373	IP Version 6 Addressing Architecture	●	●		✓	CORE	D,N
RFC 4007	IPv6 Scoped Address Architecture	●	●		✓	CORE	D,N
RFC 3484	Default Address Selection for IPv6	●	●		✓	CORE	D,N
RFC 2463	Internet Control Message Protocol (ICMPv6)	●	●		✓	CORE	J,D,N
RFC 1981	Path MTU Discovery for IPv6	●	●		✓	CORE	D,N
RFC 2710	Multicast Listener Discovery for the IPv6	●	●		✓	CORE	D,N
RFC 2461	Neighbor Discovery for IP Version 6 (IPv6)	●	●		✓	CORE	J,D,N
RFC 2462	IPv6 Stateless Address Autoconfiguration	●	●		✓	CORE	J,D,N
RFC 1812	Requirements for IP Version 4 Routers		●	✓		CORE	J
RFC 2474	Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers		●		✓	CORE	J,D,N
RFC 2328	OSPF Version 2		●	✓		CORE	J
RFC 2740	OSPF for IPv6		●		✓	CORE	J,D,N
RFC 1771	A Border Gateway Protocol 4 (BGP-4)		●	✓	✓	CORE	J,D,N
RFC 1772	Application of the Border Gateway Protocol in the Internet		●	✓	✓	CORE	J,D,N
RFC 2283	Multiprotocol Extensions for BGP-4		●		✓	CORE	J,D,N
IPv4/IPv6 Transition Option – If operating in an integrated IPv4/IPv6 network							
RFC 4213	Transition Mechanisms for IPv6 Hosts and Routers	O	O		✓	DUAL-STACK	J,D,N
IP Security Option – If operating in a system requiring IPsec support							
RFC 2401	Security Architecture for the Internet Protocol	O	O	✓	✓	IPsec-v2	J,D,N

Standard	Description	Host	Router	IPv4	IPv6	Config Pkg	Source
RFC 2402	IP Authentication Header	O	O	✓	✓	IPsec-v2	J,D
RFC 2404	The Use of HMAC-SHA-1-96 within ESP and AH	O	O	✓	✓	IPsec-v2	J,N
RFC 2406	IP Encapsulating Security Payload (ESP)	O	O	✓	✓	IPsec-v2	J,D,N
RFC 4807	IPsec Security Policy Database Configuration MIB		O		✓	IPsec-v2	D,N
Internet Key Exchange Option – If operating in a system requiring IKE support							
RFC 2407	The Internet IP Security Domain of Interpretation for ISAKMP	O	O	✓	✓	IKE-v1	J,D
RFC 2408	Internet Security Association and Key Management Protocol (ISAKMP)	O	O	✓	✓	IKE-v1	J,D
RFC 2409	Internet Key Exchange	O	O	✓	✓	IKE-v1	J,D
Explicit IP Labeling Option – If operating in a system requiring data sensitivity labels (RFC 2401, §8)							
RFC 5570	Common Architecture Label IPv6 Security Option (CALIPSO)	O	O		✓	LABEL	J,D
IETF CWG	Commercial IP Security Option (CIPSO 2.2)	O	O	✓		LABEL	J,D
IP Mobility Package Option – If operating as a Mobile End Node or Home Agent Router							
RFC 3775	Mobility Support in IPv6	O	O		✓	MIP-v3	D,N
RFC 3776	Using IPsec to Protect Mobile IPv6 Signaling between Mobile Nodes and Home Agents	O	O		✓	MIP-v3	D,N
RFC 4295	MIB for Mobile IP		O		✓	MIP-v3	D,N
IP Mobility – If operating as a Mobile Router (NEMO)							
RFC 3963	Network Mobility (NEMO) Basic Support Protocol	O	O		✓	NEMO	D,N
NETWORK APPLICATIONS							
DHCP Package Option – If operating in a system requiring DHCP support							
RFC 2131	Dynamic Host Configuration Protocol	O	O	✓		DHCP	J
RFC 2132	DHCP Options and BOOTP Vendor Extensions	O	O	✓		DHCP	J
DNS Package Option – If operating in a system requiring DNS support							
RFC 1034	Domain Names, Concepts, and Facilities	O	O	✓	✓	DNS	J

Standard	Description	Host	Router	IPv4	IPv6	Config Pkg	Source
RFC 1035	Domain Names, Implementations, and Specification	O	O	✓	✓	DNS	J
RFC 2136	Dynamic Updates in the Domain Name System (DNS UPDATE)	O	O	✓	✓	DNS	J
RFC 1886	DNS Extensions to Support IPv6	O	O		✓	DNS	J
Telnet Package Option – If operating as a system requiring Telnet support							
RFC 0854	Telnet Protocol Specifications	O	O	✓	✓	TELNET	J
RFC 0855	Telnet Option Specifications	O	O	✓	✓	TELNET	J
TFTP Package Option – If operating in a system requiring TFTP support							
RFC 1350	The TFTP Protocol (Revision 2)		O	✓	✓	TFTP	J
FTP Package Option – If operating as a system requiring FTP support							
RFC 0959	File Transfer Protocol	O		✓	✓	FTP	J
RFC 2428	FTP Extensions for IPv6 and NATs	O			✓	FTP	J,N
NTP Package Option – If operating as a system requiring NTP support							
RFC 1305	Network Time Protocol (Version 3) Specification	O		✓		NTP	J
LDAP Package Option – If operating in a system requiring LDAP support							
RFC 1777	Lightweight Directory Access Protocol	O		✓	✓	LDAP	J
HTTP Package Option – If operating in a system requiring HTTP support							
RFC 2616	Hypertext Transfer Protocol (HTTP) 1.1	O		✓	✓	HTTP	J
SNMP Package Option – If operating in a system requiring SNMP support							
RFC 1157	Simple Network Management Protocol (SNMP)		O	✓	✓	SNMP	J
RFC 1155	Structure and Identification of Management Information for TCP/IP-based Internets		O	✓	✓	SNMP	J
RFC 1212	Concise MIB Definitions		O	✓	✓	SNMP	J
RFC 1213	MIB for Network Management of TCP/IP-based Internets: MIB-II		O	✓	✓	SNMP	J
RFC 2790	Host Resources MIB		O	✓	✓	SNMP	J
RFC 2819	Remote Network Monitoring MIB		O	✓	✓	SNMP	J
RFC 2096	MIB for IP Forwarding Table		O		✓	SNMP	D,N

Standard	Description	Host	Router	IPv4	IPv6	Config Pkg	Source
RFC 3289	MIB for the Differentiated Services Architecture		O		✓	SNMP	D,N

Glossary

The following terms and phrases are defined both with respect to present day computer and software engineering as well as their specific meaning within the FACE Standard.

Aircraft Platform
: Represents an airframe that hosts mechanical, computing, and other resources necessary to perform a particular mission within the aviation domain.

Application
: A software executable designed to allow a user to complete tasks.

API
: An application programming interface (API) is a particular set of rules and specifications that a software program can follow to access and make use of the services and resources provided by another particular software program that implements that API. It serves as an interface between different software programs and facilitates their interaction, similar to the way the user interface facilitates interaction between humans and computers.

Application Framework
: A set of software deliverables providing a programming language-specific set of APIs which support application programming in a component paradigm and often provide services which are useful to a wide range of applications.

Application Characterization
: A quantified description of the “Unit of Portability” for the purpose of facilitating integration without exposing the internal design of the deliverable.

Build Time
: The time when source code written in a programming language is configured, compiled, and linked to form one or more machine-readable binary executables.

Capability
: A set of software deliverables that provides one or more mission-level facilities to the existing functionality of the current software suite.

CSCI
: Computer Software Configuration Item (see Software Configuration Item for details).

Compile Time
: The time where source code written in a programming language is analyzed and transformed into a machine-readable target language.

Centralized Configuration
: There exists a singular software component that manages the initialization parameters of the individual software components or software configuration items.

Computing Platform
: The combination of hardware and the software infrastructure (operating system, device drivers, middleware) that supports software components. Typically refers to

a processing hardware within a WRA, or LRU, along with its associated software infrastructure.

Configuration
: The selection of values or features of a component or system such that the intended operational characteristics are achieved.

Conformant
: With respect to the FACE Standard, this term applies when software components and/or a computing architecture, environment, or platform meets all of the stated requirements.

Data Movement
: Refers to the conveyance and/or transfer of data from one component to another.

Distributed Configuration
: Distributed configuration of the initialization parameters is the responsibility of the individual software components or software configuration items.

Dynamic Personalization
: Dynamic personalization occurs when the software and hardware setting are adjusted for a specific user during run-time.

Execution Time
: The time post-initialization where a software program is loaded in memory and in a running state (i.e., in the run-time state).

Framework
: See Application Framework.

FACE Application
: A component composed of one or more software components that may be bundled into a configuration item to deliver hardware and I/O-agnostic aircraft platform-level capabilities.

FACE-Conformant
: An implementation is considered FACE-conformant if it adheres to a subset of the FACE Standards without going outside of the FACE Standards.

FACE Computing Environment
: Includes all elements of the FACE Reference Architecture necessary to deploy FACE-conformant applications. The FACE Computing Environment is composed of the underlying computing and networking hardware, the software infrastructure (TS, OS, I/O Segments), the Platform-Specific Services Segment (PSS), and any common services required by the FACE applications. The FACE Computing Environment is a generic concept and instantiated for a particular system under development.

FACE Computing Platform
: A computing platform that hosts FACE applications (see Computing Platform).

FACE Computing Software Infrastructure
: Consists of the Operating System Segment, the Transport Services Segment, and the I/O Segment.

FACE Software Infrastructure
: See FACE Computing Software Infrastructure.

Initialization Time
: The time after software is loaded (i.e., within the run-time state) where the software is given specific values using initialization parameters to perform set-up and prepare for execution time activities.

I/O Service
: Refers to the Input and Output Services provided by a Computing Platform, specifically with respect to the movement of electronic data between the data bus and platform-specific service.

Language Run-time
: A set of software deliverables constituting a software layer which provides a software programming language API and the capability to execute programs written to that API.

Logical Ports
: The path over which data is moved between FACE architectural segments. This is in reference to system data addressability between partitions or computing platforms.

Monolithic Code Deliverable
: A singular bundle of highly inter-dependent software components characterized by a lack of modularity or that any single change to the code has vast ramifications that are reviewed and tested.

Memory Management
: The act of managing computer memory. Involves the allocation and de-allocation of memory resources during run-time based on application needs. This has the advantage of flexibility of operation, but it also has the disadvantage of potential inefficiencies and lack of predictability during operations. Since efficiency and predictability are extremely important aspects of most real-time systems, pre-allocation of computing resources such as memory, processes, threads, file descriptors, etc. is required, and is mandatory in the Safety and Security Profile. The preferred design approach to control these potential sources of non-determinism is through the use of pre-allocated resource pools. With modern languages and operating systems, it is possible to defer binding of many system resources until run-time.

Mission-level Capabilities
: Software adding high-level discernible value to the user's mission. An example is "situational awareness" which adds value to some missions. However, a bubble-sort is a supporting algorithm and while it may be a component of a mission capability, it is not the mission capability itself.

Partition — Originates from the ARINC 653 Standard, which discusses the allocation and isolation of computing platform resources on a partition basis (the POSIX Standard covers service interfaces only; it does not specify nor prohibit use of partitioning). A partition is an operating system allocation of computing platform and processor resources, including time. An executable, implementing an application or portion of an application, is associated with each partition. A partition provides a means to isolate resources to a particular application, preventing other applications from using or consuming them. In support of application independence, each partition has its own virtual address space for resources to be assigned within.

System designers use partitions to host multiple applications on a computing platform. By assigning separate time, memory, and other resources, applications may execute with sufficient independence that changes in one application cannot impact the execution of other applications running on the same computing platform.

Processor time is allocated by dividing a fixed-length period of time (referred to as the major-frame) into fixed-length time windows. One or more time windows is allocated to each partition. By selecting time windows that are harmonic to the major frame, the system designer can cause some partitions to execute at a faster rate relative to other partitions (e.g., 20 Hz, 10 Hz, 5 Hz). When the partition schedule is started, the operating system selects the first time window of the major frame. When this time window has completed, the next time window is selected. When the last time window of the major frame is complete, the operating system starts the processor-time allocation sequence over with the first time window of the major frame. This sequence is repeated continuously until the computing platform is power-downed, halted, or reset.

System designers may define partitions whose memory and other non-time resources are separately allocated, but whose time allocations are identical. When so designated, all threads associated with the applications assigned to the partitions will compete on a priority pre-emptive basis. This capability permits applications to be tightly coupled (e.g., client/server) in terms of time response (e.g., sever can respond immediately instead of waiting for a uniquely allocated time window to execute in). Note that applications allocated to partitions that share time resources are considered dependent applications (i.e., change in one application may directly impact the execution and time resources available to the other time-coupled applications), which prevents their qualification at different criticality levels.

Personalization — Configuration of software and hardware setting are adjusted for a specific user prior to run-time. For example, a pilot changing the zoom on a screen, powering it off, and then powering it back up.

Platform — Refers to one of two related things with respect to the FACE Standard: Aircraft (to include one or more computing platforms) and Computing (comprised of electronic circuitry and software).

Process — The term “process”, as discussed in this FACE Standard, originates from the POSIX Standard (in ARINC 653, the term “process” is equivalent to the POSIX

term “thread”). In concept, every partition has an associated process. Within a partition, a process represents basic capabilities for execution, including a thread schedulable by the operating system. Historically, processes, like partitions, are allocated resources and an address space by the operating system. Like partitions, multi-process support historically permitted multiple applications to execute on a computing platform. Within an application, threads provide support for multiple independent execution sequences. Control over the partition time allocations provides support for tightly-coupled applications (using shared time allocations) or loosely-coupled applications (using unique time allocations).

Point Solution
: A solution meant to solve a unique issue or fill a capability gap.

Portable
: Portability is an attribute that describes the re-use of existing code instead of creating new code when moving software from an environment to another. The pre-requirement for portability is the generalized abstraction between the application logic and system interfaces. When targeting several platforms with the same application, portability is the key issue for development cost reduction.

  Software is portable if it can easily be ported to multiple platforms. A FACE-conformant application shall be portable between FACE Computing Environments and is the main goal of FACE.

Programming Language Run-time
: See Language Run-time.

Run-time Application
: A software program that is only functioning and operational once a computing platform has completed the start-up sequence and is now ready for normal run-time operations.

Segment
: A logical grouping of applications and/or services within a boundary whereby elements within are allowed to vary based on system needs and the interface to elements outside the segment boundary adhere to the FACE Reference Architecture.

Service
: A software utility providing capability to software applications or other services.

Software Component
: A functionally or logically distinct part of a system, distinguished for the purpose of convenience in designing and specifying a complex system as an assembly of subordinate elements (ISO/IEC 24765:2010).

Software Configuration Item
: An aggregation of software designated for configuration management and treated as a single entity in the configuration management process (ISO/IEC 24765:2010). This entity satisfies an end use function and can be uniquely identified at a given reference point (ISO/IEC 12207:2008 §4.7).

Technical Data Package
: A consolidation of information used by designers and manufacturers describing a software product or component. A collection of all the information needed to integrate and deploy a specific software component or Unit of Portability.

Third Party
: An entity (business, person, or component) that is beyond the two primary interacting parties.

Thread
: A schedulable execution sequence within an application (the ARINC 653 Standard uses the term “process” when referring to thread behavior).

Traditional Transport Middleware
: A general description for a software component(s) that is responsible for transferring data between applications and the operating system.

UML
: Unified Modeling Language is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard was created and is managed by the Object Management Group. UML includes a set of graphic notation techniques to create visual models of object-oriented software-intensive systems.

XML
: Extensible Markup Language is a set of rules for encoding documents in machine-readable form. XML's design goals emphasize simplicity, generality, and usability over the Internet. It is widely used for the representation of arbitrary data structures; for example, in web services. This Standard was created and managed by the World Wide Web Consortium (W3C).

XSD
: Like all XML schema languages, XML Schema Definition can be used to express a set of rules to which an XML document conforms in order to be considered “valid” according to that schema. However, unlike most other schema languages, XSD was also designed with the intent that determination of a document's validity would produce a collection of information adhering to specific data types. This Standard was created and managed by the World Wide Web Consortium (W3C).

Annex A: Conformance Verification Matrix (CVM) Notes

Note #	CVM Line #(s)	Standard Section #(s)	Verification Note
Note 1	14, 22, 23, 25, 26, 53, 55, 56, 62, 73, 74, 78, 120, 121, 135, 149, 159, 164, 167, 173, 1057, 1065, 1066, 1068, 1071, 1080, 1126, 1127, 1129, 1138, 1139, 1140, 1141, 1164, 1165	§1.2: Para 4 §1.3: Para 3,4,6,7 §2.2.1.1: Para 3,5,6 §2.2.2: Para 6,7 §2.2.2.1: Para 1 §2.5: Para 1,2 §2.5.1.2: Para 1 §2.5.1.3: Para 1 §2.5.2: Para 1 §2.6: Para 1 §2.7: Para 2,5 §2.7.3: Para 1 §4.2.1: Para 1,3, Bullet 2,3 §4.2.1, Para 4 §4.2.2: Para 2,4 §5.2.1: Para 1,2 §5.2.2: Para 1,5,6,7,8 §6: Para 3,4	Sections 1, 2, 4, 5, and 6 were not intended to be normative sections and the use of “must”, “will”, and “shall” is removed in Edition 2.0. This paragraph is no longer a normative requirement in Edition 1.0.
Note 2	98, 163, 166	§2.4: Para 4 §2.7: Para 1,4	Sections 1, 2, 4, 5, and 6 were not intended to be normative sections and the use of “must”, “will”, and “shall” is removed in Edition 2.0. This paragraph is no longer a normative requirement in Edition 1.0. The requirements are defined in Section 3.9 and Appendix A.

Note #	CVM Line #(s)	Standard Section #(s)	Verification Note
Note 3	139	§2.5.1.3: Para 1	Sections 1, 2, 4, 5, and 6 were not intended to be normative sections and the use of “must”, “will”, and “shall” is removed in Edition 2.0. This paragraph is no longer a normative requirement in Edition 1.0. The requirements are defined in Section 3.5.1.
Note 4	169	§2.7.1: Para 1	Sections 1, 2, 4, 5, and 6 were not intended to be normative sections and the use of “must”, “will”, and “shall” is removed in Edition 2.0. This paragraph is no longer a normative requirement in Edition 1.0. The requirements are defined in Section 3.9.1.
Note 5	171	§2.7.2: Para 1	Sections 1, 2, 4, 5, and 6 were not intended to be normative sections and the use of “must”, “will”, and “shall” is removed in Edition 2.0. This paragraph is no longer a normative requirement in Edition 1.0. The requirements are defined in Section 3.9.2.
Note 6	187, 232, 271, 306, 348, 475, 476, 477, 478, 573, 710, 844, 853, 857, 997, 998, 999, 1196	§3.1: Para 3 §3.3: Para 1 §3.4: Para 1 §3.5: Para 1 §3.6: Para 1 §3.7.1: Para 3,4,5,6 §3.8.1: Para 3, Bullet 6 §3.8.13: Para 1 §3.10: Para 1 §3.11: Para 1 §3.11.1: Para 1 §3.13.1: Para 1, Bullet 2,3 §B.1: Para 1	Each section has an overview introduction that was not intended to be normative and requirements are outlined later in this section for Edition 1.0. This paragraph is reworded to be non-normative in Edition 2.0. This paragraph is no longer a normative requirement in Edition 1.0.

Note #	CVM Line #(s)	Standard Section #(s)	Verification Note
Note 7	212	§3.1.1: Para 1, Bullet 5(b)	This is a requirement on the standard to select APIs that support this. See Appendix A. This statement is no longer a normative requirement in Edition 1.0.
Note 8	227, 840	§3.1.1: Para 1, Bullet 6(f) §3.9.4: Para 1	This paragraph is reworded to be non-normative in Edition 2.0. This paragraph is no longer a normative requirement in Edition 1.0.
Note 9	230, 301, 316, 317, 321, 323, 325, 328, 335, 336, 341, 622, 638, 659, 708, 768, 769, 770, 838, 882, 883, 957, 958, 962, 978, 1008, 1293, 1303, 1314, 1355, 1392, 1494, 1497, 1503, 1539, 1544, 1547, 1553, 1559, 1568, 1577, 1584, 1620	§3.2: Para 2 §3.4.2, Bullet 4 §3.5.1: Para 1,2 §3.5.2: Para 1 §3.5.3: Para 1 §3.5.4: Para 1 §3.5.5: Para 2 §3.5.6: Para 1, Bullet 4(a), 8(a) §3.8.4.1: Para 7, Bullet 3 §3.8.4.3, Para 2 §3.8.5: Para 2 §3.8.12: Para 1 §3.9.1: Para 5, Bullet 1,2,3 §3.9.3: Para 4 §3.11.1.2: Para 2, Table Row 2,3, Column 1 §3.12.1: Para 1, Bullet 1(f),1(f(i)) §3.12.1: Para 1, Bullet 4 §3.12.2: Bullet 1 §3.13.2: Para 1, Bullet 5 §C.5: Para 2,3,4,10,11 §C.7: Para 1,2,5,9 §C.8: Para 1,2,5,5 (Code),6 (Code),9 §C.10: Para 1	Conformance verification is not intended for this Edition 1.0 requirement. This requirement is removed in Edition 2.0. This paragraph is no longer a normative requirement in Edition 1.0.

Note #	CVM Line #(s)	Standard Section #(s)	Verification Note
Note 10	238	§3.3.1: Para 1, Bullet 2	This is an introductory statement for requirements which are broken out in Sections 3.3.3, 3.3.4, and 3.3.5. This statement is no longer a normative requirement in Edition 1.0.
Note 11	262	§3.3.3.2: Bullet 4	This is a duplicate requirement and is removed in Edition 2.0. It is a duplicate of Section 3.3.2, item 4. This statement is no longer a normative requirement in Edition 1.0.
Note 12	281	§3.4.1: Para 3	Conformance verification is not intended for this Edition 1.0 requirement. This requirement is removed in Edition 2.0. Section 3.4.2 items 2 and 3 more succinctly state the only conformance requirements contained in this statement. The remainder is commentary and not normative.
Note 13	330	§3.5.6: Para 1	This is an introductory statement to the following requirements and the “shall” is removed in Edition 2.0. This paragraph is no longer a normative requirement in Edition 1.0.
Note 14	350, 352, 355, 358, 361, 364, 720	§3.6.1: Para 1 §3.6.2: Bullet 1,2,3,4,5 §3.9: Para 2	Conformance verification is not intended for this Edition 1.0 requirement. This statement is a requirement on the API rather than the software. This requirement is removed in Edition 2.0. This paragraph is no longer a normative requirement in Edition 1.0.

Note #	CVM Line #(s)	Standard Section #(s)	Verification Note
Note 15	26, 494	§3.7.3: Bullet 3	This is a hierarchical requirement that is more accurately captured in items 4 and 5 below. This bullet is no longer a normative requirement in Edition 1.0.
Note 16	621, 669, 964	§3.8.4.1: Para 7, Bullet 2 §3.8.6: Table Row 2, Column 2 §3.12.1: Para 1, Bullet 6	This is a duplicate requirement and is removed in Edition 2.0. This statement is no longer a normative requirement in Edition 1.0.
Note 17	665, 666	§3.8.6: Table Row 1, Column 2, Para 1, Bullet 1,2	This is not intended to be a conformance requirement. This statement is no longer a normative requirement in Edition 1.0.
Note 18	692	§3.8.11.1: Para 1, Bullet 1(a)	This is not a normative statement. It is only providing examples of UoPs.
Note 19	751	§3.9.1: Para 3	The specific details of this requirement are defined in Appendix A. This wording is clarified in Edition 2.0. This statement is no longer a normative requirement in Edition 1.0.
Note 20	766	§3.9.1: Para 4	Conformance verification is not intended for this Edition 1.0 requirement. The second statement is a valid requirement for Edition 1.0. This requirement is removed in Edition 2.0.

Note #	CVM Line #(s)	Standard Section #(s)	Verification Note
Note 21	767	§3.9.1: Para 5	This is an introductory statement and the “shall” applies to the following bullet points making them the actual requirement. Conformance verification is not intended for this Edition 1.0 requirement. This is a requirement on the standard. This requirement is removed in Edition 2.0. This statement is no longer a normative requirement in Edition 1.0.
Note 22	780, 781, 782, 783, 784, 785, 786, 787, 788, 789, 790, 791, 792, 793, 794, 799, 800, 801, 802, 803, 804, 805, 806, 807, 808, 809, 810, 811, 812, 813, 814, 815, 816, 817, 818, 1171, 1172, 1173, 1174, 1175, 1176, 1177, 1178, 1179, 1180, 1181, 1182, 1183, 1184, 1185, 1186, 1187, 1188, 1189, 1190, 1191, 1192, 1193	§3.9.2, Para 4 §3.9.2: Para 4, Bullet 1,1(a),2,2(a,b),2(b(i-iii)) §3.9.2, Para 5 §3.9.2: Para 5, Bullet 1-5 §3.9.2, Para 8 §3.9.2: Para 8, Bullet 1,1(a-o),2,2(a),3 §A.1, Para 2 §A.1: Para 2, Bullet 1-22	Conformance verification is not intended for this Edition 1.0 requirement. This is a requirement on the standard.
Note 23	831	§3.9.3: Para 3	Conformance verification is not intended for this Edition 1.0 requirement. This statement is a requirement on the API definition rather than the implementation of that API. This requirement is removed in Edition 2.0. This statement is no longer a normative requirement in Edition 1.0.
Note 24	841	§3.9.4: Para 2	The requirement is defined in Sections 3.9.1, 3.9.2, and 3.9.3. This statement is no longer a normative requirement in Edition 1.0.

Note #	CVM Line #(s)	Standard Section #(s)	Verification Note
Note 25	913	§3.11.4: Para 3	The specific requirements are defined in Section 3.5.6 and Appendix C. This statement is no longer a normative requirement in Edition 1.0.
Note 26	1358, 1362, 1373, 1377, 1381, 1541, 1586	§C.5: Para 10 (Code) §C.7: Para 9 (Code) §C.8: Para 9 (Code)	Conformance verification is not intended for this Edition 1.0 requirement. Requirement language is not normative when embedded in a note inside a code remark. This requirement is removed in Edition 2.0. This note is no longer a normative requirement in Edition 1.0.